Altera OpenCL Tutorial
Compiling OpenCL to a Streaming Dataflow Architecture
CGO 2015

Desh Singh & Doris Chen

Altera Toronto Technology Center



© 2015 Altera Corporation—Public

FPGA Basics for Software Programmers

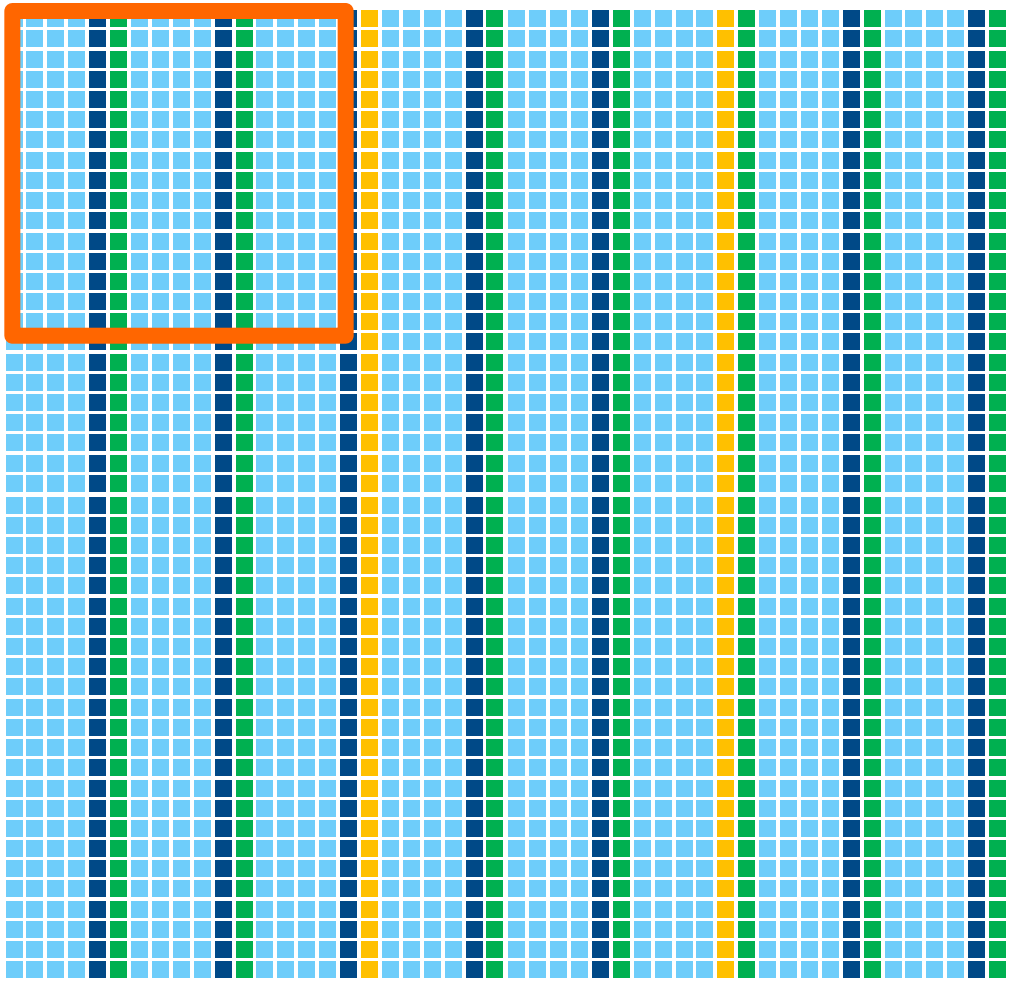


© 2015 Altera Corporation—**Public**

FPGA architecture: fine-grained massive parallelism

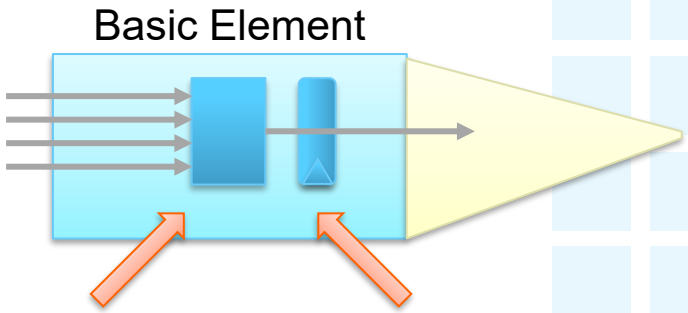
Let's zoom in

Over 1 Million
Basic Elements!



© 2015 Altera Corporation—**Public**

FPGA architecture: fine-grained massive parallelism

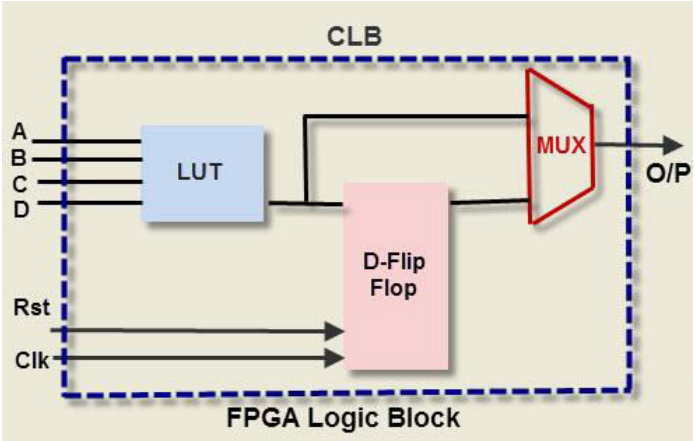


1-bit configurable operation

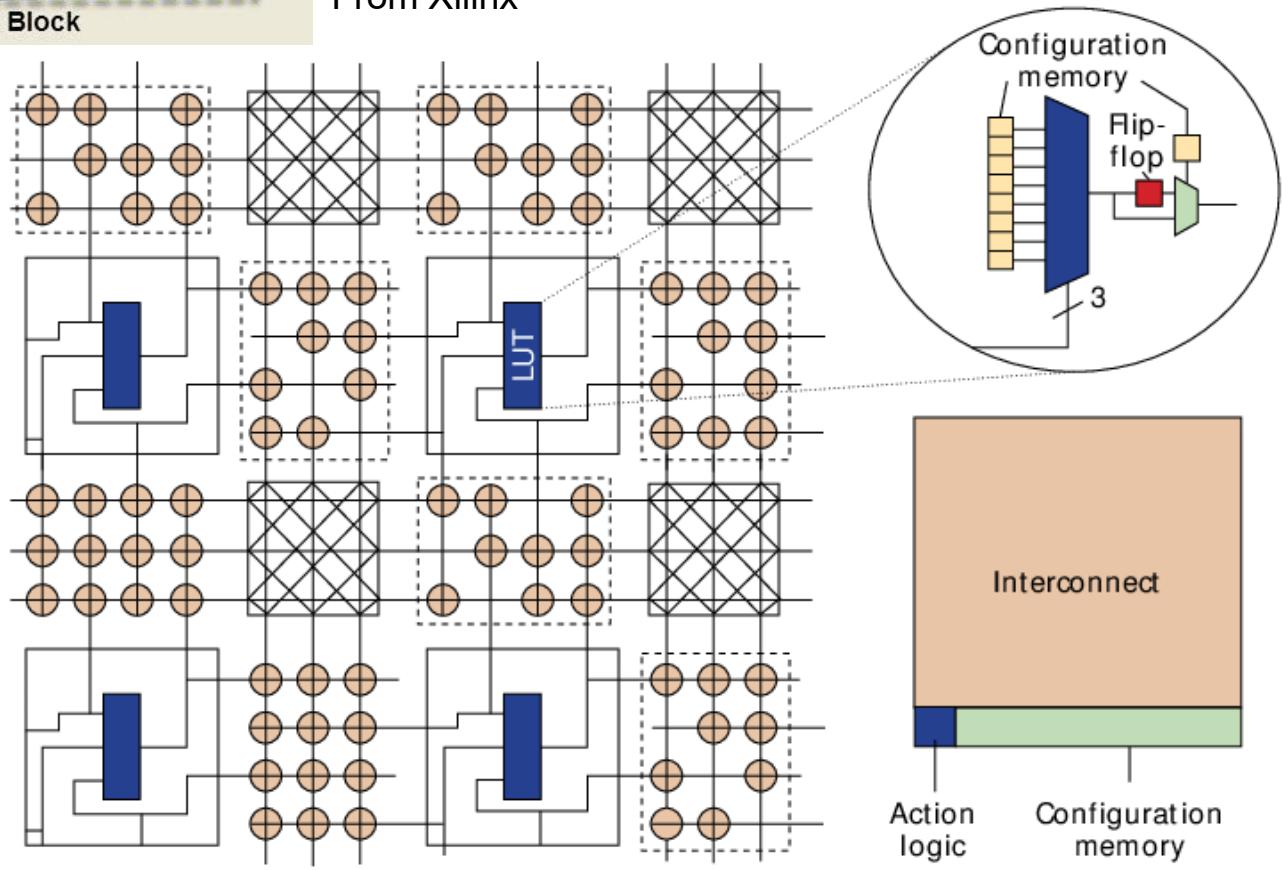
1-bit register (store result)

***Configured** to perform any 1-bit operation:*
AND, OR, NOT, ADD, SUB

© 2015 Altera Corporation—**Public**



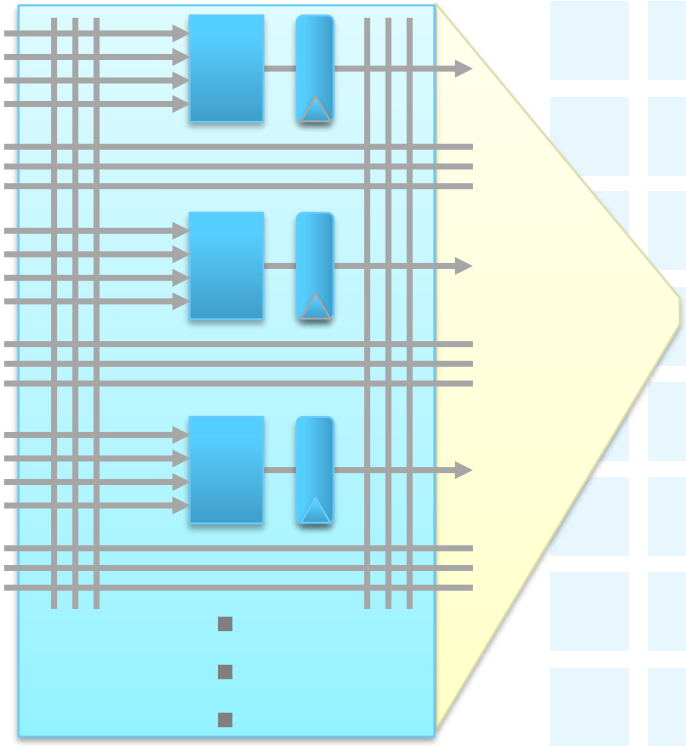
From Xilinx



Francesco Renzini, et al, 2019

© 2015 Altera Corporation—Public

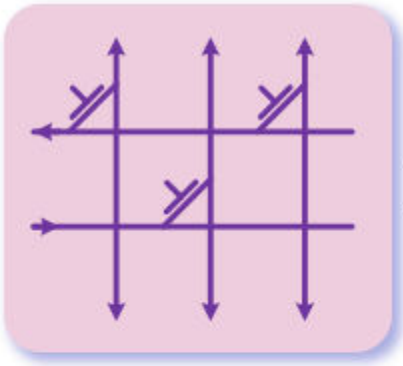
FPGA architecture: flexible interconnect

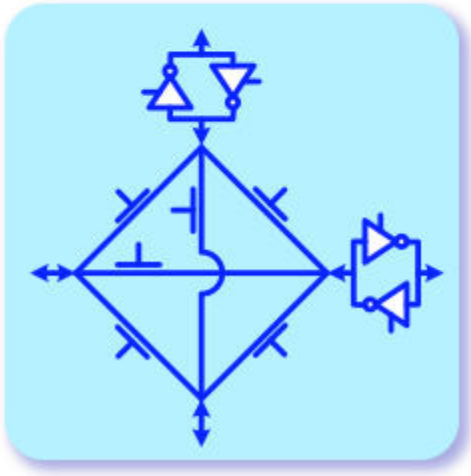


Basic Elements are surrounded with a flexible interconnect

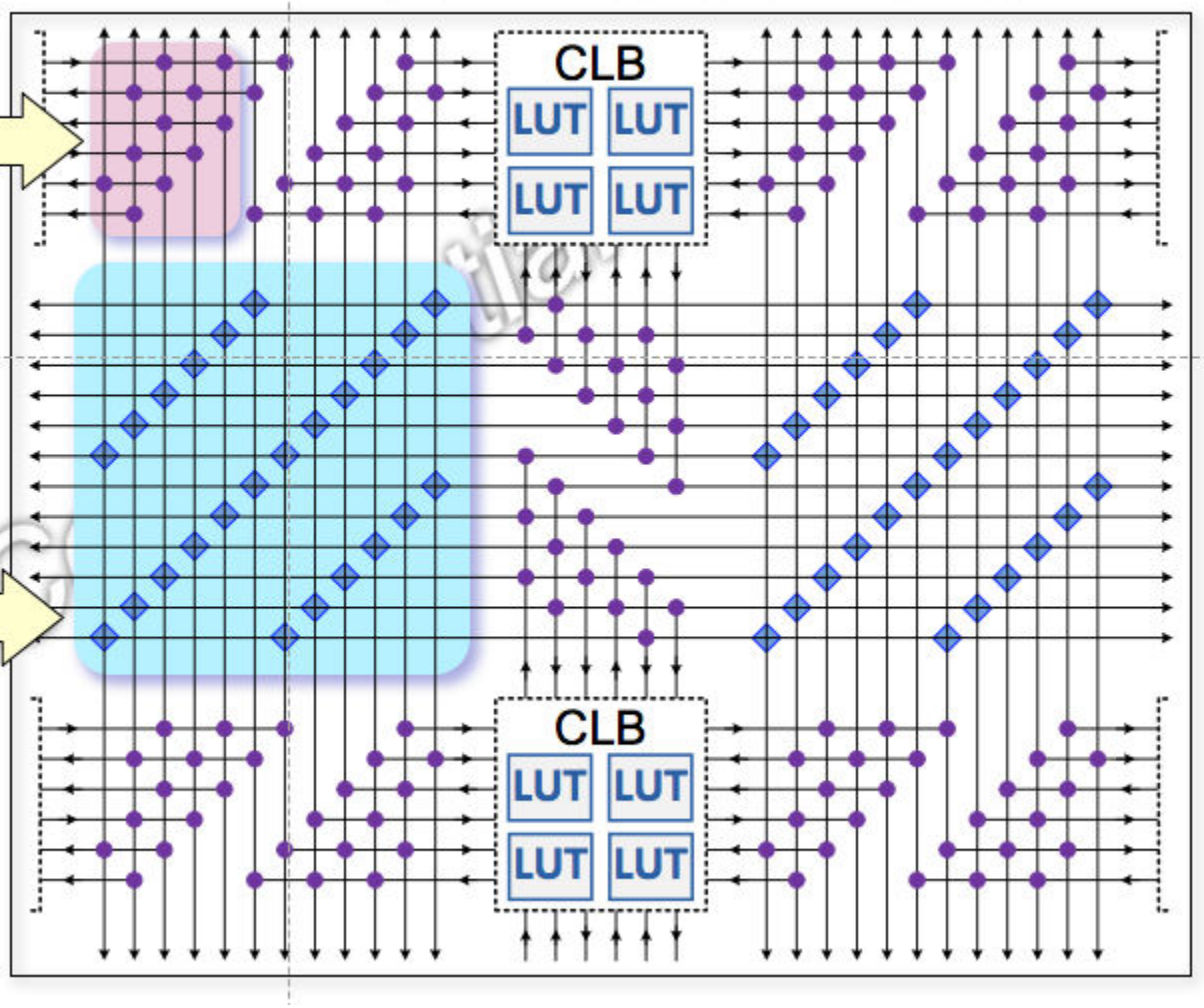
© 2015 Altera Corporation—**Public**

2D-Mesh Interconnects





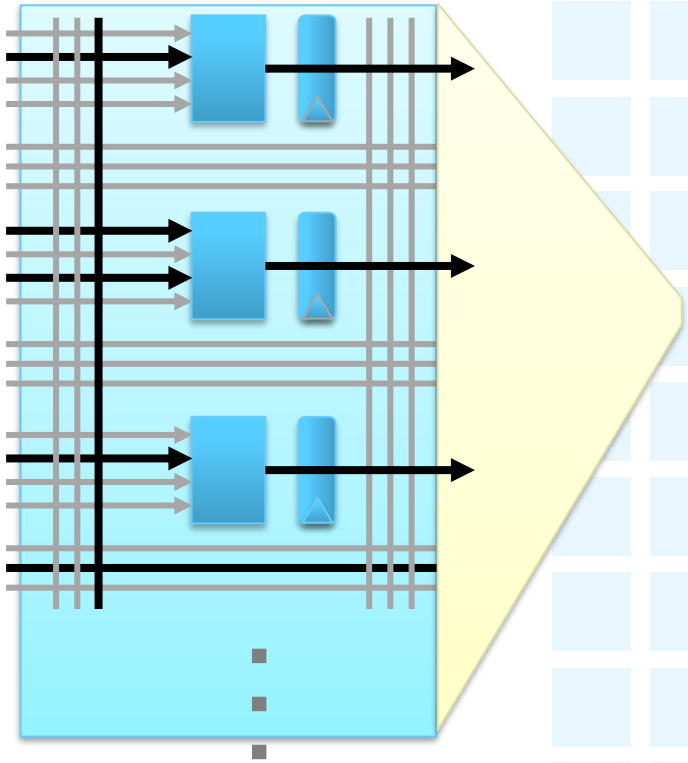
O(N²) worst-case complexity



From Flex Logix, https://semiengineering.com/new-interconnect-makes-efpga-dense-and-portable/

© 2015 Altera Corporation—**Public**

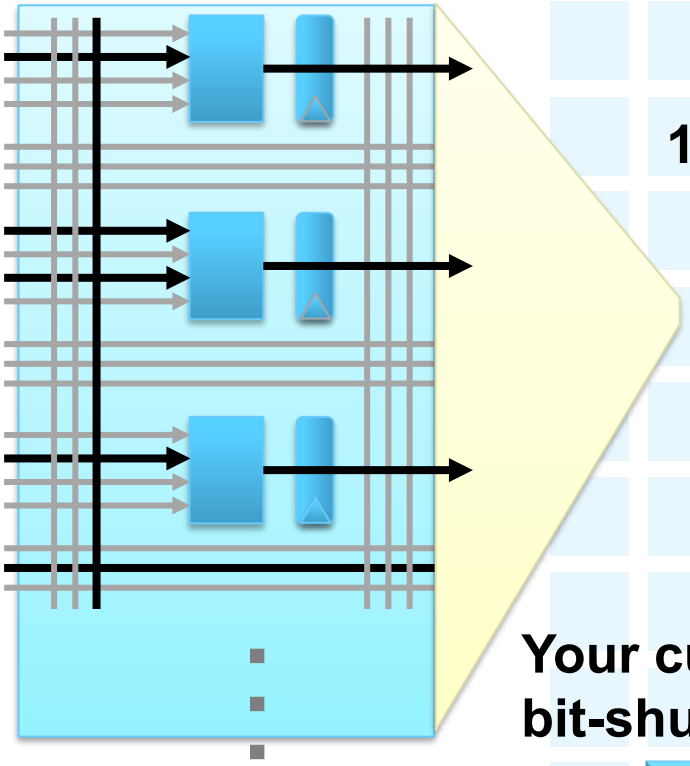
FPGA architecture: flexible interconnect



Wider *custom* operations are implemented by configuring and interconnecting Basic Elements

© 2015 Altera Corporation—**Public**

FPGA architecture: custom operations using basic elements



16-bit add

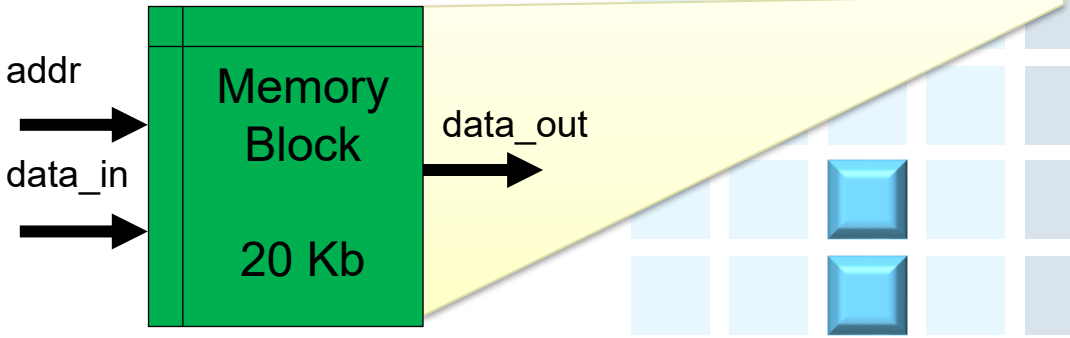
32-bit sqrt

Your custom 64-bit
bit-shuffle and encode

Wider *custom* operations are implemented by configuring and interconnecting Basic Elements

© 2015 Altera Corporation—Public

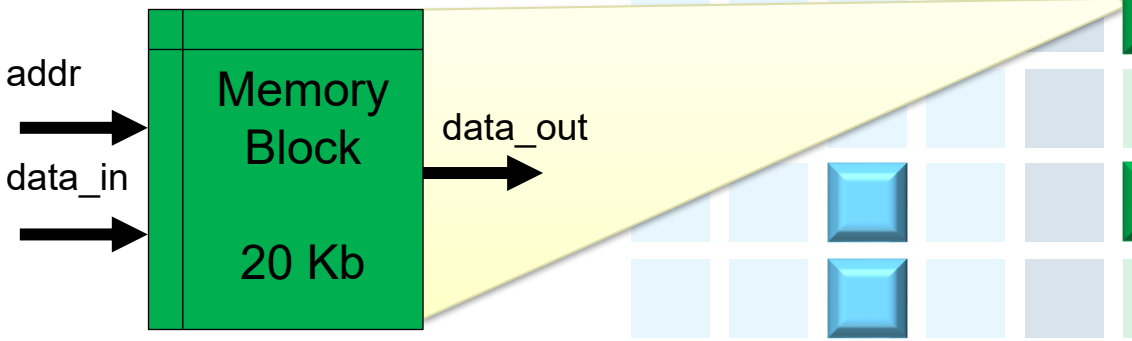
FPGA architecture: memory blocks



Can be configured and grouped using the interconnect to create various cache architectures

© 2015 Altera Corporation—Public

FPGA architecture: memory blocks



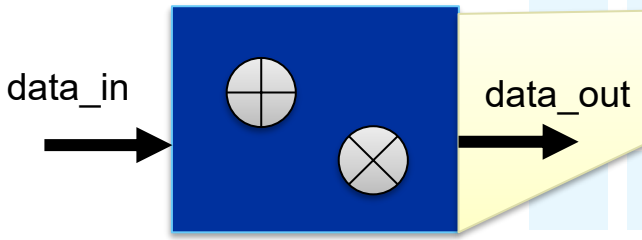
Can be configured and grouped using the interconnect to create various cache architectures

Lots of smaller caches

Few larger caches

© 2015 Altera Corporation—**Public**

FPGA architecture: floating point multiplier/adder blocks

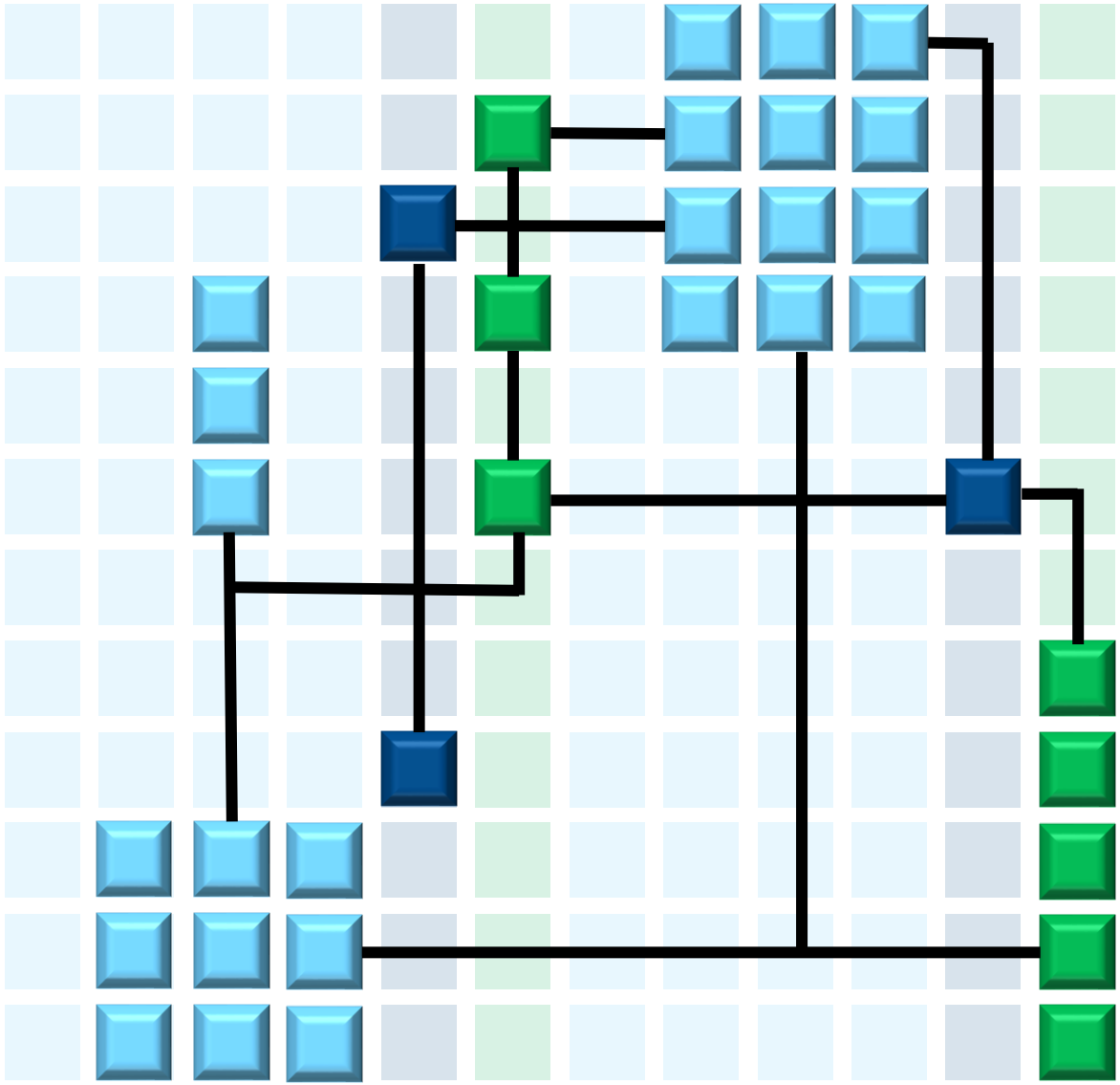


Dedicated floating point multiply and add blocks

© 2015 Altera Corporation—**Public**

FPGA architecture

*Blocks are connected into a **custom data-path** that matches your application.*



© 2015 Altera Corporation—**Public**

Mapping a simple program to an FPGA

High-level code

```
Mem[100] += 42 * Mem[101]
```

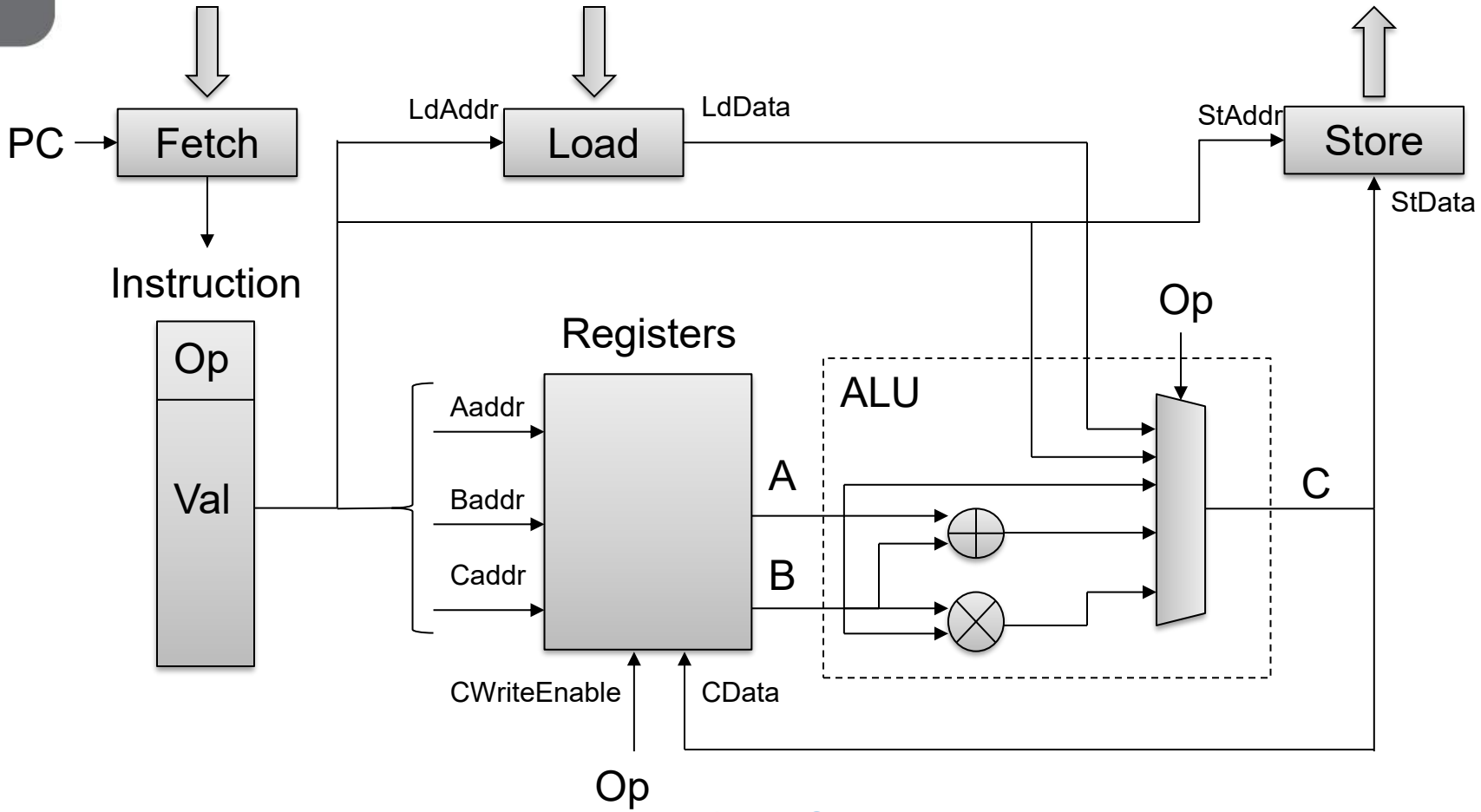


CPU instructions

```
R0 ← Load Mem[100]
R1 ← Load Mem[101]
R2 ← Load #42
R2 ← Mul R1, R2
R0 ← Add R2, R0
Store R0 → Mem[100]
```


© 2015 Altera Corporation—Public

First let's take a look at execution on a *simple* CPU

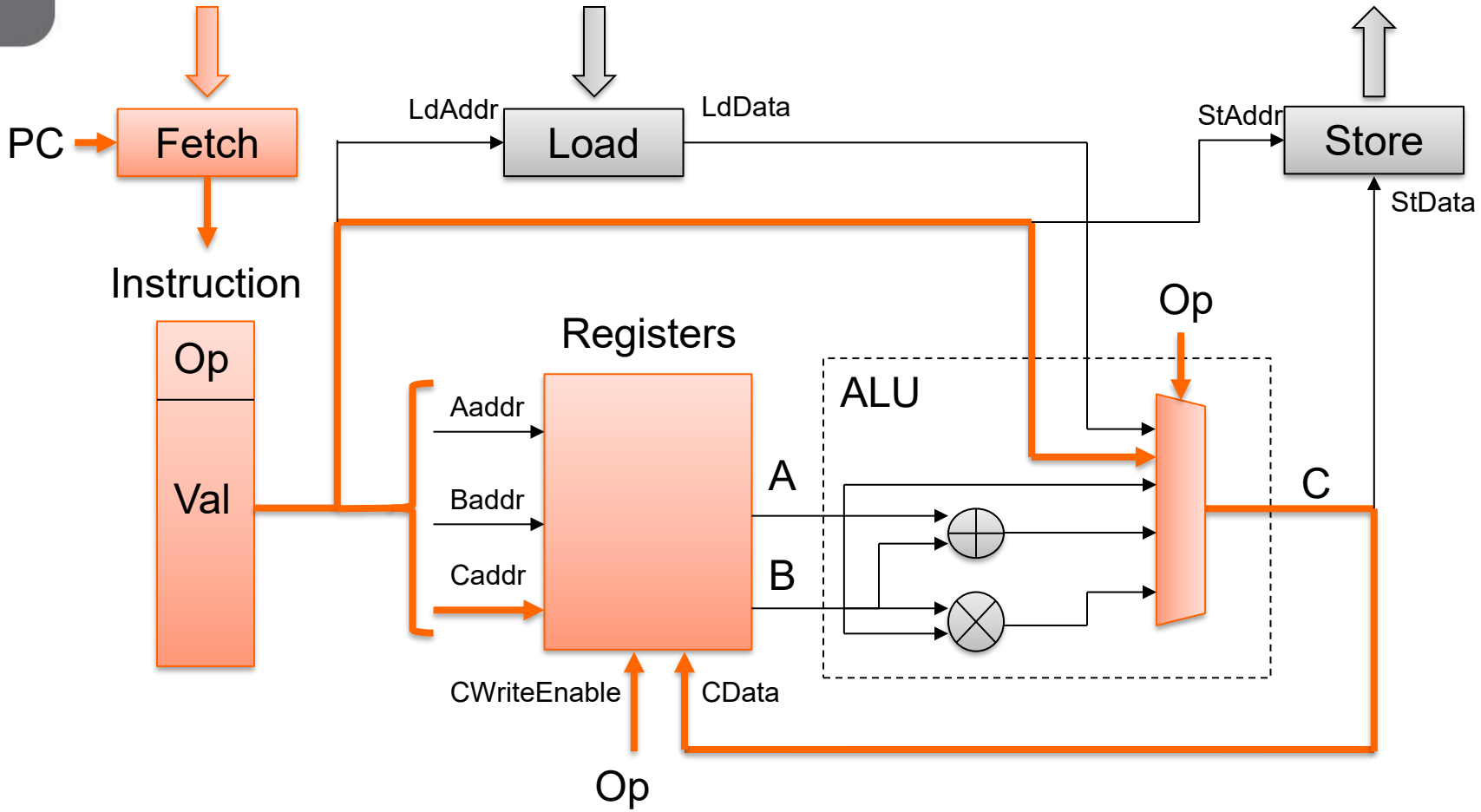


Fixed and general architecture:

- **General "cover-all-cases" data-paths**
- **Fixed data-widths**
- **Fixed operations**

© 2015 Altera Corporation—Public

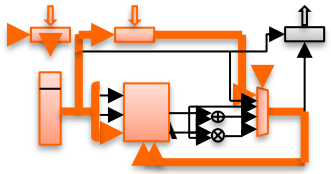
Load constant value into register



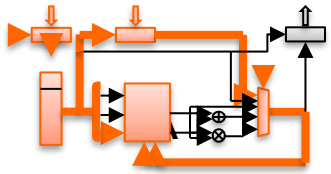
© 2015 Altera Corporation—Public

CPU activity, step by step

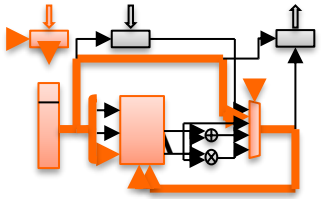
R0 ← Load Mem[100]



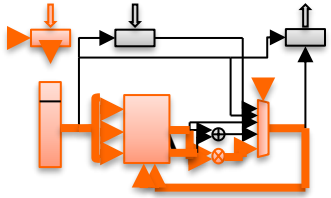
R1 ← Load Mem[101]



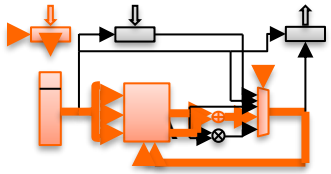
R2 ← Load #42



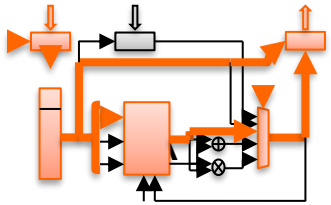
R2 ← Mul R1, R2



R0 ← Add R2, R0



Store R0 → Mem[100]



Time



© 2015 Altera Corporation—Public

On the FPGA we unroll the CPU hardware…

R0 ← Load Mem[100]

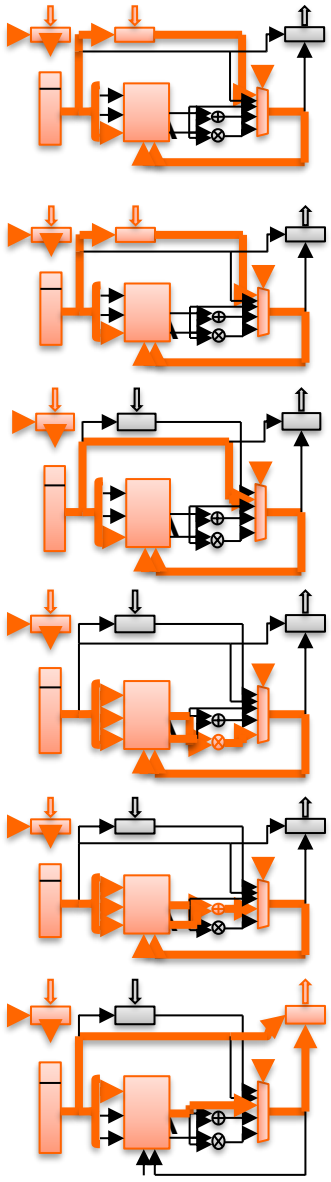
R1 ← Load Mem[101]

R2 ← Load #42

R2 ← Mul R1, R2

R0 ← Add R2, R0

Store R0 → Mem[100]



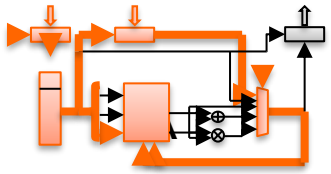
Space



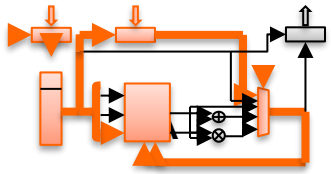
© 2015 Altera Corporation—Public

… and specialize by position

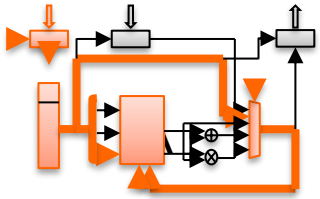
R0 ← Load Mem[100]



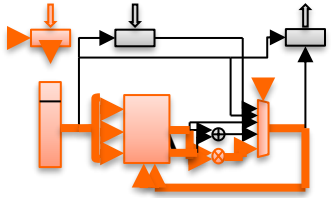
R1 ← Load Mem[101]



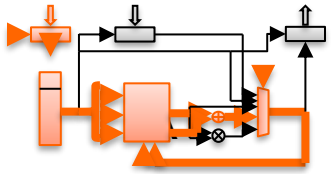
R2 ← Load #42



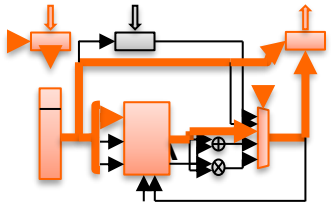
R2 ← Mul R1, R2



R0 ← Add R2, R0



Store R0 → Mem[100]



1. Instructions are fixed. Remove “Fetch”

© 2015 Altera Corporation—Public

… and specialize

R0 ← Load Mem[100]

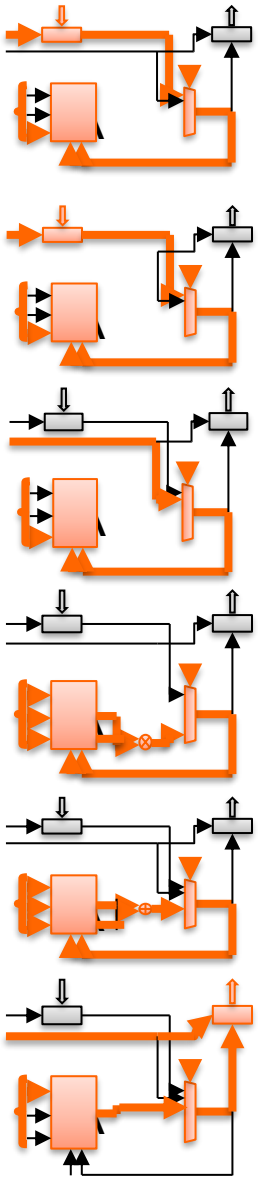
R1 ← Load Mem[101]

R2 ← Load #42

R2 ← Mul R1, R2

R0 ← Add R2, R0

Store R0 → Mem[100]

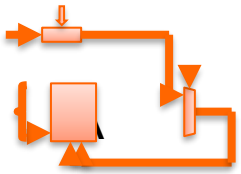


1. Instructions are fixed. Remove “Fetch”
2. Remove unused ALU ops

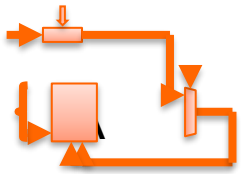
© 2015 Altera Corporation—Public

… and specialize

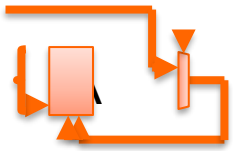
R0 ← Load Mem[100]



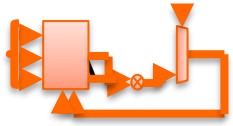
R1 ← Load Mem[101]



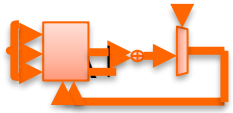
R2 ← Load #42



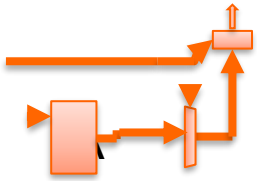
R2 ← Mul R1, R2



R0 ← Add R2, R0



Store R0 → Mem[100]



1. Instructions are fixed. Remove “Fetch”
2. Remove unused ALU ops
3. Remove unused Load / Store

© 2015 Altera Corporation—Public

… and specialize

R0 ← Load Mem[100]

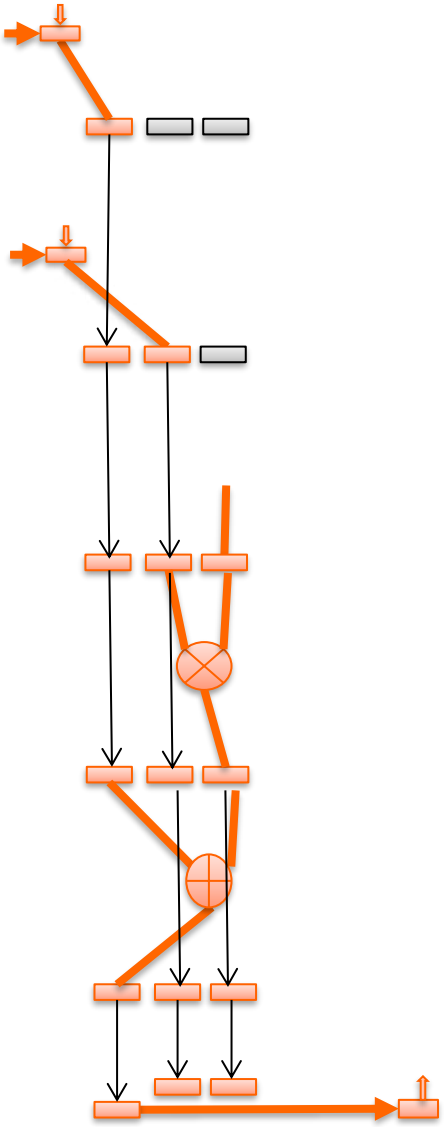
R1 ← Load Mem[101]

R2 ← Load #42

R2 ← Mul R1, R2

R0 ← Add R2, R0

Store R0 → Mem[100]



1. Instructions are fixed.
   Remove “Fetch”
2. Remove unused ALU ops
3. Remove unused Load / Store
4. Wire up registers properly!
   And propagate state.

© 2015 Altera Corporation—Public

… and specialize

R0 ← Load Mem[100]

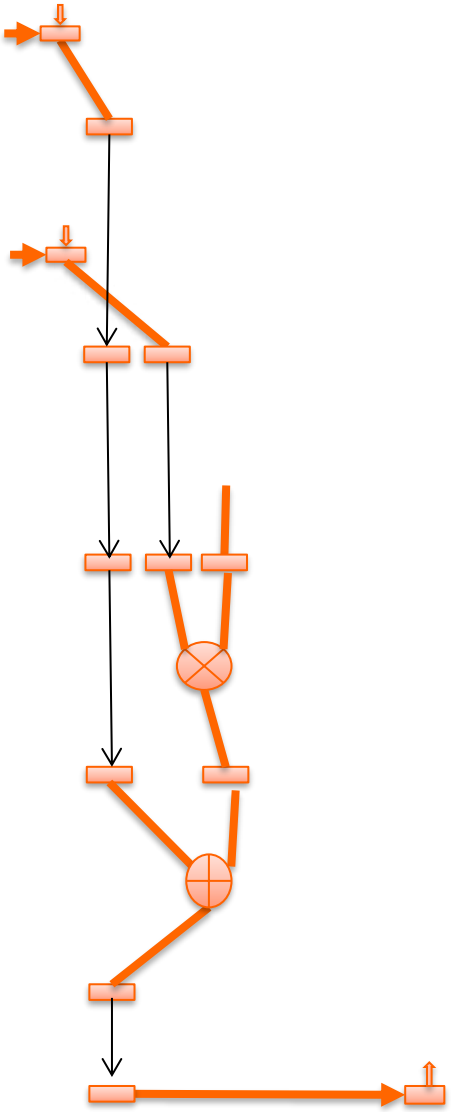
R1 ← Load Mem[101]

R2 ← Load #42

R2 ← Mul R1, R2

R0 ← Add R2, R0

Store R0 → Mem[100]



1. Instructions are fixed.
   Remove “Fetch”
2. Remove unused ALU ops
3. Remove unused Load / Store
4. Wire up registers properly!
   And propagate state.
5. Remove dead data.

© 2015 Altera Corporation—Public

… and specialize

R0 ← Load Mem[100]

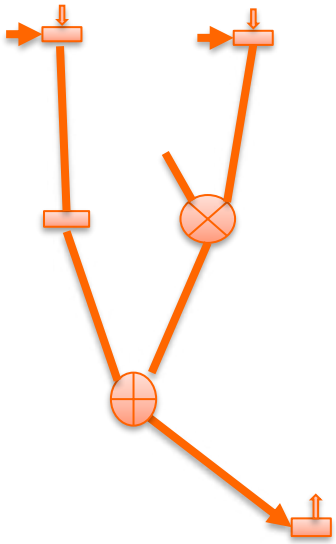
R1 ← Load Mem[101]

R2 ← Load #42

R2 ← Mul R1, R2

R0 ← Add R2, R0

Store R0 → Mem[100]



1. Instructions are fixed.
   Remove “Fetch”
2. Remove unused ALU ops
3. Remove unused Load / Store
4. Wire up registers properly!
   And propagate state.
5. Remove dead data.
6. Reschedule!

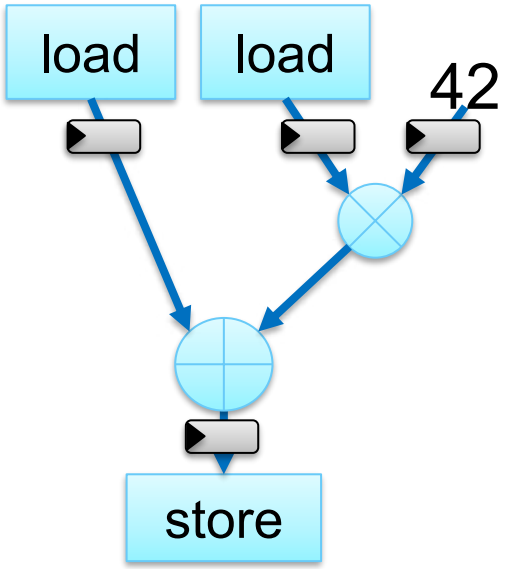
© 2015 Altera Corporation—Public

Custom data-path on the FPGA matches your algorithm!

High-level code

```
Mem[100] += 42 * Mem[101]
```

Custom data-path



Build exactly what you need:

Operations

Data widths

Memory size & configuration

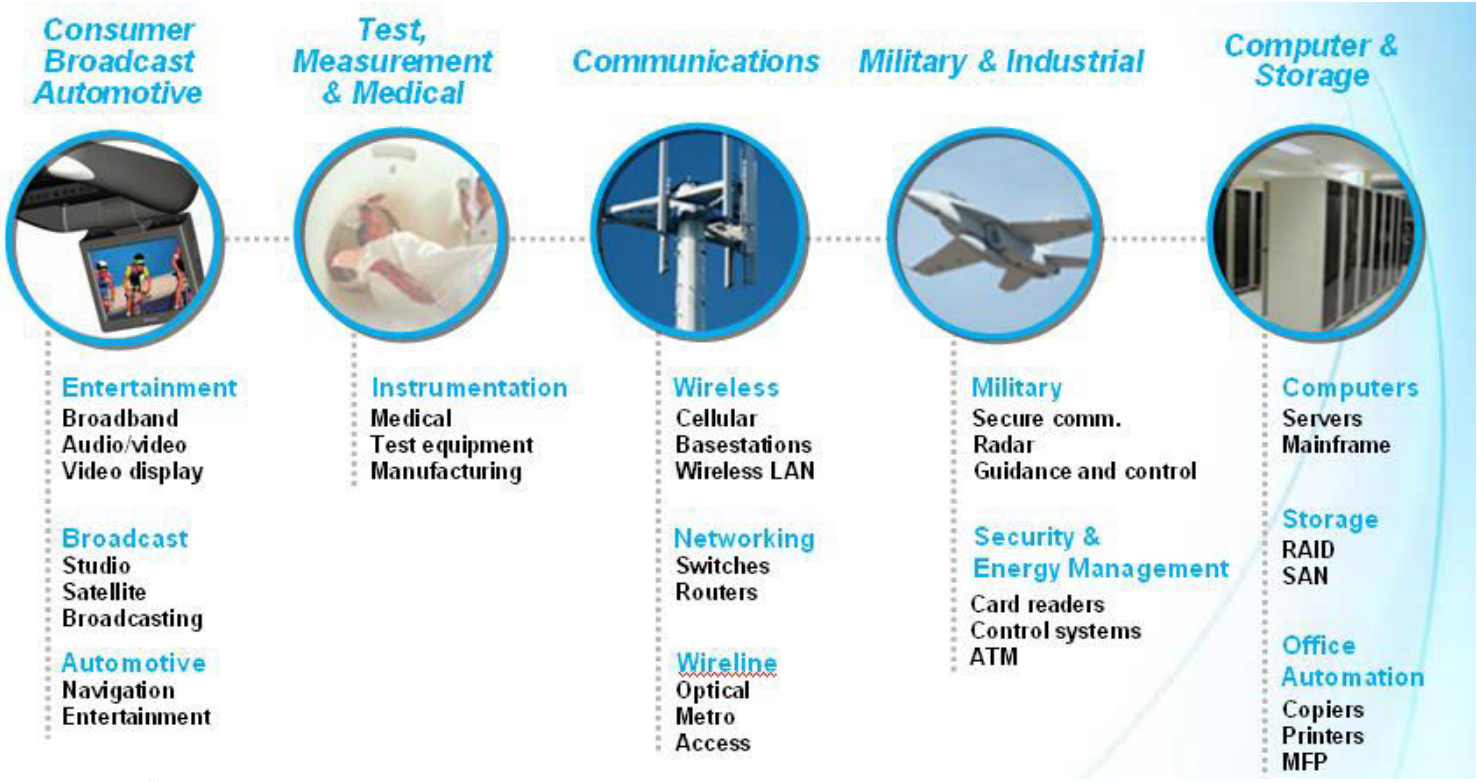
Efficiency:

Throughput / Latency / Power

© 2015 Altera Corporation—Public

Traditional use of FPGAs

- Used by hardware designers only
- Required knowledge of
  - Hardware expertise and hardware description language (HDL)
  - FPGA CAD tools
- Design time from several months to several years!

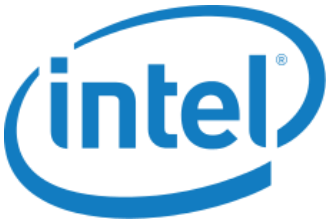


FPGAs entering HPC and data centers

Heterogeneous architectures with FPGAs announced in 2014



"Catapult fabric:
FPGAs in data center servers
accelerate search rankings"



"Xeon processor with a coherent
FPGA in a single package, pin
compatible"



"Data analytics: FPGAs used for
compression and filtering"
"FPGA-accelerated POWER Systems with
Coherent Shared Memory"



"FPGA-accelerated neural network
algorithms for image classification
and recognition in search engines"

© 2015 Altera Corporation—Public

OpenCL: a standard for programming heterogeneous architectures



© 2015 Altera Corporation—Public

OpenCL: Open Computing Language

- Open, industry standard programming model
- Aimed for parallel programming of heterogeneous systems
  - CPUs, GPUs, FPGAs, DSPs and other accelerators
- Ensures functional portability across platforms and vendors
- Specification developed by the Khronos Group consortium
  - Originally developed by Apple

KHRONOS GROUP



© 2015 Altera Corporation—Public

OpenCL: industry-wide adoption

- Adopters must pass conformance tests
  - Over 8000 tests
  - *Altera: the first FPGA vendor to pass conformance*

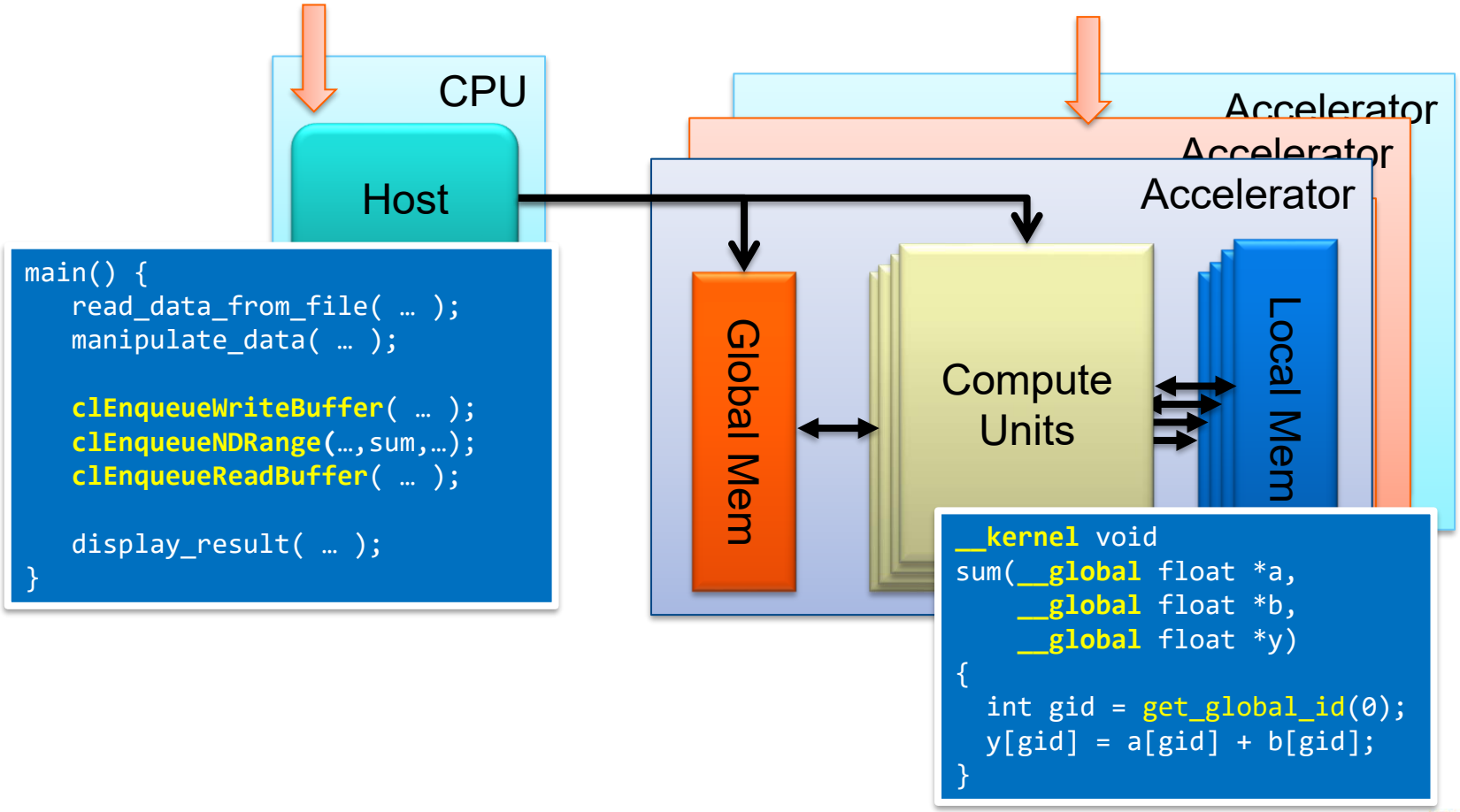


© 2015 Altera Corporation—Public

OpenCL Programming Model: Host + Accelerators

Sequential host program on CPU

Parallel functions (kernels) are offloaded onto highly parallel accelerator devices




```
main() {
   read_data_from_file( … );
   manipulate_data( … );

   clEnqueueWriteBuffer( … );
   clEnqueueNDRange(…,sum,…);
   clEnqueueReadBuffer( … );

   display_result( … );
}
```

```
__kernel void
sum(__global float *a,
    __global float *b,
    __global float *y)
{
  int gid = get_global_id(0);
  y[gid] = a[gid] + b[gid];
}
```


© 2015 Altera Corporation—Public

OpenCL Host Program

- Pure software written in standard ‘C’
- Communicates with the *accelerator devices* via an API which abstracts the communication between the host processor and the kernels

Copy data from Host to FPGA

Tell the FPGA to run a particular kernel

Copy data from FPGA to Host

```
main()
{
    read_data_from_file( … );
    manipulate_data( … );

    clEnqueueWriteBuffer( … );
    clEnqueueNDRange(…, sum, …);
    clEnqueueReadBuffer( … );

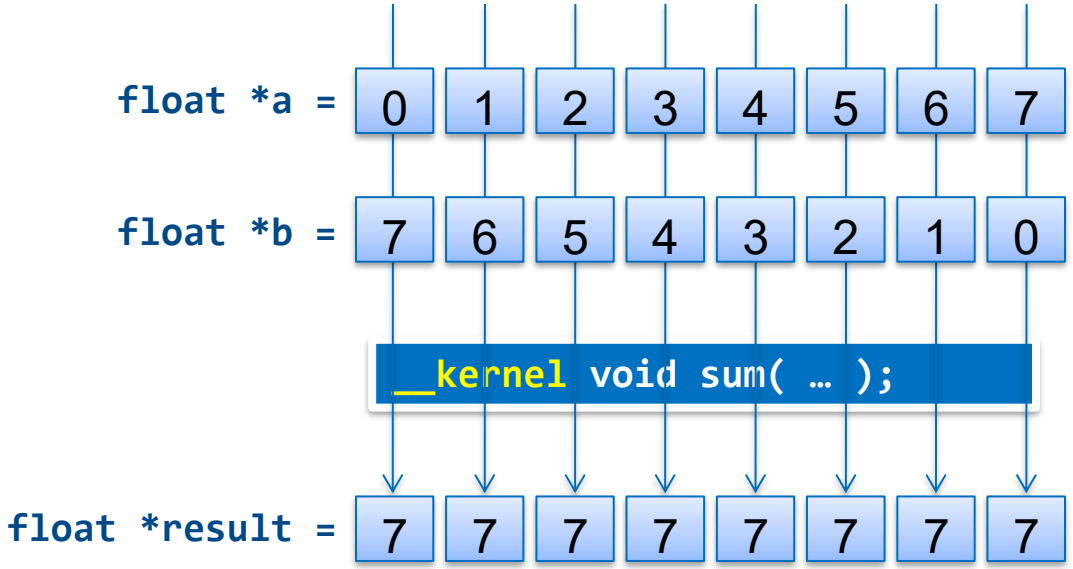
    display_result ( … );
}
```


© 2015 Altera Corporation—Public

OpenCL Kernels

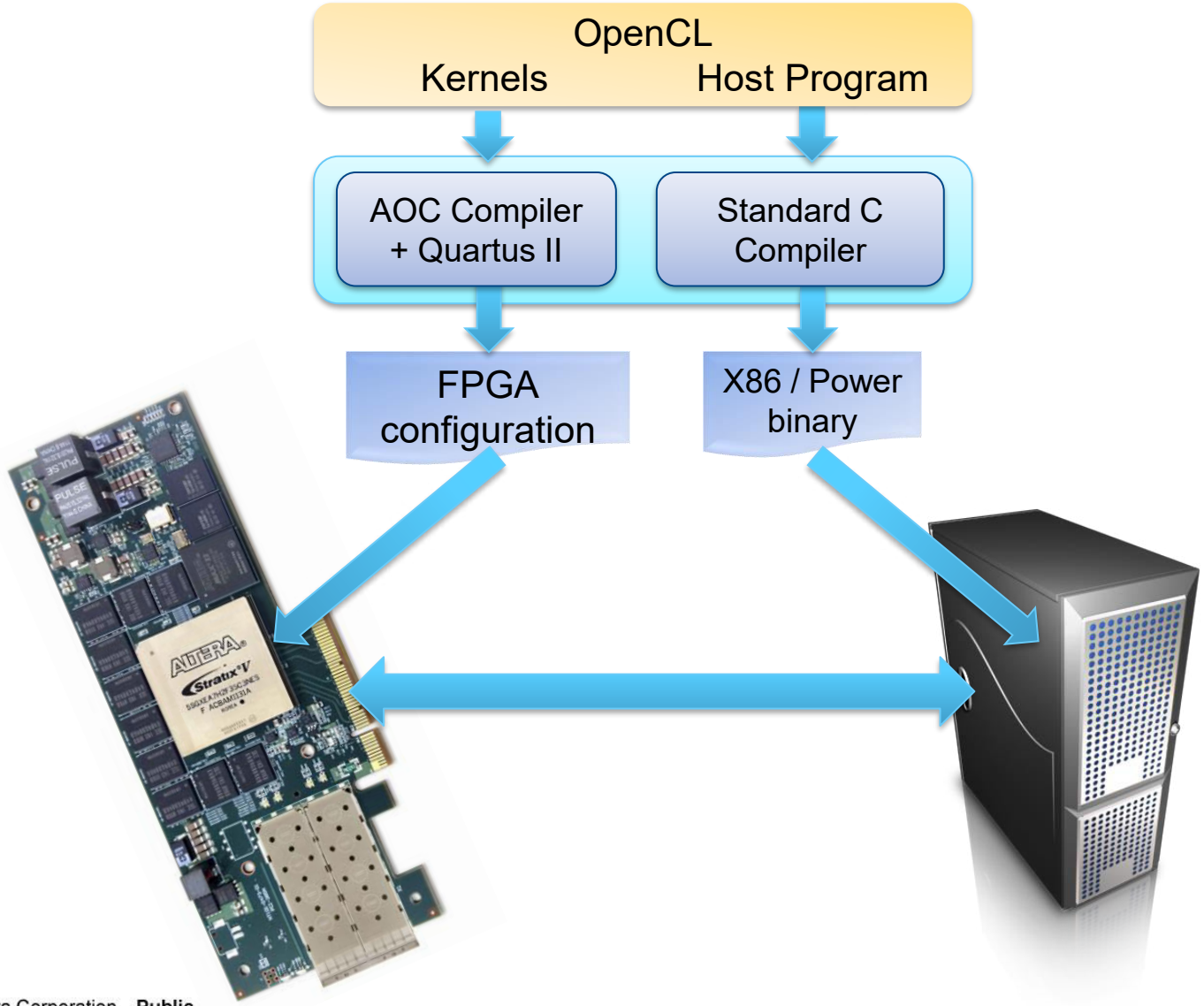
- Kernel: Data-parallel function
  - Defines many parallel threads
  - Keyword extensions to specify parallelism and memory hierarchy
- Executed by an OpenCL device
  - *CPU*
  - *GPU*
  - *FPGA*
  - *DSP*
  - Other Accelerators

```
__kernel void sum(
    __global float *a,
    __global float *b,
    __global float *answer)
{
  int xid = get_global_id(0);
  result[xid] = a[xid] + b[xid];
}
```



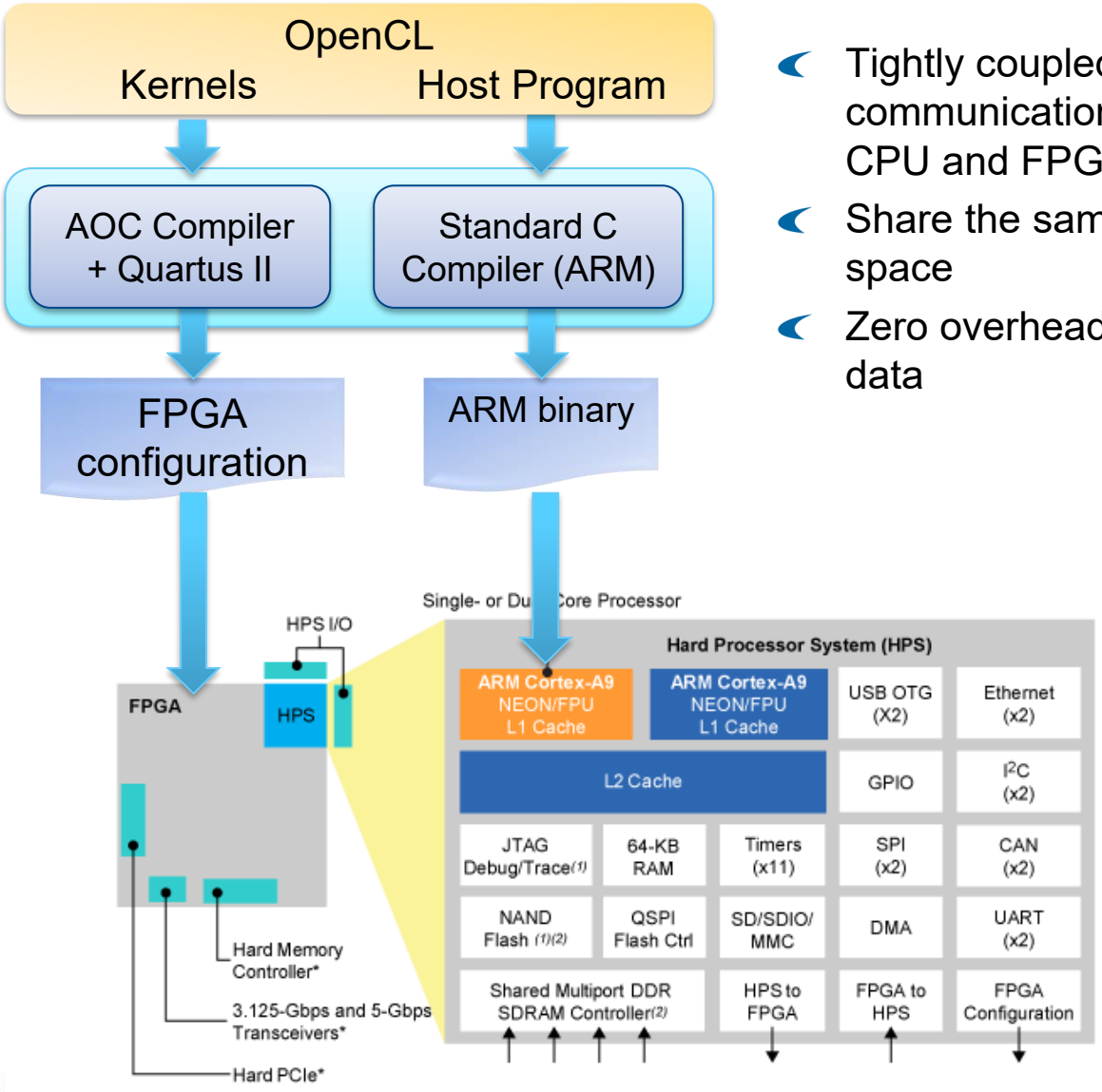
© 2015 Altera Corporation—Public

Mapping OpenCL: FPGA-based Heterogeneous System



© 2015 Altera Corporation—Public

Mapping OpenCL: FPGA-based Heterogeneous SoC

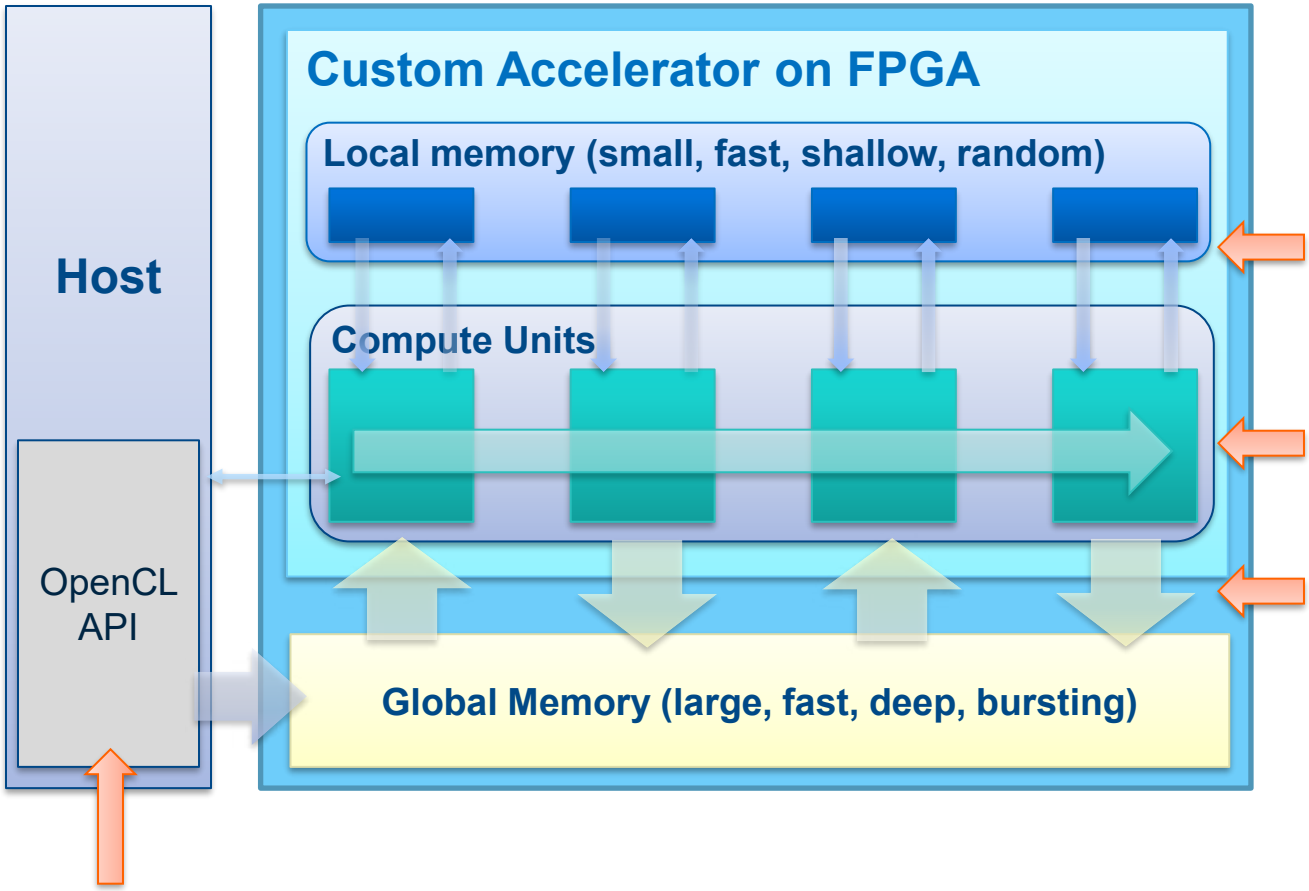


- Tightly coupled communication between CPU and FPGA
- Share the same memory space
- Zero overhead for copying data

- Cyclone V
- Arria V
- Arria 10
- Stratix 10

© 2015 Altera Corporation—Public

Software Programmer's View of an OpenCL Platform



Custom local memory architecture: banking and arbitration

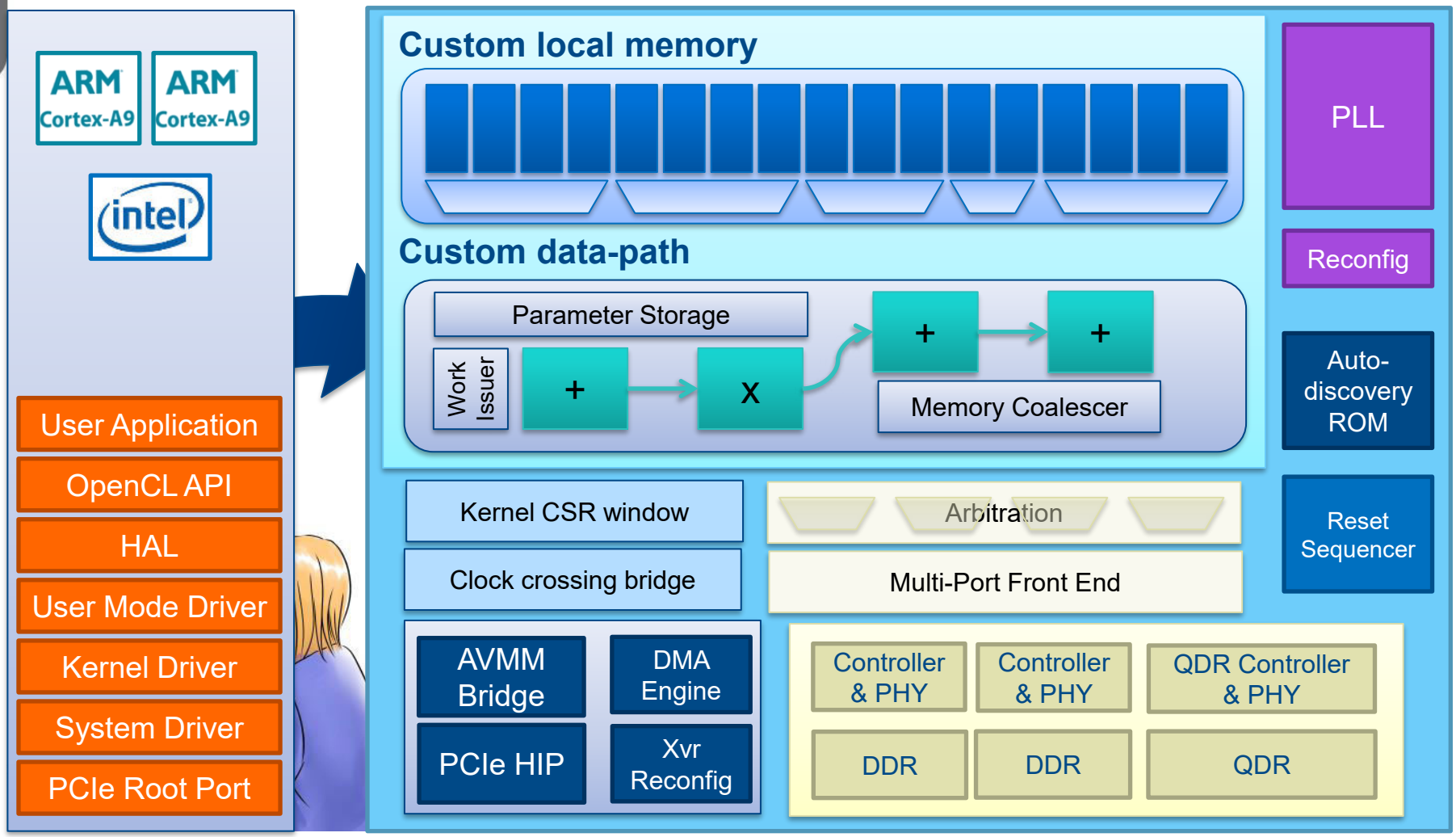
Custom compute units that run the kernels

Custom load/store units and interconnect

Host talks to global memory through OpenCL API routines

© 2015 Altera Corporation—Public

The FPGA-based OpenCL Platform



SW stack on host

Custom FPGA Accelerator

© 2015 Altera Corporation—Public

Fundamentals #1: OpenCL Basics



© 2015 Altera Corporation—Public

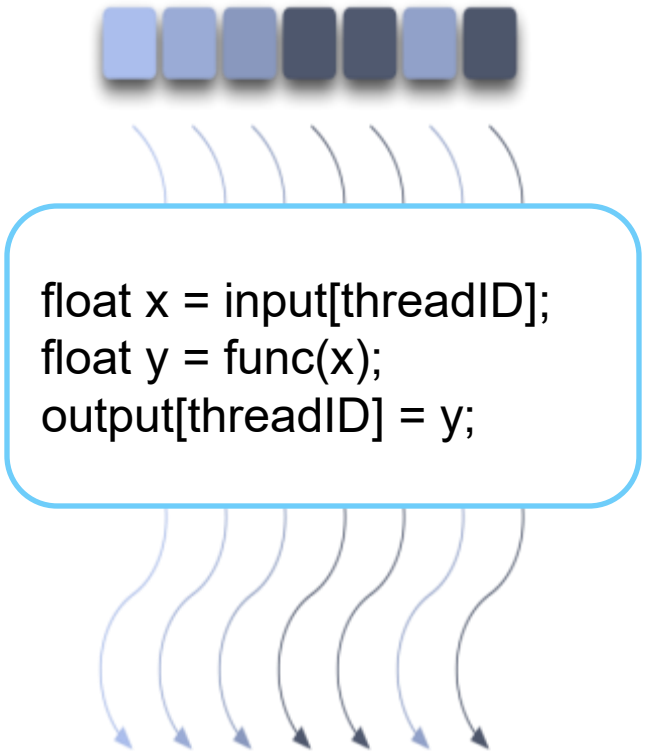
OpenCL

- Derived from ISO C99
  - No standard C99 headers, function pointers, recursion, variable length arrays, and bit fields
- Additions to the language for parallelism
  - Work-items and workgroups
  - Vector types
  - Synchronization
- Address space qualifiers
- Built-in functions

© 2015 Altera Corporation—Public

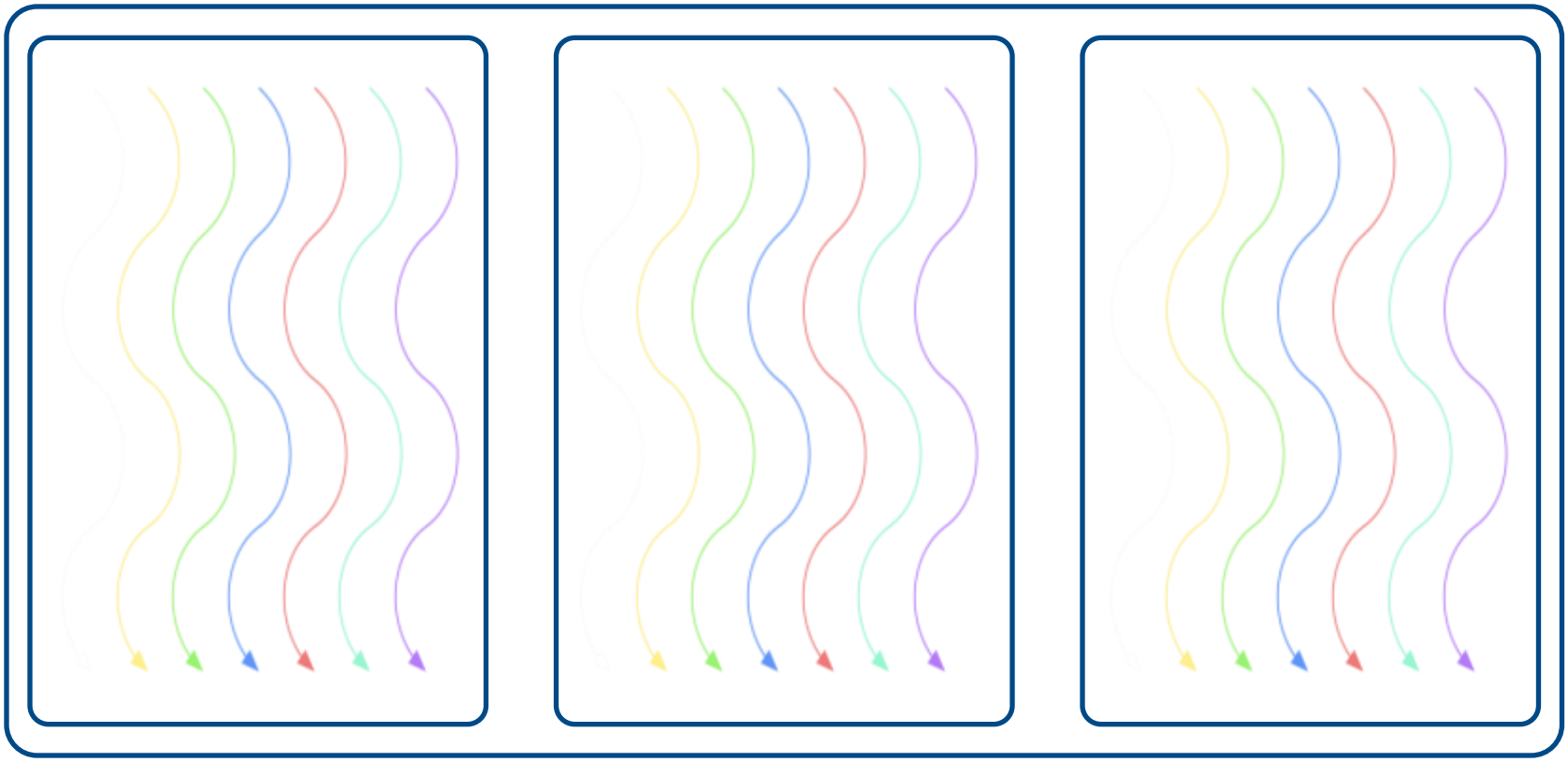
OpenCL Kernels: Parallel Threads

- **A kernel is a function executed on an Accelerator device**
  - Array of threads, in parallel
- **All threads (or work-items) execute the same code, can take different paths**
- **Each thread has an ID**
  - Select input/output data
  - Control decisions



© 2015 Altera Corporation—Public

OpenCL Kernels: Divide into Workgroups



- **Threads in workgroups can cooperate with each through fast local (on-chip) memory**

© 2015 Altera Corporation—Public

Synchronization

- **The OpenCL barrier is a commonly used synchronization primitive**

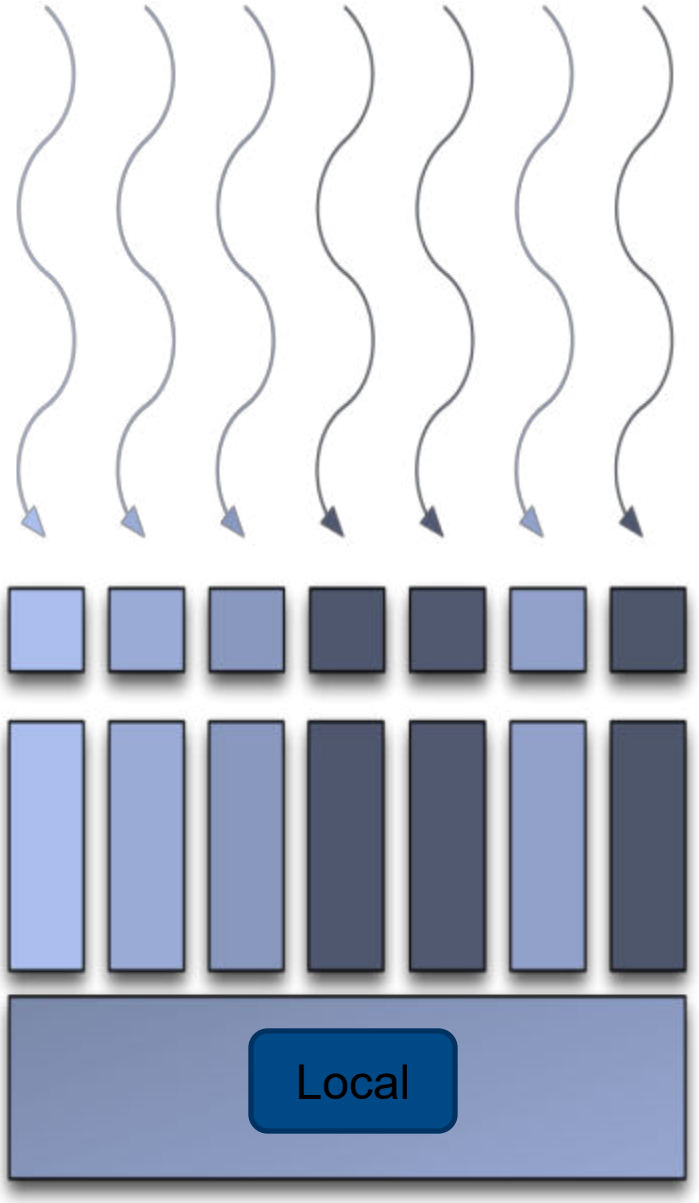
```
__kernel void example()
{
  Code_Segment_A;
  . . .
  barrier( . . . );
  . . .
  Code_Segment_B;
}
```

- **All threads within a workgroup must complete Code_Segment_A before any thread in the workgroup proceeds to Code_Segment_B**

© 2015 Altera Corporation—Public

Memory Hierarchy

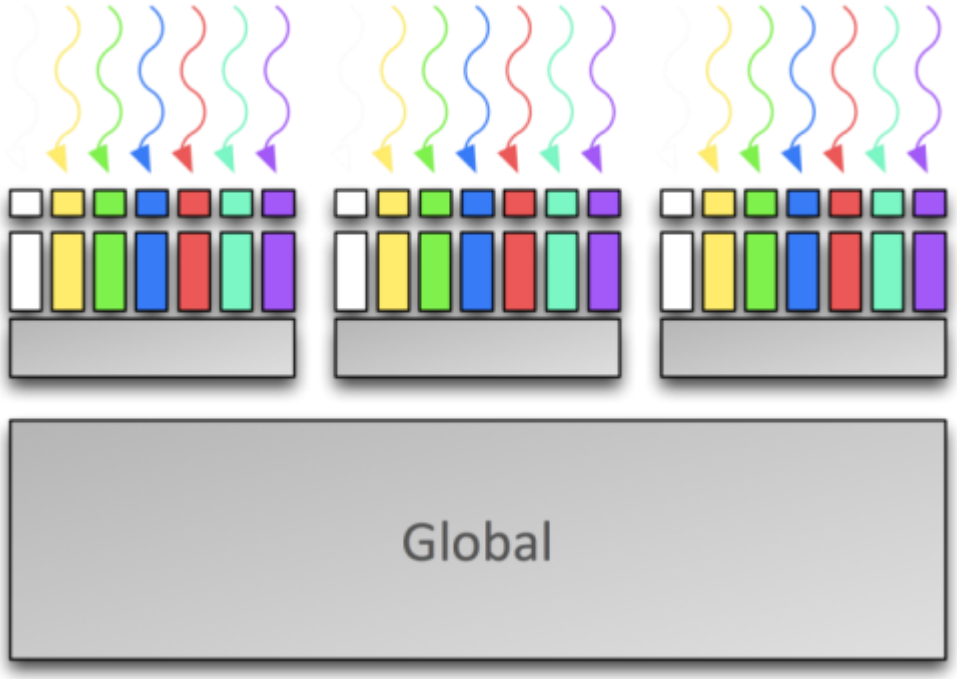
- Thread:
  - Registers
- Thread:
  - Private memory
- Workgroups:
  - Local or Shared memory
- All Workgroups:
  - Global memory



© 2015 Altera Corporation—Public

Memory Hierarchy

- Thread:
  - Registers
- Thread:
  - Private memory
- Workgroups:
  - Local or Shared memory
- All Workgroups:
  - Global memory



© 2015 Altera Corporation—Public

Fundamentals #2: Altera Use Model



© 2015 Altera Corporation—Public

Use Model: Abstracting the FPGA away

```
main() {
   read_data( … );
   manipulate( … );
   clEnqueueWriteBuffer( … );
   clEnqueueNDRange(…,sum,…);
   clEnqueueReadBuffer( … );
   display_result( … );
}
```

OpenCL
Host Program + Kernels

```
__kernel void sum
   (__global float *a,
    __global float *b,
    __global float *y)
{
  int gid = get_global_id(0);
  y[gid] = a[gid] + b[gid];
}
```

Standard C Compiler

Altera Offline Compiler

Verilog

Quartus II

EXE

AOCX

x86





© 2015 Altera Corporation—Public

Altera OpenCL Utility
AOCL

Command	Description
`aocl install`	Install drivers
`aocl diagnostic`	Troubleshooting routine for board installation
`aocl flash <aocx>`	Flash routine for system
`aocl program <aocx>`	Reprogram the FPGA in a live system

Altera Offline Compiler
AOC

Command	Description
`aoc -c <kernel_file>`	Creates **program** object file

	`aoc -c foo.cl`
	⇨`foo.aoco`
	`aoc -c -o <output>.aoco foo.cl`
	⇨`<output>.aoco`

`aoc <kernel_file>` `aoc <object_file>`	Creates **program** executable

	`aoc foo.cl`
	⇨`foo.aocx`
	`aoc -o <output>.aocx foo.cl`
	⇨`<output>.aocx`

	`aoc foo.oaco`
	⇨`foo.aocx`
	`aoc -o <output>.aocx foo.oaco`
	⇨`<output>.aocx`

© 2015 Altera Corporation—**Public**

Compiling Kernel

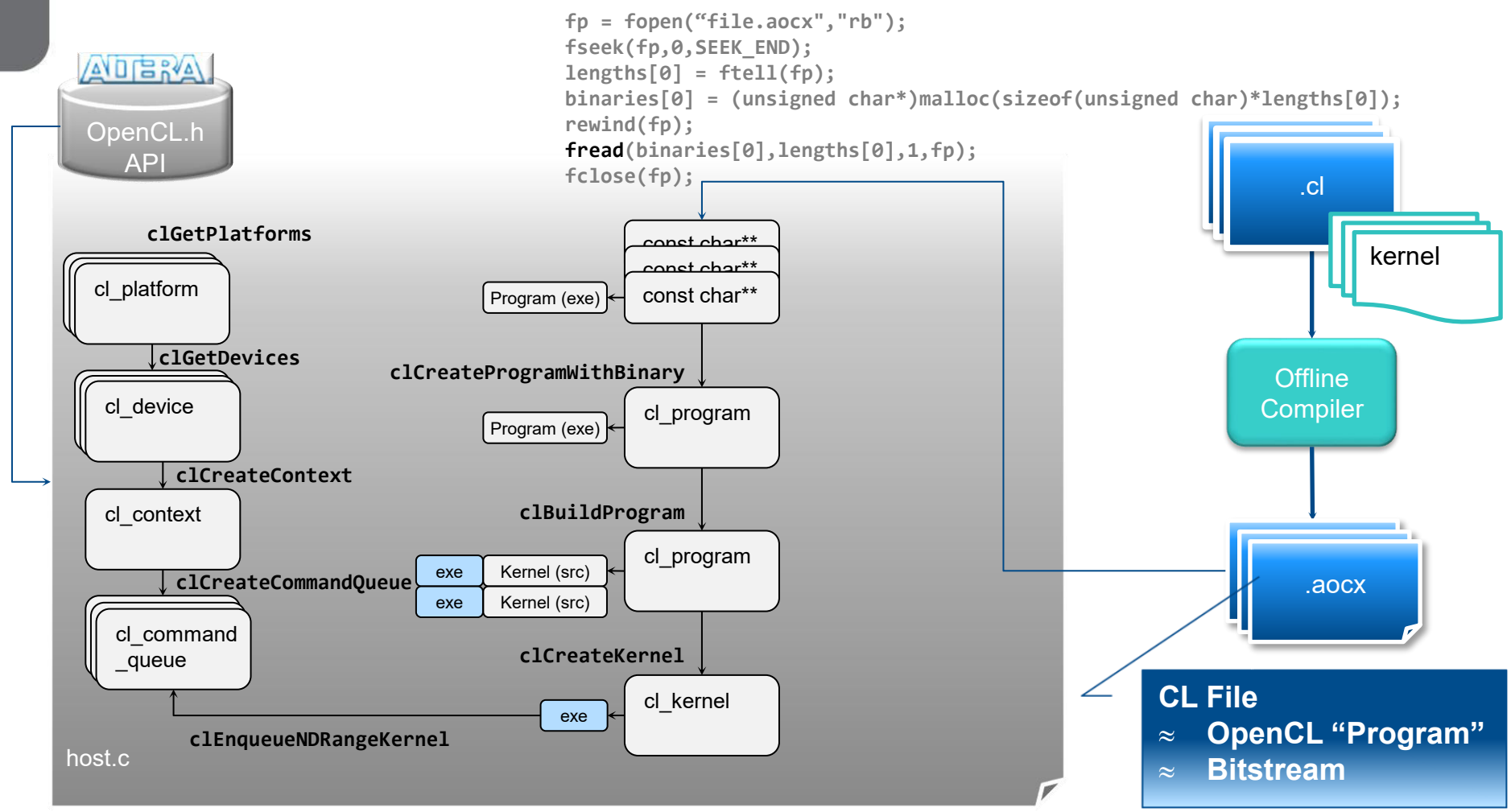
- **Run the Altera Offline Compiler in command prompt**
  - `aoc --board <board> <Kernel.cl>`
  - Run `aoc --list-boards` to see all available boards
- **AOC performs system integration to generate the kernel hardware system and the Quartus II software to compile the design**

```
/mydesigns/matrixMult$ aoc matrixMul.cl
aoc: Selected target board bittware_s5pciehq

+--------------------------------------------------------------------+
; Estimated Resource Usage Summary                                  ;
+----------------------------------------+---------------------------+
; Resource                               + Usage                     ;
+----------------------------------------+---------------------------+
; Logic utilization                      ;   52%                     ;
; Dedicated logic registers              ;   23%                     ;
; Memory blocks                          ;   31%                     ;
; DSP blocks                             ;   54%                     ;
+----------------------------------------+---------------------------;
```


© 2015 Altera Corporation—Public

Use Model: clCreateProgramWithBinary



© 2015 Altera Corporation—Public

Fundamentals #3 : Mapping Parallel Threads



© 2015 Altera Corporation—Public

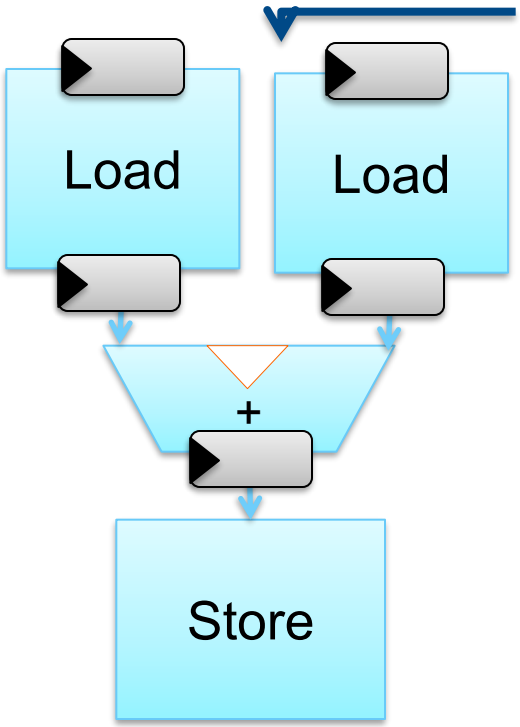
Mapping Multithreaded Kernels to FPGAs

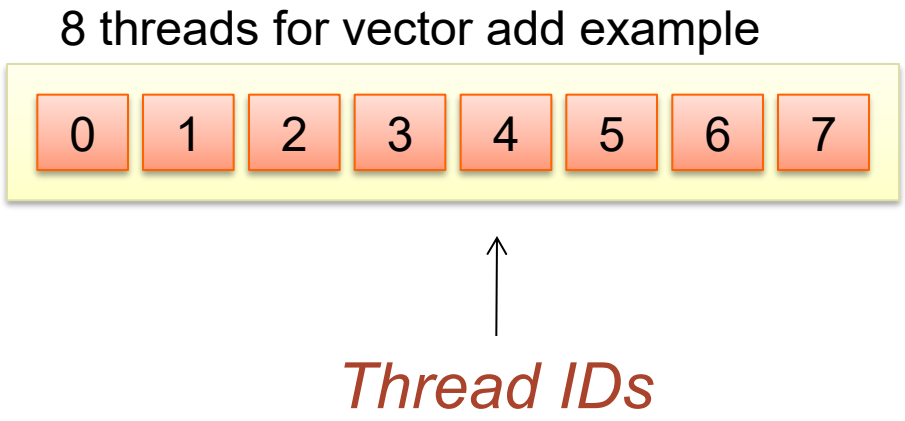
- **The most simple way of mapping kernel functions to FPGAs is to replicate the unrolled hardware for each thread**
  - Inefficient and wasteful

- **Better method involves taking advantage of *pipeline parallelism***
  - Attempt to create a deeply pipelined representation of a kernel
  - On each clock cycle, we attempt to send in input data for a new thread
  - Method of mapping coarse grained thread parallelism to fine-grained FPGA parallelism

© 2015 Altera Corporation—**Public**

Example Pipeline for Vector Add

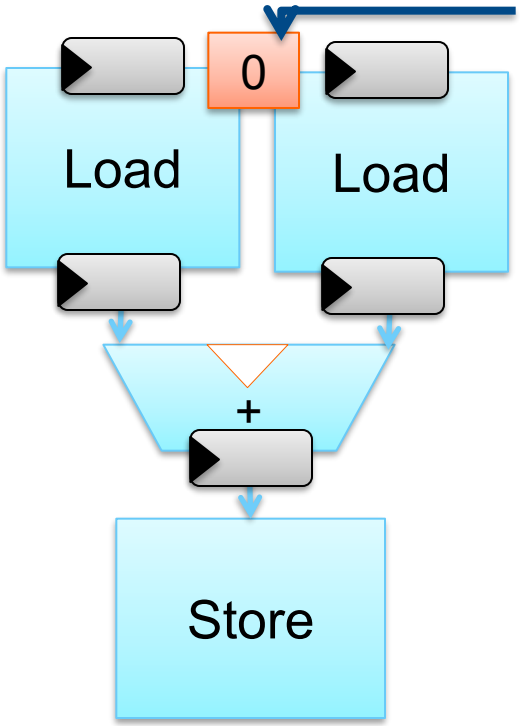


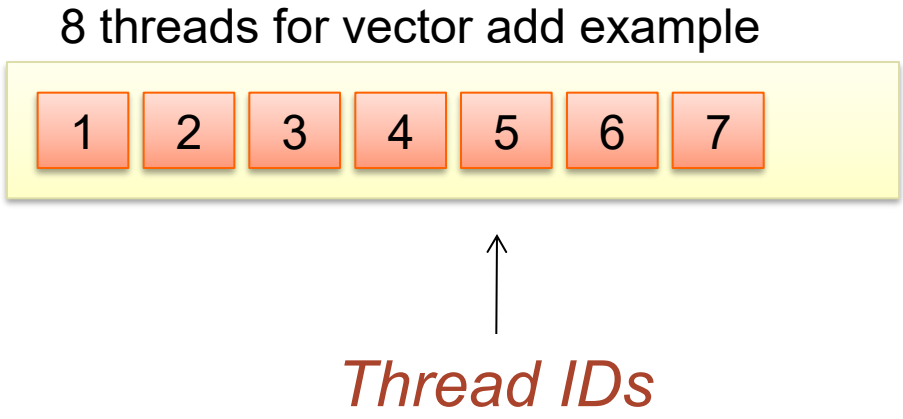


- **On each cycle the portions of the pipeline are processing different threads**
- **While thread 2 is being loaded, thread 1 is being added, and thread 0 is being stored**

© 2015 Altera Corporation—Public

Example Pipeline for Vector Add

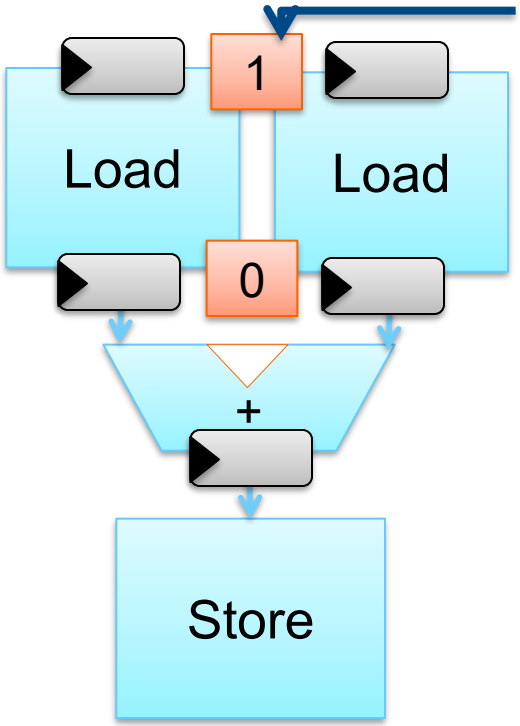


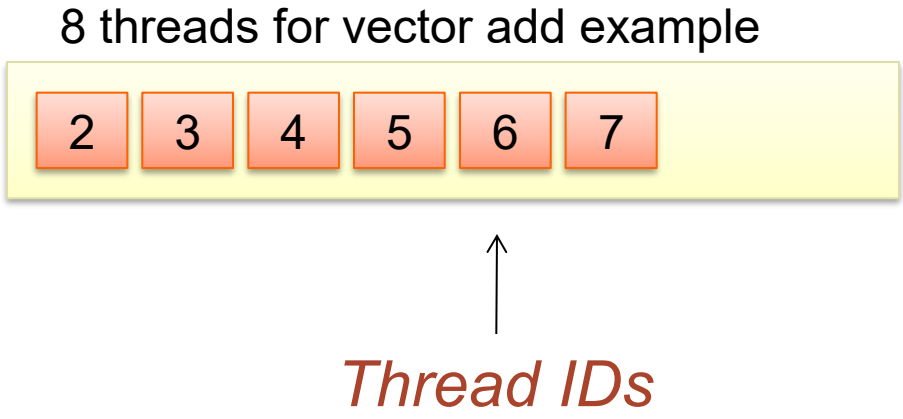


- **On each cycle the portions of the pipeline are processing different threads**
- **While thread 2 is being loaded, thread 1 is being added, and thread 0 is being stored**

© 2015 Altera Corporation—Public

Example Pipeline for Vector Add

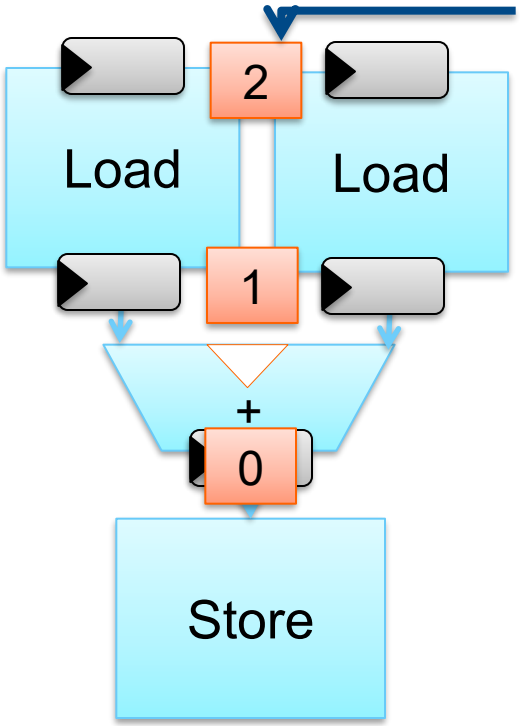


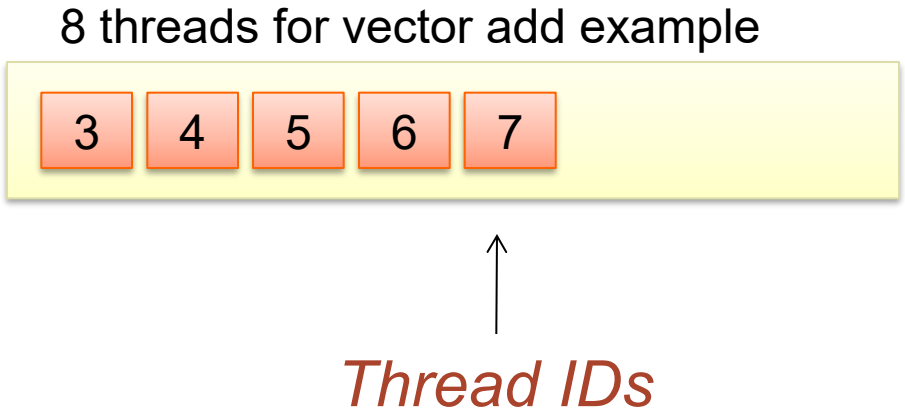


- **On each cycle the portions of the pipeline are processing different threads**
- **While thread 2 is being loaded, thread 1 is being added, and thread 0 is being stored**

© 2015 Altera Corporation—**Public**

Example Pipeline for Vector Add

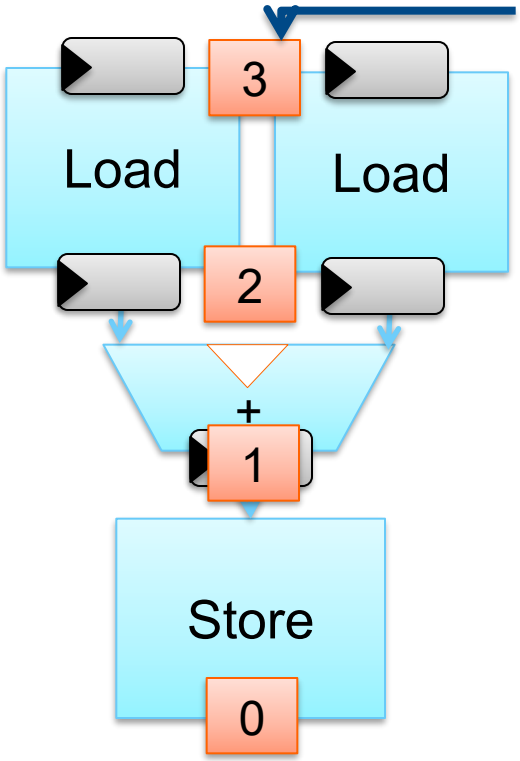




- **On each cycle the portions of the pipeline are processing different threads**
- **While thread 2 is being loaded, thread 1 is being added, and thread 0 is being stored**

© 2015 Altera Corporation—Public

Example Pipeline for Vector Add



8 threads for vector add example





Thread IDs

- **On each cycle the portions of the pipeline are processing different threads**
- **While thread 2 is being loaded, thread 1 is being added, and thread 0 is being stored**

© 2015 Altera Corporation—**Public**

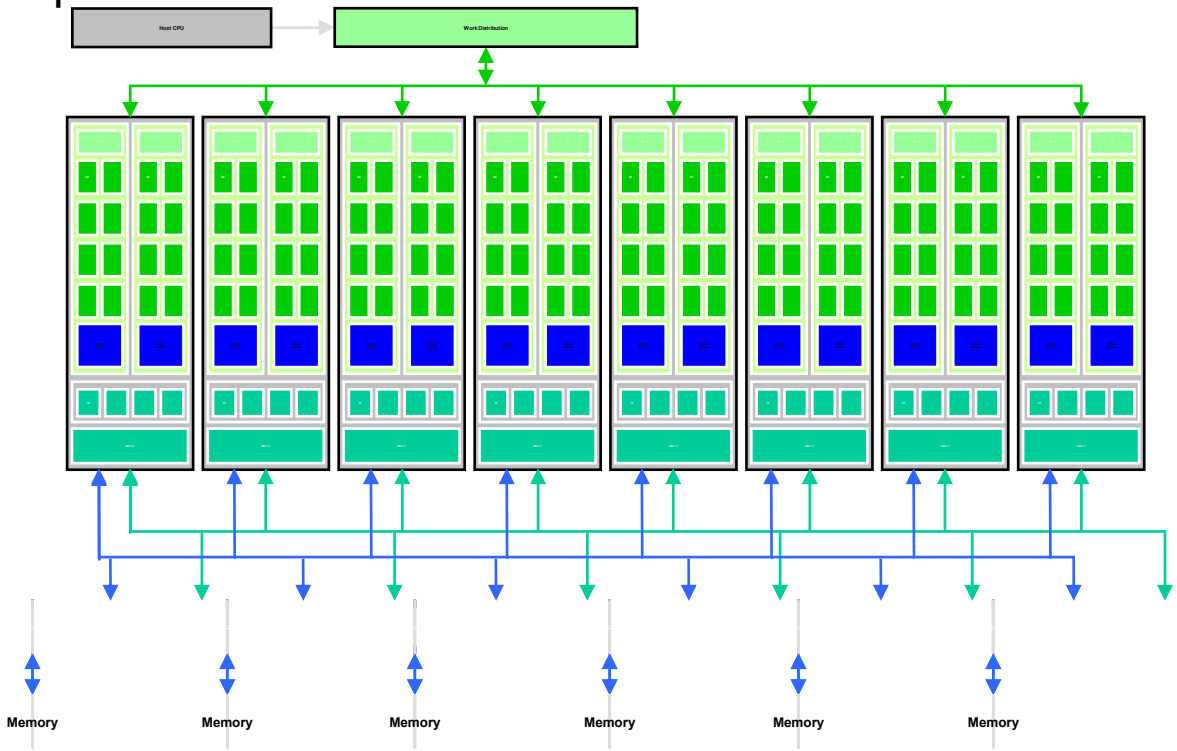
Fundamentals #4 : More Parallel Performance



© 2015 Altera Corporation—Public

OpenCL on GPU / CPU Architectures

- **Conceptually many parallel threads**
  - Simplified View → Each thread runs sequentially on a different processing element (PE)
    - Fixed #s of Functional Units, Registers available on each PE
  - Many processing elements are available to provide significant parallel speedup



© 2015 Altera Corporation—Public

OpenCL on FPGA

- **OpenCL kernels are translated into a highly parallel circuit**
  - ***A unique functional unit is created for every operation in the kernel***
    - Memory loads / stores, computational operations, registers
  - Functional units are only connected when there is some data dependence dictated by the kernel
- **Pipeline the resulting circuit with a new thread on each clock cycle to keep functional units busy**

Amount of parallelism is dictated by the number of pipelined computing operations in the generated HW

© 2015 Altera Corporation—Public

Simple Example

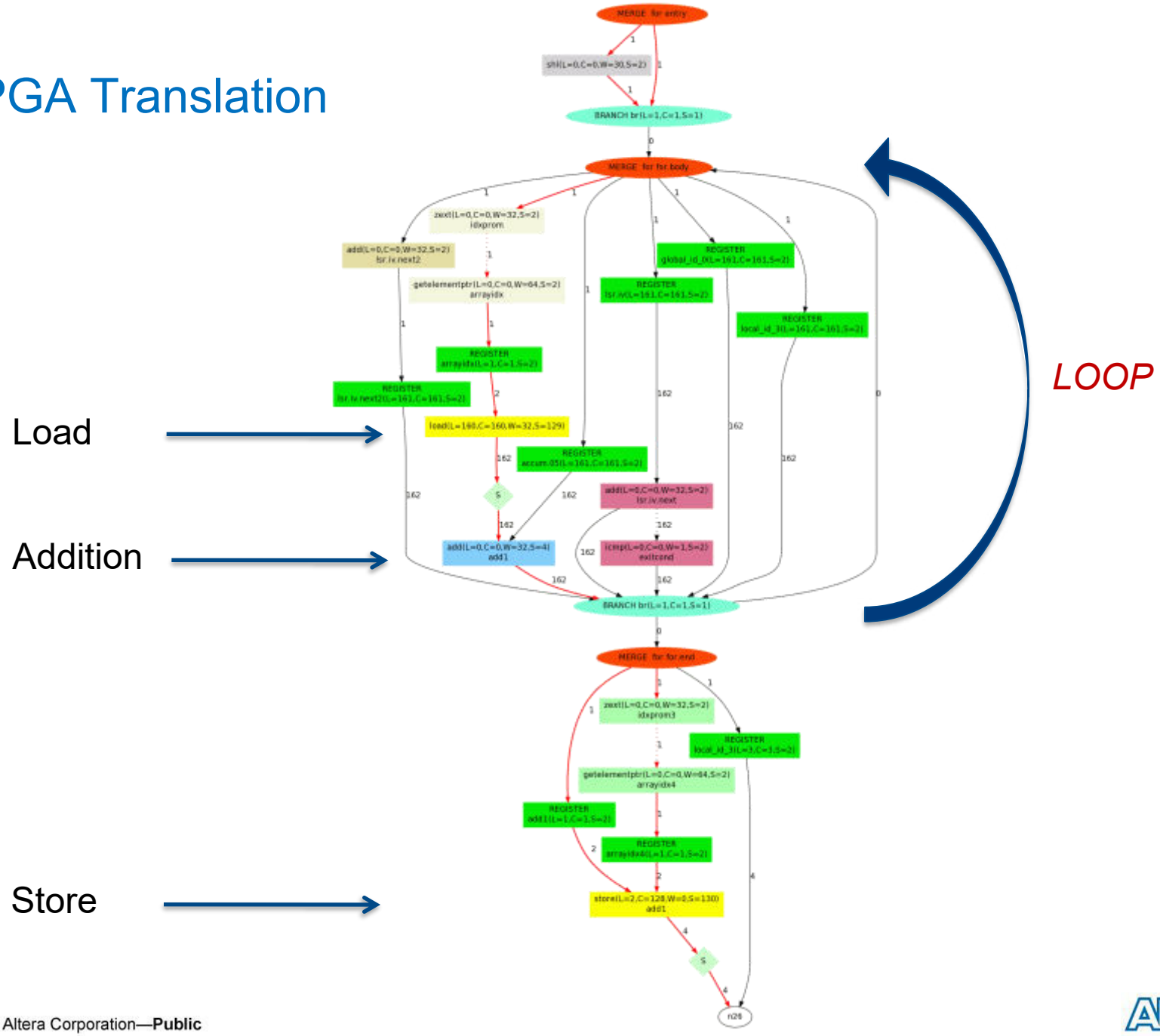
```
__kernel void example(__global int *x, __global int
*sum)
{
   int accum = 0;

   for (int i=0; i<4; i++) {
      accum += x[ i + get_global_id(0) * 4 ];
   }
   sum[get_global_id(0)] = accum;
}
```

- **Simple accumulation of 4 integers for every thread**

© 2015 Altera Corporation—Public

FPGA Translation



© 2015 Altera Corporation—Public

Modified Example

```
__kernel void example(__global int *x, __global int
*sum)
{
   int accum = 0;

#pragma unroll
   for (int i=0; i<4; i++) {
      accum += x[ i + get_global_id(0) * 4 ];
   }
   sum[get_global_id(0)] = accum;
}
```

- **Unroll all instructions inside the *for* loop**

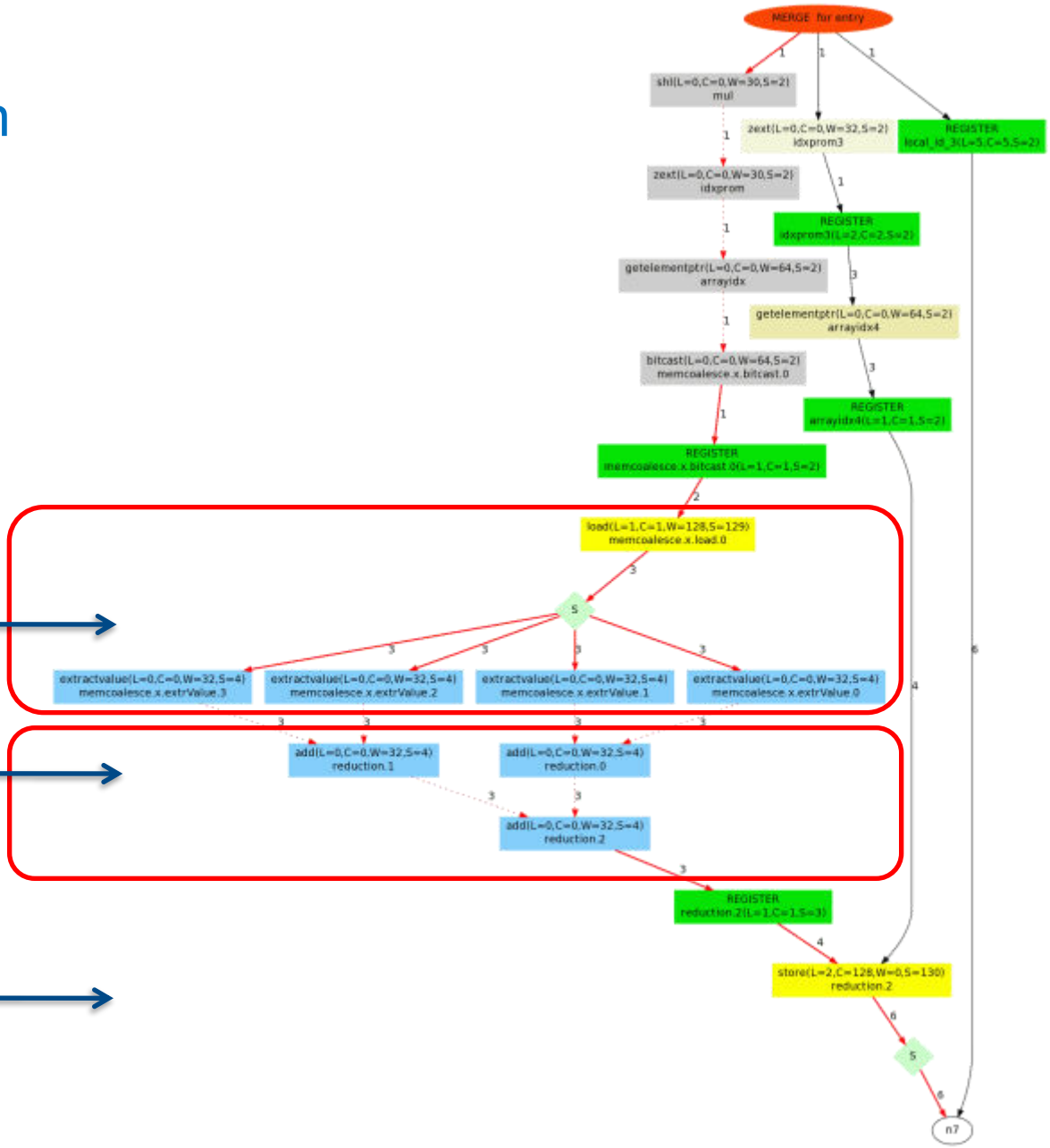
© 2015 Altera Corporation—Public

New Translation

4 Load

3 Addition

1 Store



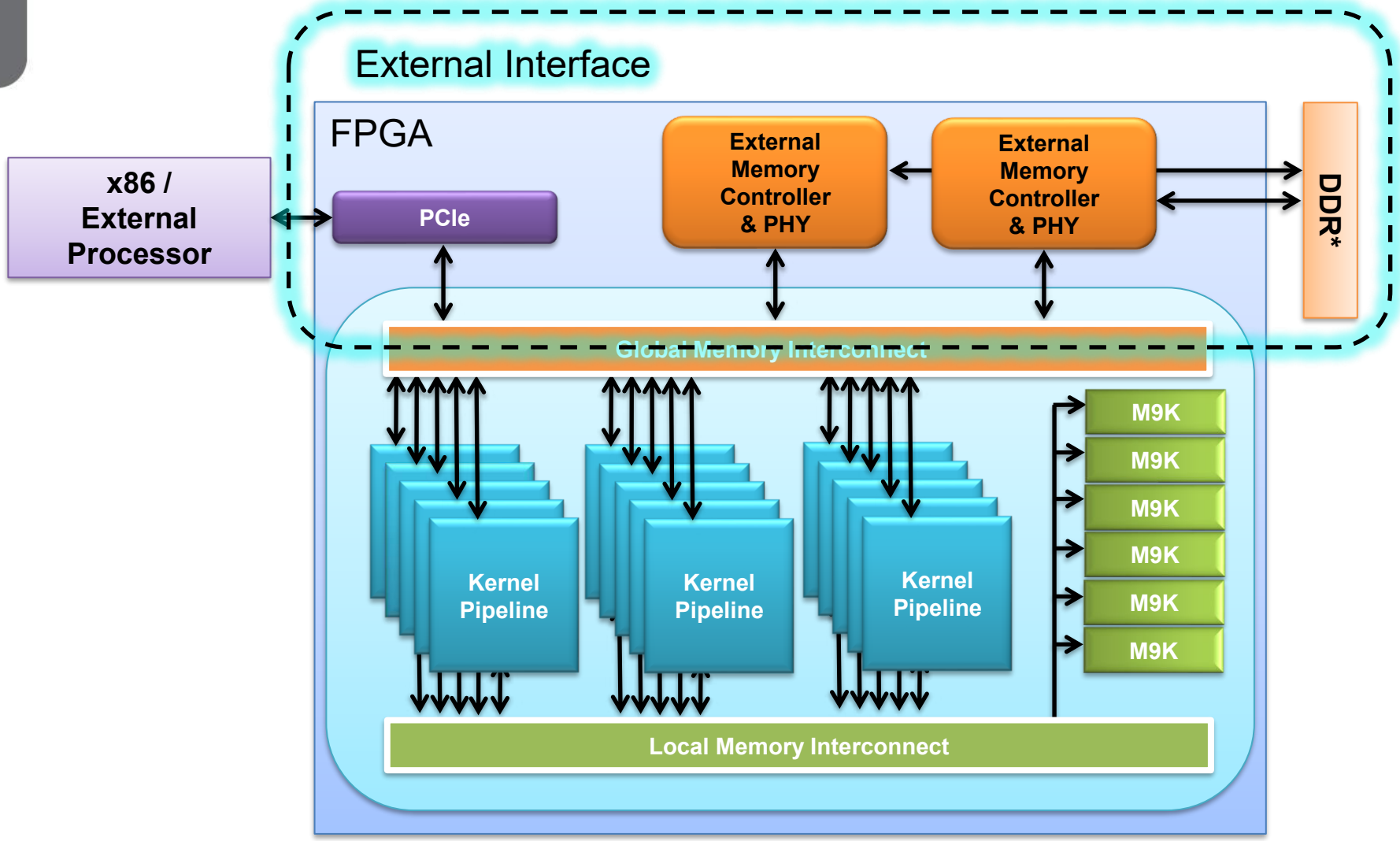
© 2015 Altera Corporation—Public

Fundamentals #5: Memory Access



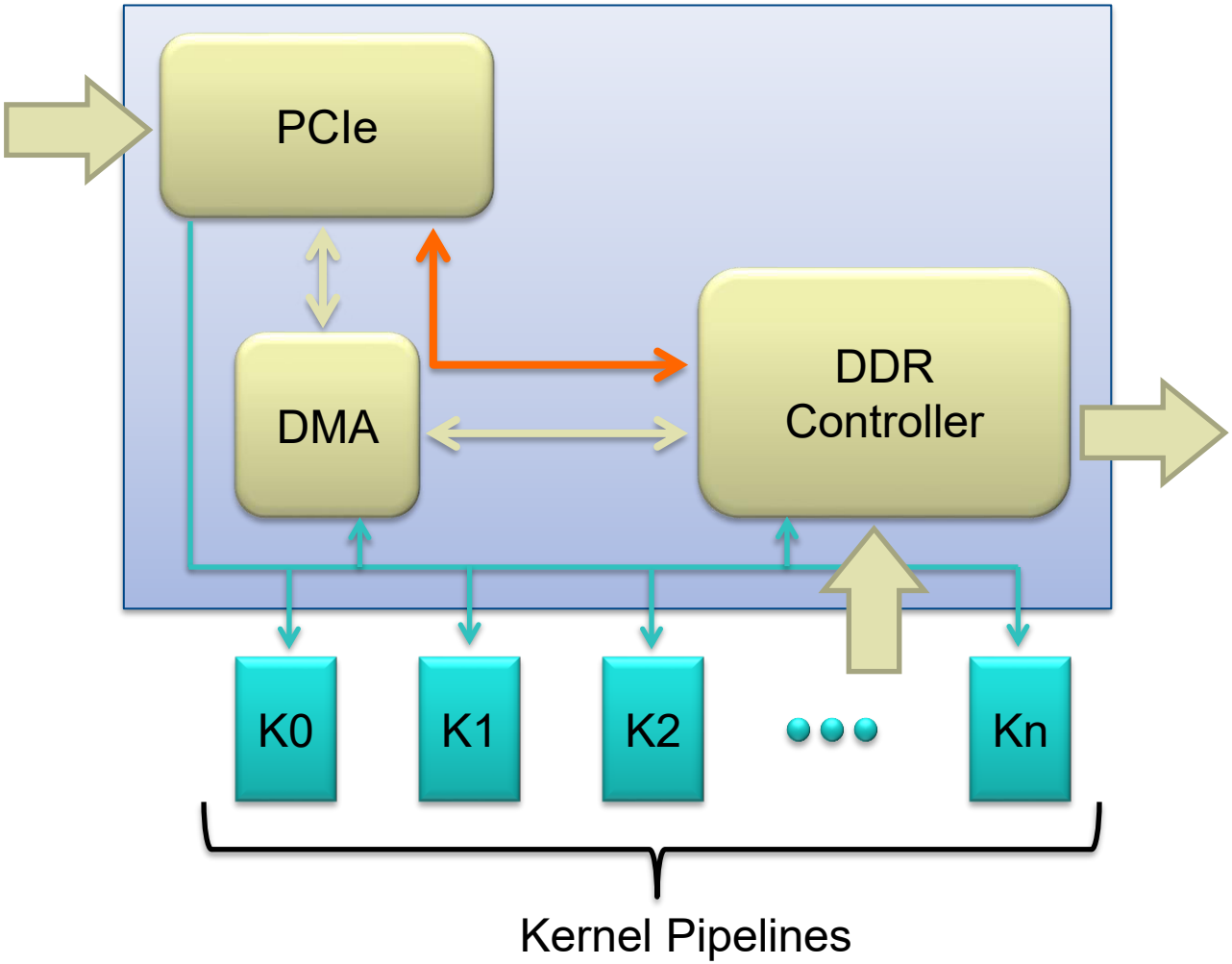
© 2015 Altera Corporation—Public

Simplified OpenCL System Architecture



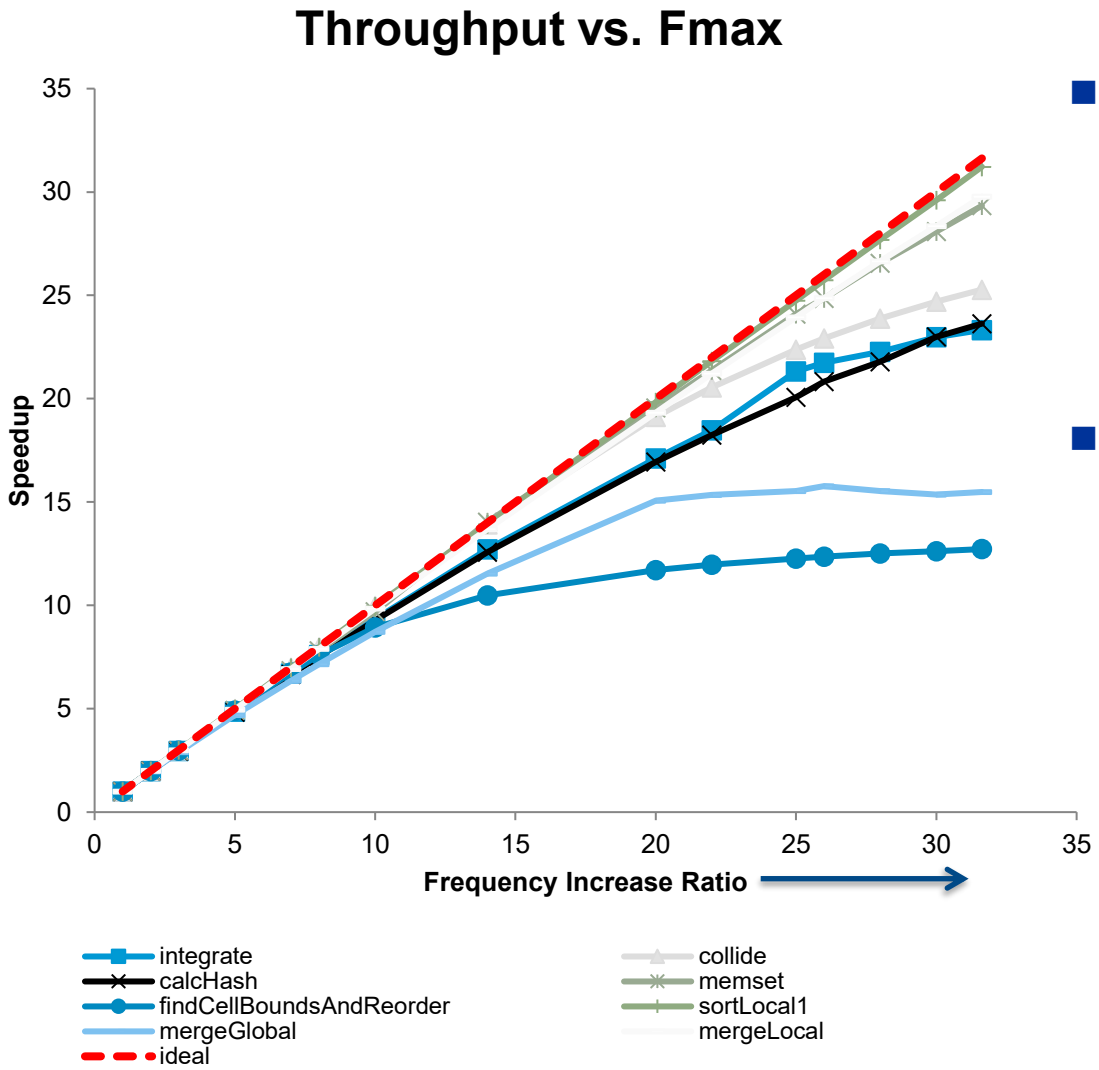
© 2015 Altera Corporation—Public

External Interface : High Level



© 2015 Altera Corporation—Public

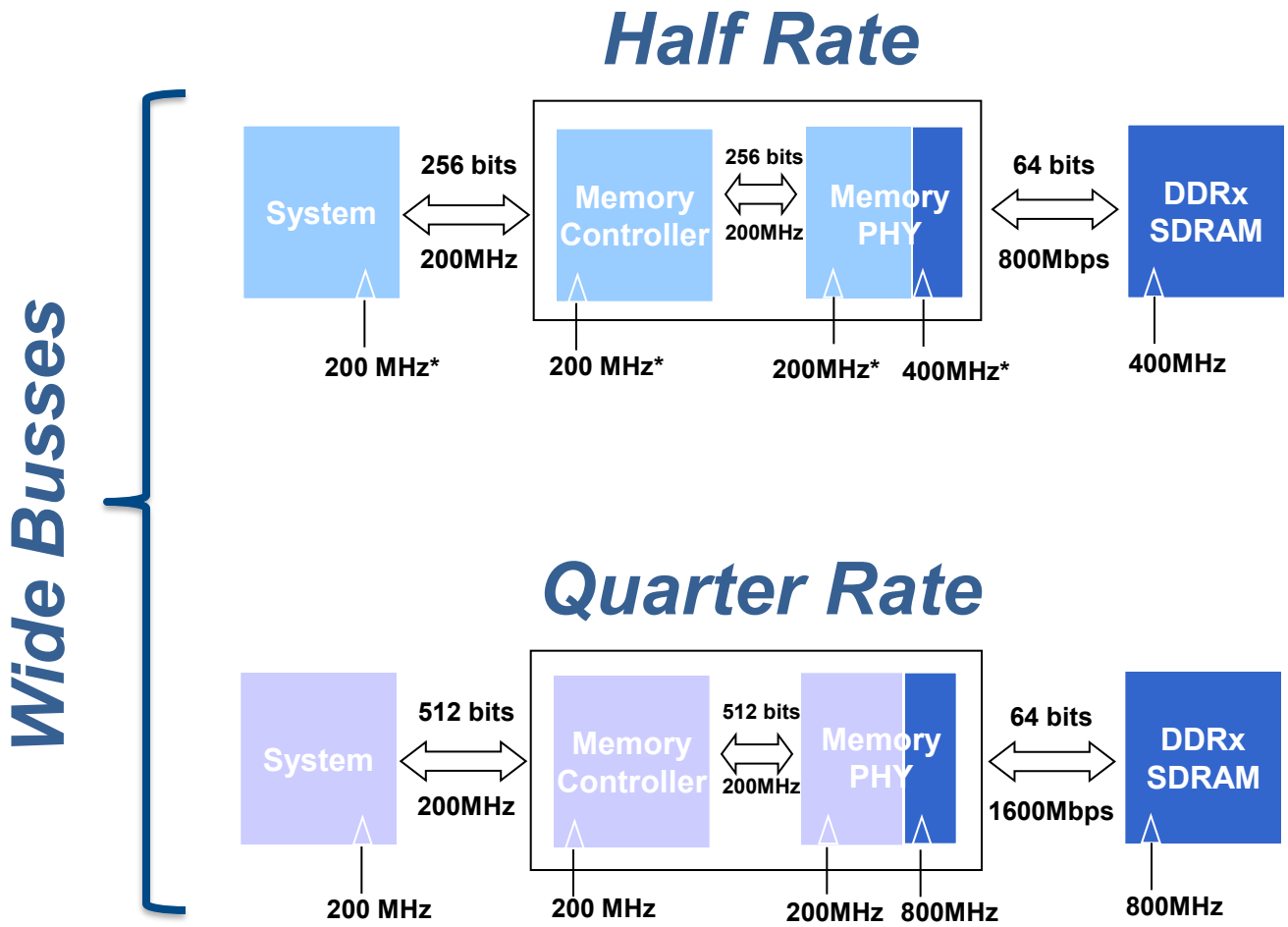
Memory Bandwidth and Throughput



- **Global memory bandwidth is often a "hard cap" on kernel performance**
- **Compiler needs to understand kernel-specific memory patterns to optimize use of bandwidth**

© 2015 Altera Corporation—Public

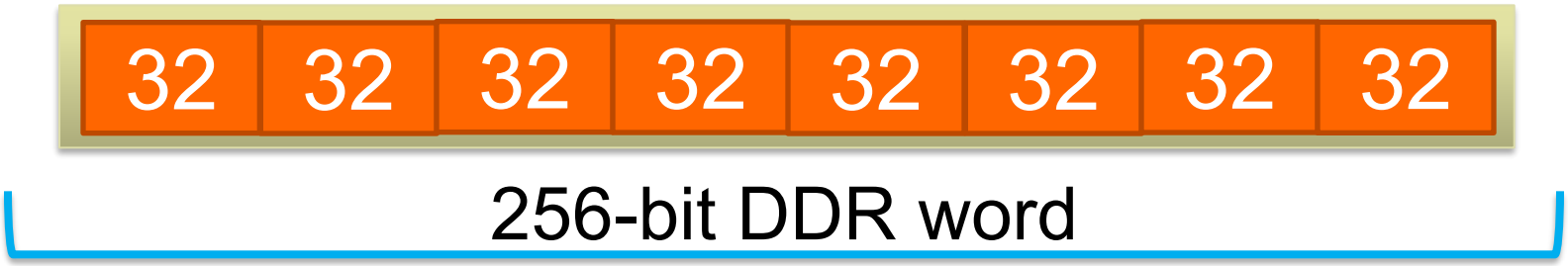
DDRx Configuration



© 2015 Altera Corporation—Public

Load/Store Memory Coalescing

- **External memory has wide words (256/512 bits)**
- **Loads/stores typically access narrower words (32 or 128 bits)**



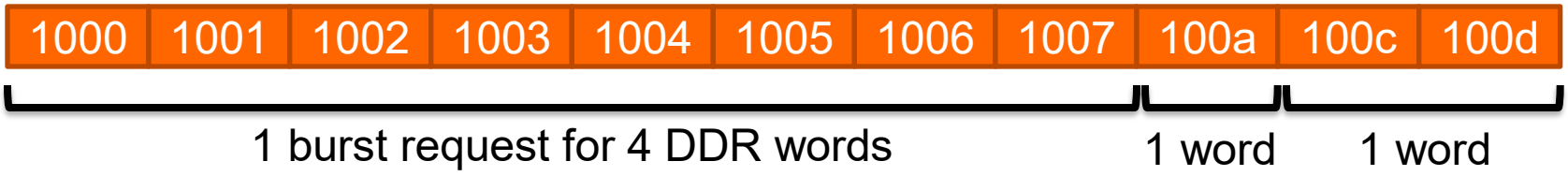
- **Given a sequence of loads/stores, we want to make as few external memory read/write requests as possible**

© 2015 Altera Corporation—Public

Load/Store Memory Coalescing

- **Coalescing is important for good performance**
  - Combine loads/stores that access the same DDR word **or the one ahead of the previously-accessed DDR word**
  - Make one big multi-word burst request to external memory whenever possible
  - ➢ Fewer requests → less contention to global memory
  - ➢ Contiguous bursts → less external memory overhead

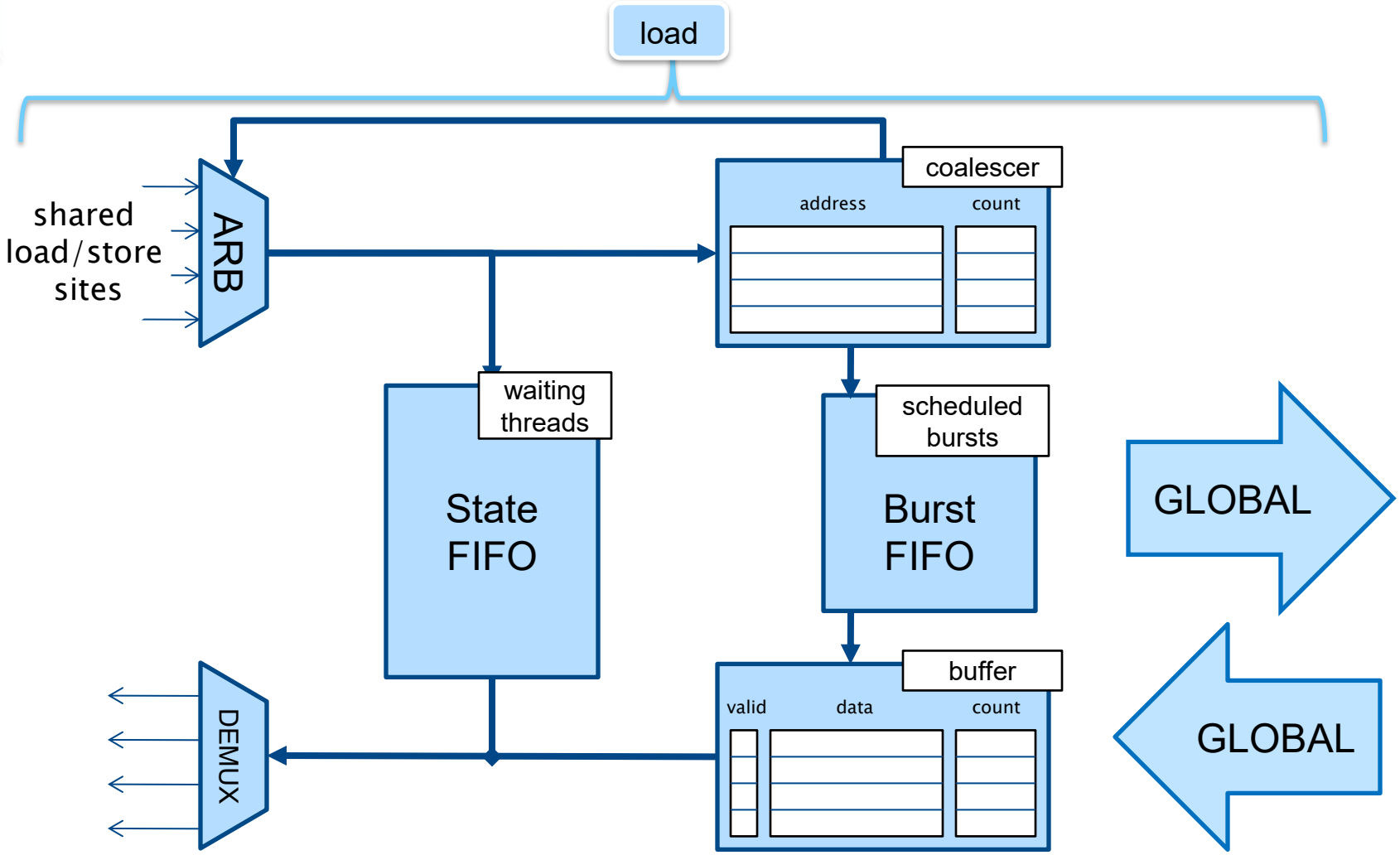
Load/Store Addresses (128-bit words):



→ 3 requests in total

© 2015 Altera Corporation—**Public**

High-Level Block Diagram



© 2015 Altera Corporation—Public

Using Local Memory

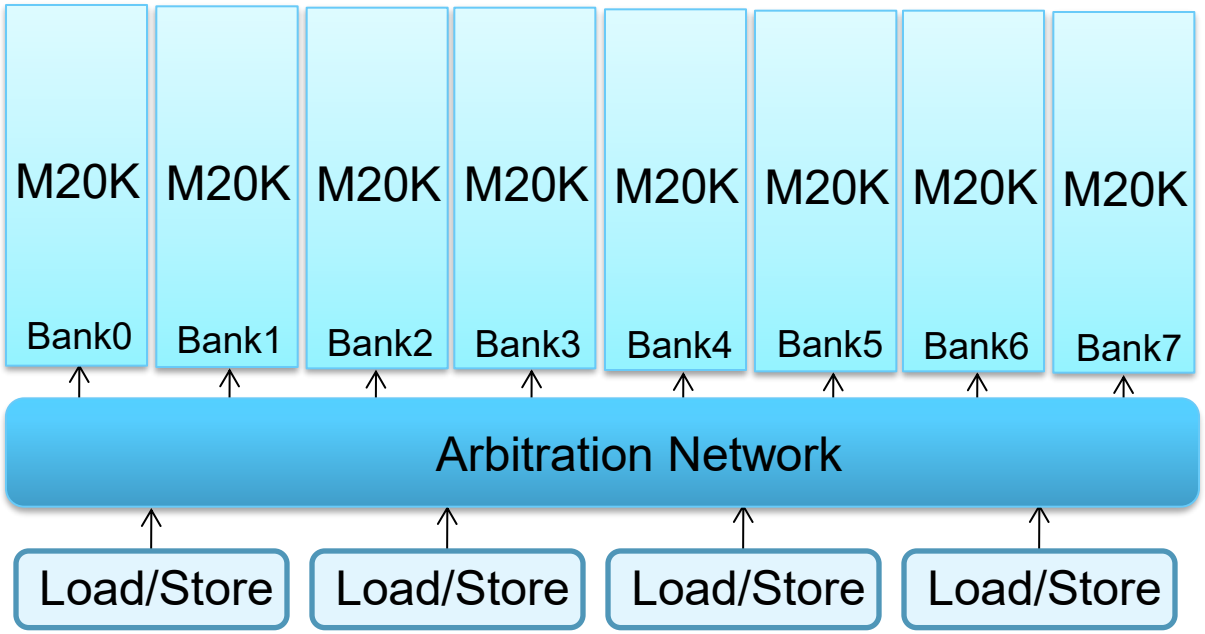
- **Local memory is implemented as a small amount of high bandwidth onchip memory that has excellent random access behavior**

- **Since global memory is high latency and relatively lower bandwidth, a common OpenCL programming methodology is to:**

Copy blocks of data from global memory to local memory and perform as much compute as possible in the local buffers before moving on to the next block.

© 2015 Altera Corporation—**Public**

FPGA Local Memory



- **Split memory into logical banks**
  - An N-bank configuration can handle N-requests per clock cycle as long as each request addresses a different bank
  - Manipulate memory addresses so that parallel threads likely to access different banks – reduce collisions

© 2015 Altera Corporation—Public

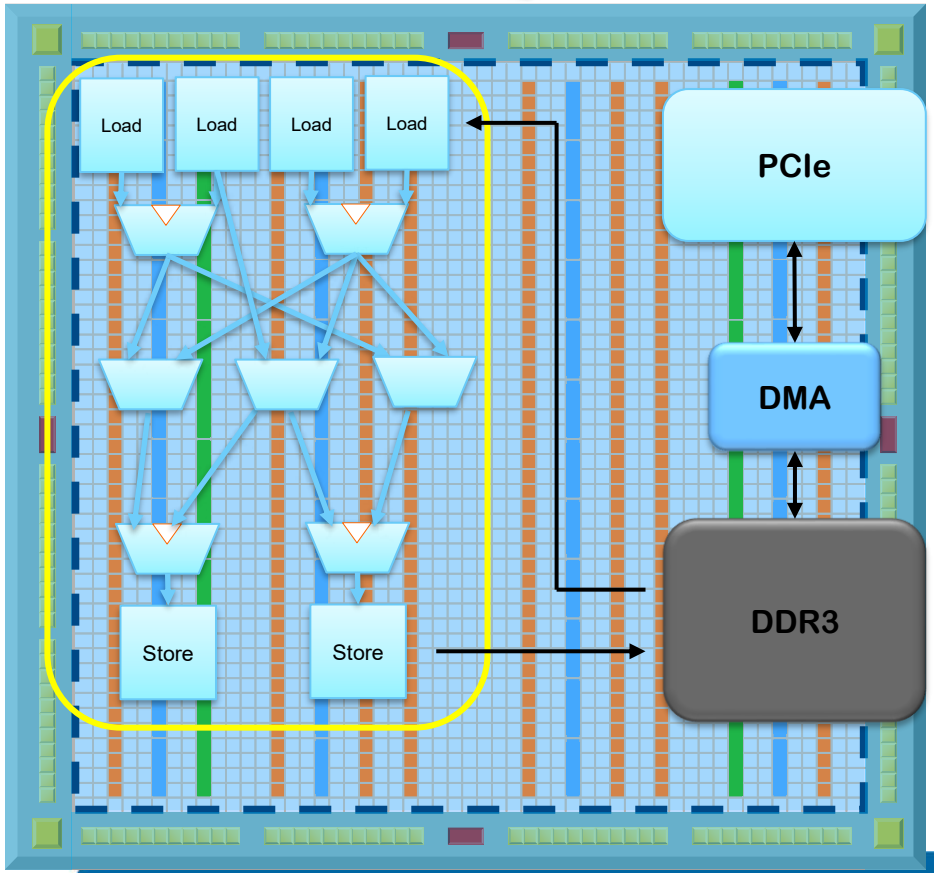
Compiling OpenCL to FPGAs (cont'd)

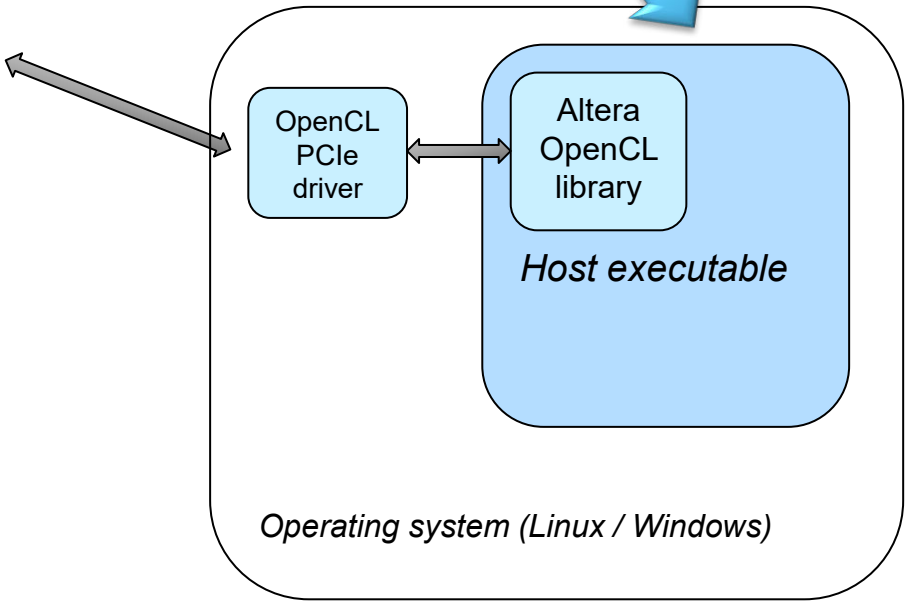
```
__kernel void
sum(__global const float *a,
__global const float *b,
__global float *answer)
{
int xid = get_global_id(0);
answer[xid] = a[xid] + b[xid];
}
```

```
main()
{
   read_data_from_file( … );
   maninpulate_data( … );

   clEnqueueWriteBuffer( … );
   clEnqueueKernel(…, sum, …);
   clEnqueueReadBuffer( … );

   display_result_to_user( … );
}
```





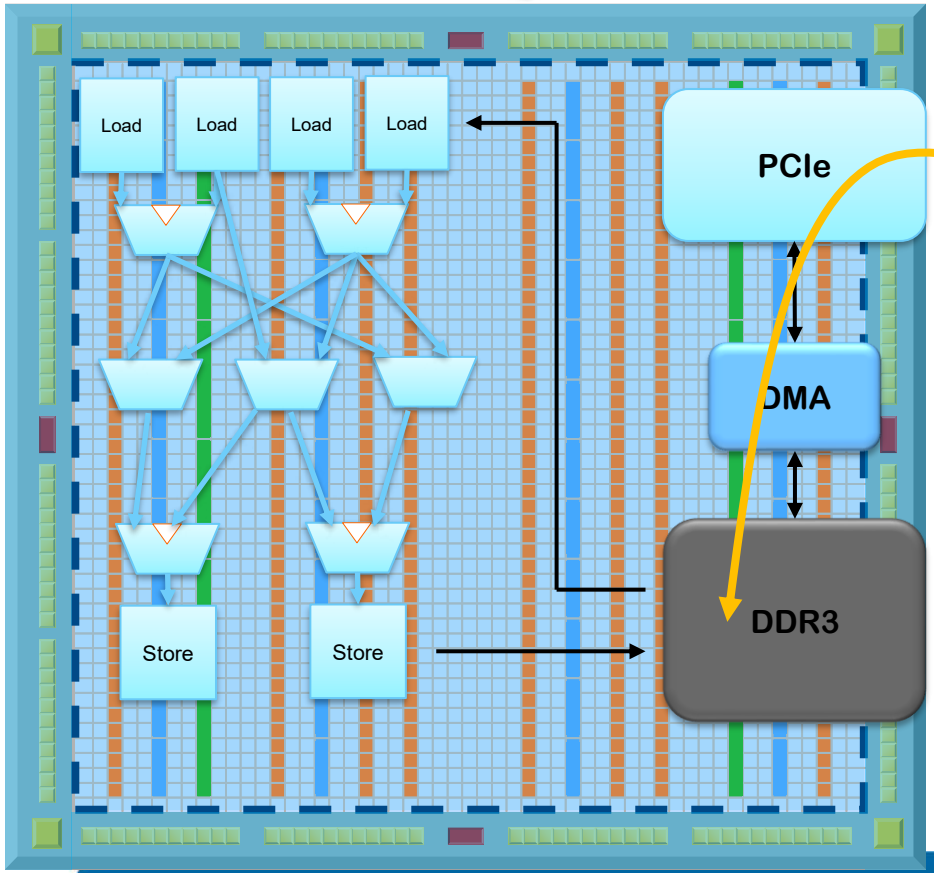
Processing flow

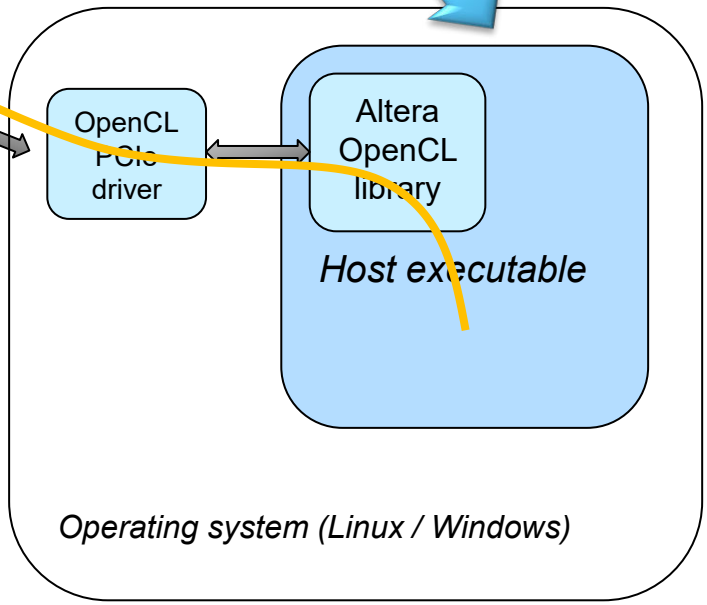
```
__kernel void
sum(__global const float *a,
__global const float *b,
__global float *answer)
{
int xid = get_global_id(0);
answer[xid] = a[xid] + b[xid];
}
```

```
main()
{
   read_data_from_file( … );
   maninpulate_data( … );

   clEnqueueWriteBuffer( … );
   clEnqueueKernel(…, sum, …);
   clEnqueueReadBuffer( … );

   display_result_to_user( … );
}
```





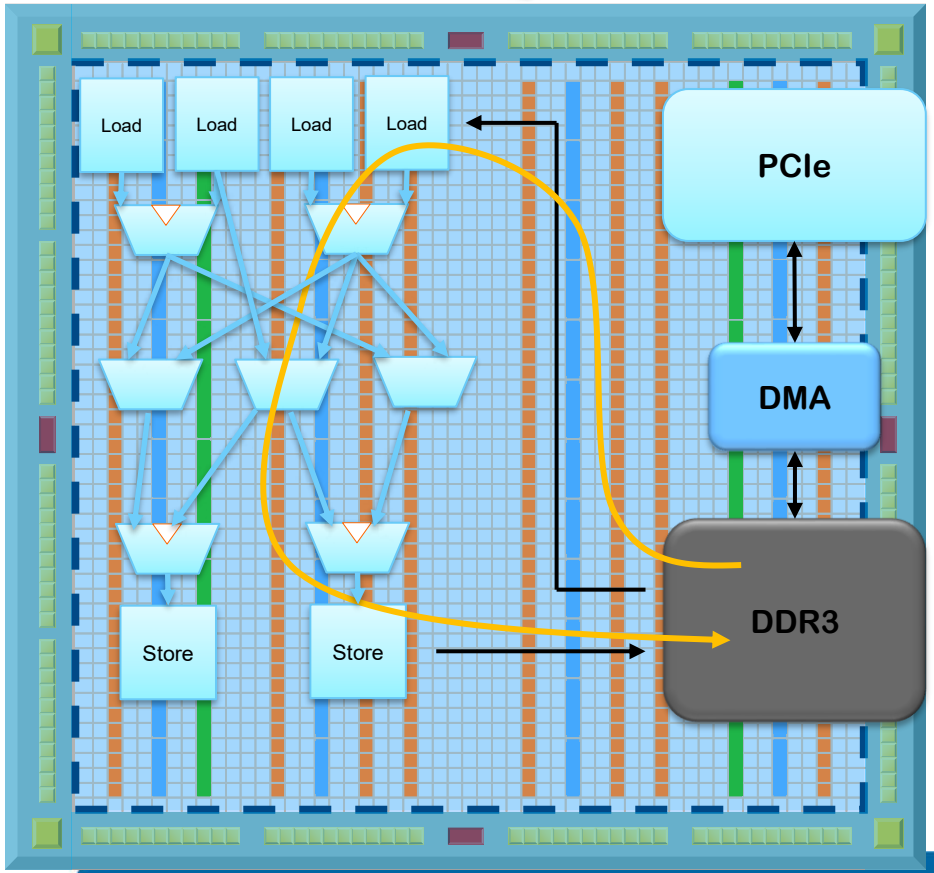
Processing flow

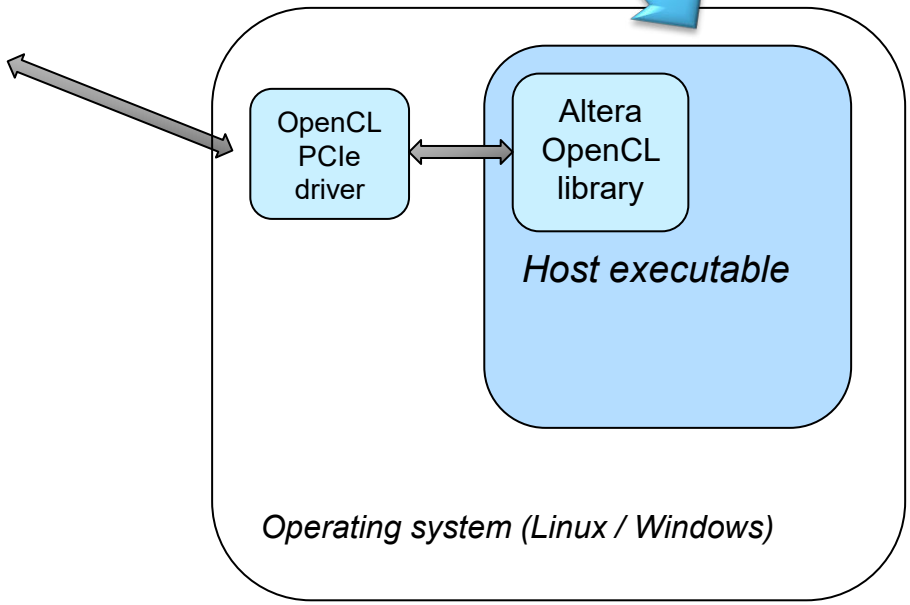
```
__kernel void
sum(__global const float *a,
__global const float *b,
__global float *answer)
{
int xid = get_global_id(0);
answer[xid] = a[xid] + b[xid];
}
```

```
main()
{
   read_data_from_file( … );
   maninpulate_data( … );

   clEnqueueWriteBuffer( … );
   clEnqueueKernel(…, sum, …);
   clEnqueueReadBuffer( … );

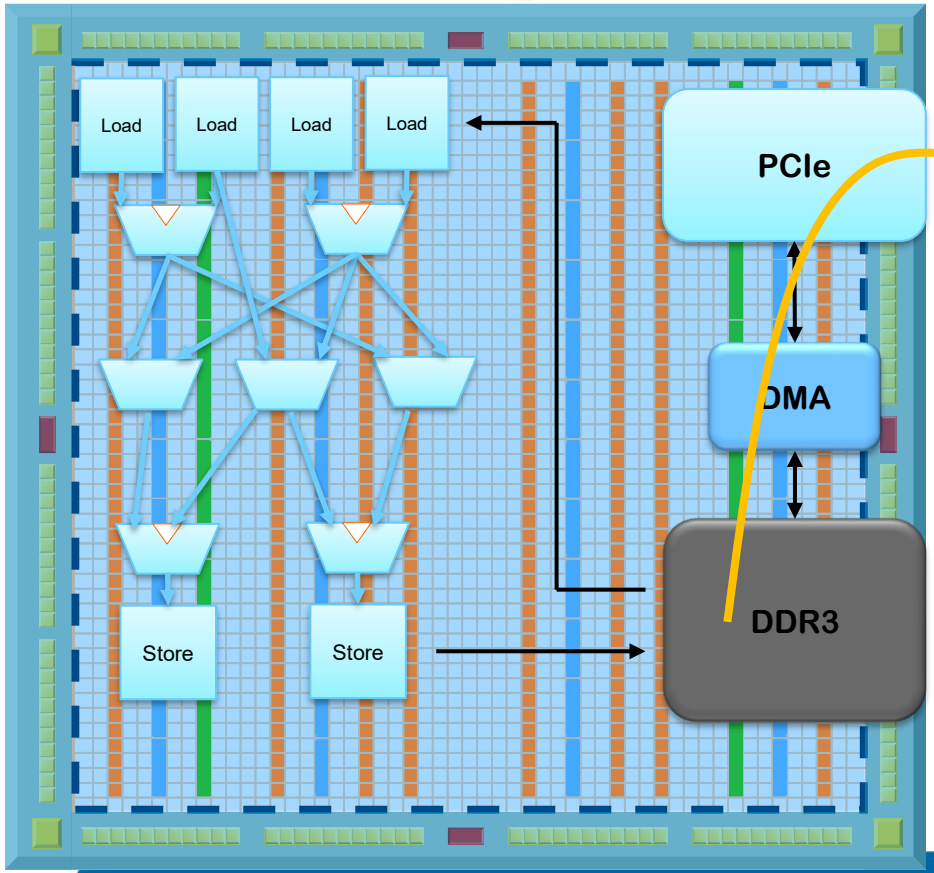
   display_result_to_user( … );
}
```





Processing flow

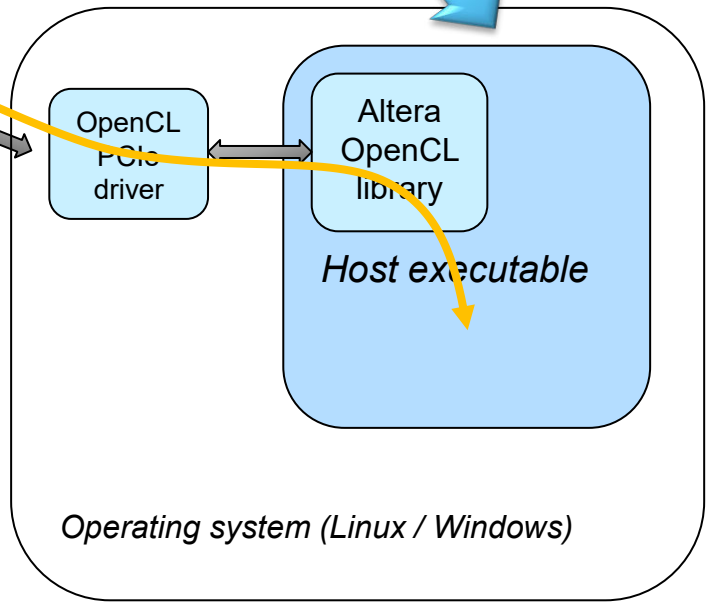
```
__kernel void
sum(__global const float *a,
__global const float *b,
__global float *answer)
{
int xid = get_global_id(0);
answer[xid] = a[xid] + b[xid];
}
```




```
main()
{
   read_data_from_file( … );
   maninpulate_data( … );

   clEnqueueWriteBuffer( … );
   clEnqueueKernel(…, sum, …);
   clEnqueueReadBuffer( … );

   display_result_to_user( … );
}
```



Case Study : F.I.R. Filter



© 2015 Altera Corporation—**Public**

OpenCL Data Parallelism

- **OpenCL kernels expresses parallelism explicitly**

```
for (int i=0; i < n; i++)
{
   answer[i] = a[i] + b[i];
}
```



Host Code

```
setup_memory_buffers();
transfer_data_to_fpga();
size_t global_size = {N, 1, 1};

clEnqueueNDRangeKernel(
sum_kernel, .., &global_size, ..);

read_data_from_fpga();
```

Kernel Code

```
__kernel void
sum(__global const float *a,
__global const float *b,
__global float *answer)
{
int xid = get_global_id(0);
answer[xid] = a[xid] + b[xid];
}
```

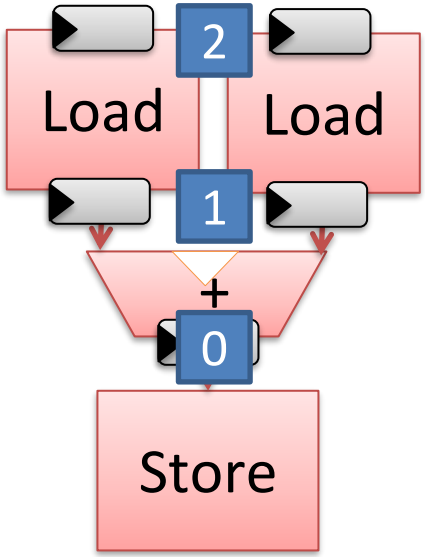
Implicit for loop

© 2015 Altera Corporation—Public

Pipeline Parallelism Implementation

- **On the FPGA, we use the idea of pipeline parallelism to achieve acceleration**

```
__kernel void
sum(__global const float *a,
__global const float *b,
__global float *answer)
{
int xid = get_global_id(0);
answer[xid] = a[xid] + b[xid];
}
```

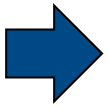


© 2015 Altera Corporation—Public

Parallelizable with Dependencies between Iterations

- **Suited for applications where each loop iteration is independent**

```
for (int i=0; i < n; i++)
{
   answer[i] = a[i] + b[i];
}
```



```
float array[M];

for (int i=0; i < n; i++)
{
  for (int j=0; j < M-1; j++)
      array[j] = array[j+1];
  array[M-1] = a[i];

  for (int j=0; j < M; j++)
    answer[i] += array[j] * coefs[j];
}
```

- **What if each loop iteration depends on the previous iteration?**

This affects all subsequent iterations

© 2015 Altera Corporation—Public

Loop Pipelining

- **To achieve acceleration, we can *pipeline* each iteration of the loop**
  - Analyze any dependencies between iterations
  - Schedule these operations
  - Launch the next iteration as soon as possible

```
float array[M];

for (int i=0; i < n*numSets; i++)
{
  for (int j=0; j < M-1; j++)
      array[j] = array[j+1];
  array[M-1] = a[i];

  for (int j=0; j < M; j++)
    answer[i] += array[j] * coefs[j];
}
```

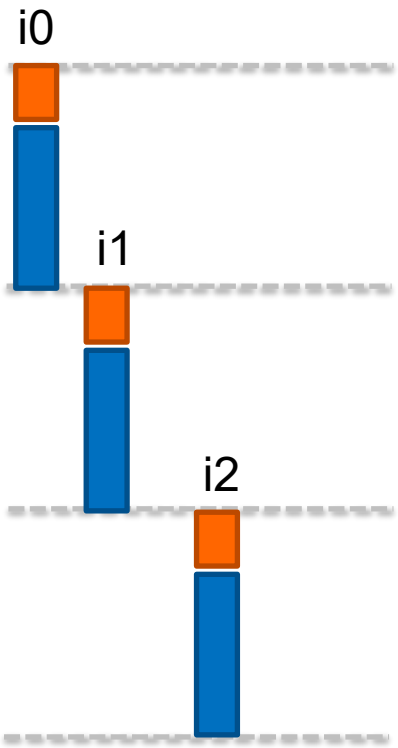


At this point, we can launch the next iteration

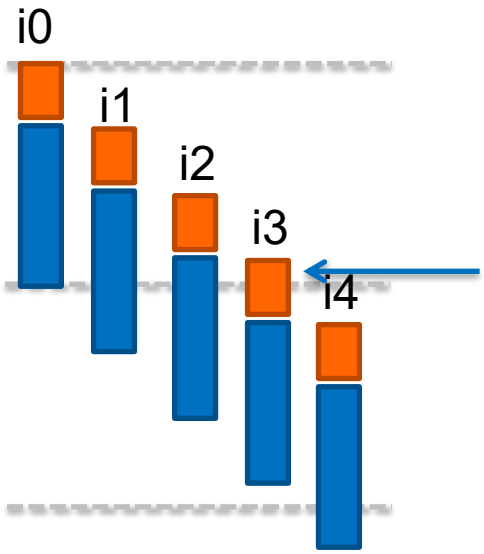
© 2015 Altera Corporation—Public

Loop Pipelining Example

- **No Loop Pipelining**



- **With Loop Pipelining**

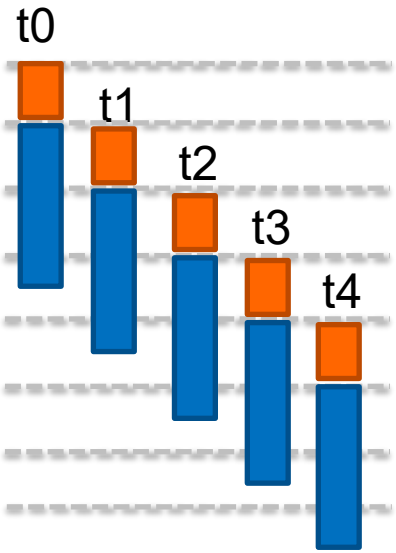


Looks almost like parallel thread execution!

© 2015 Altera Corporation—Public

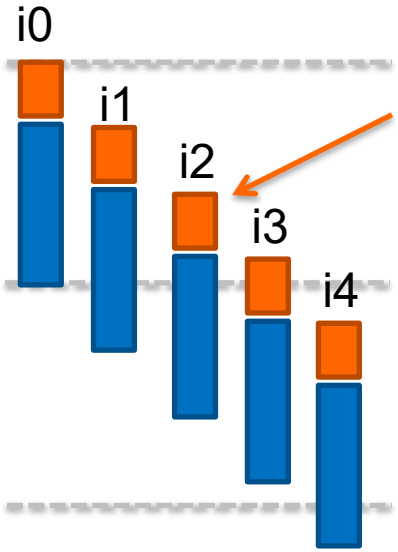
Parallel Threads vs. Loop Pipelining

- **So what's the difference?**



Parallel Threads

Parallel threads launch 1 thread per clock cycle in pipelined fashion



Loop Pipelining

Loop dependencies may not be resolved in 1 clock cycle

- **If the number of clock cycles to resolve loop dependencies is 1, then the resulting computational hardware is exactly the same**
  - With better memory access patterns (makes coalescing possible)

© 2015 Altera Corporation—Public

Advantages of Loop Pipelining

- **Some applications may require loop pipelining to be efficient**
  - Streaming applications



  - When data cannot arrive in parallel
- **Loop Pipelining automatically extracts parallelism within a kernel**
- **No need to figure out workgroup sizes explicitly**
- **May make porting of C-based applications easier**

© 2015 Altera Corporation—Public

OpenCL Task

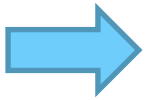
- **Currently expose Loop Pipelining through the task attribute**
  - __attribute(task)

```
setup_memory_buffers();
transfer_data_to_fpga();

clEnqueueTask( sum_kernel,..);

read_data_from_fpga();
```

Host Code



```
__attribute(task)
__kernel void
sum(__global const float *a,
__global const float *coefs,
__global float *answer)
{
  float array[M];

  for (int i=0; i < n*numSets; i++) {
     for (int j=0; j < M-1; j++)
         array[j] = array[j+1];
     array[M-1] = a[i];

     for (int j=0; j < M; j++)
       answer[i] += array[j] * coefs[j];
} }
```

- **Automatically extract parallelism by analyzing loop dependencies**
- **Many C-based algorithms can directly compile to OpenCL Task**

© 2015 Altera Corporation—Public

Case Study #3:
Time Domain FIR Filter (TDFIR)



© 2015 Altera Corporation—Public

Basic Building Block for many Market Segments

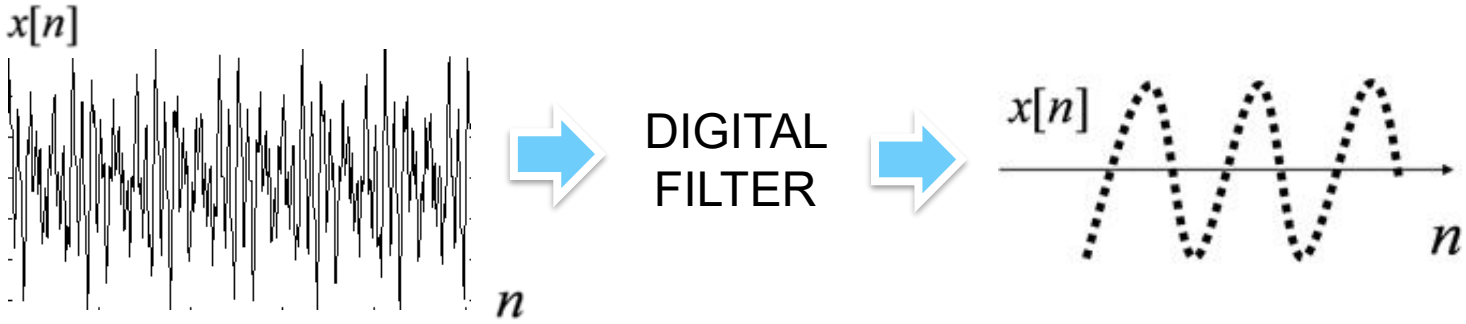


FIR Filters

© 2015 Altera Corporation—Public

Purpose of the FIR Filter

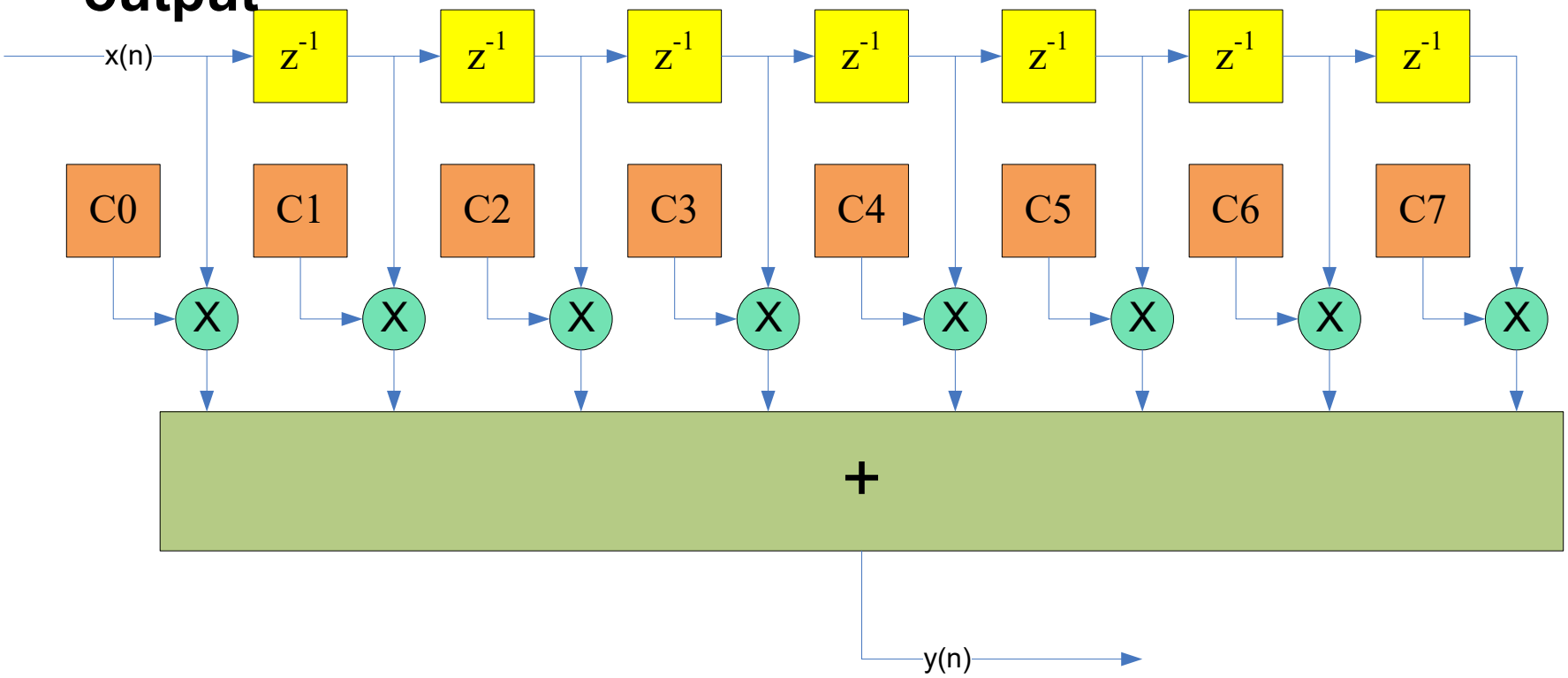
- **A digital equivalent of the analog filter**
- **Allows discrete signals in the time domain to be filtered (remove noise, high-frequency components, etc.)**



© 2015 Altera Corporation—Public

FIR Filter Structure

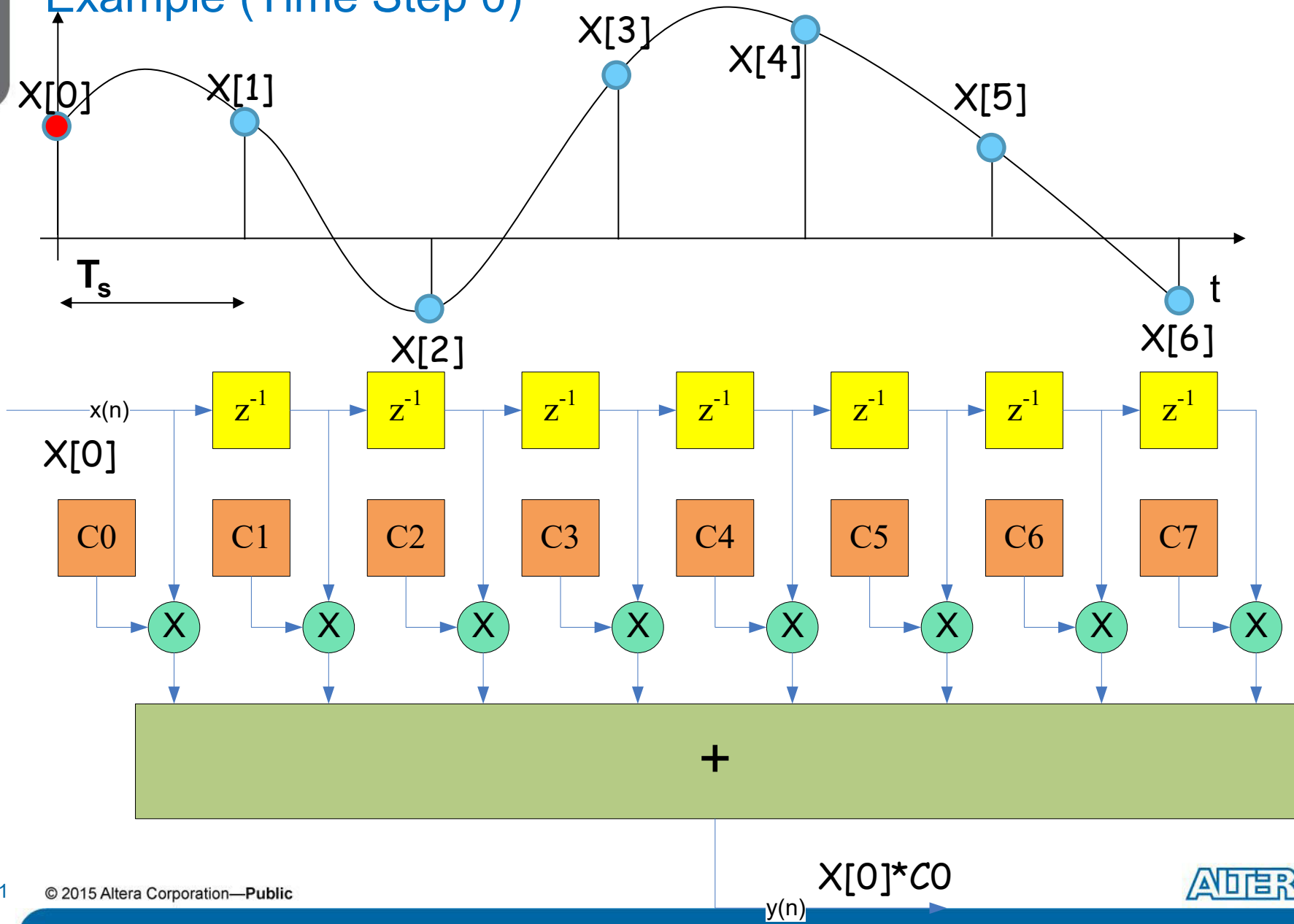
- **Multiply a sequence of input samples by various coefficients and add their results to achieve an output**



- **This series of multiplying and adding will dictate how filter characteristics are shaped**

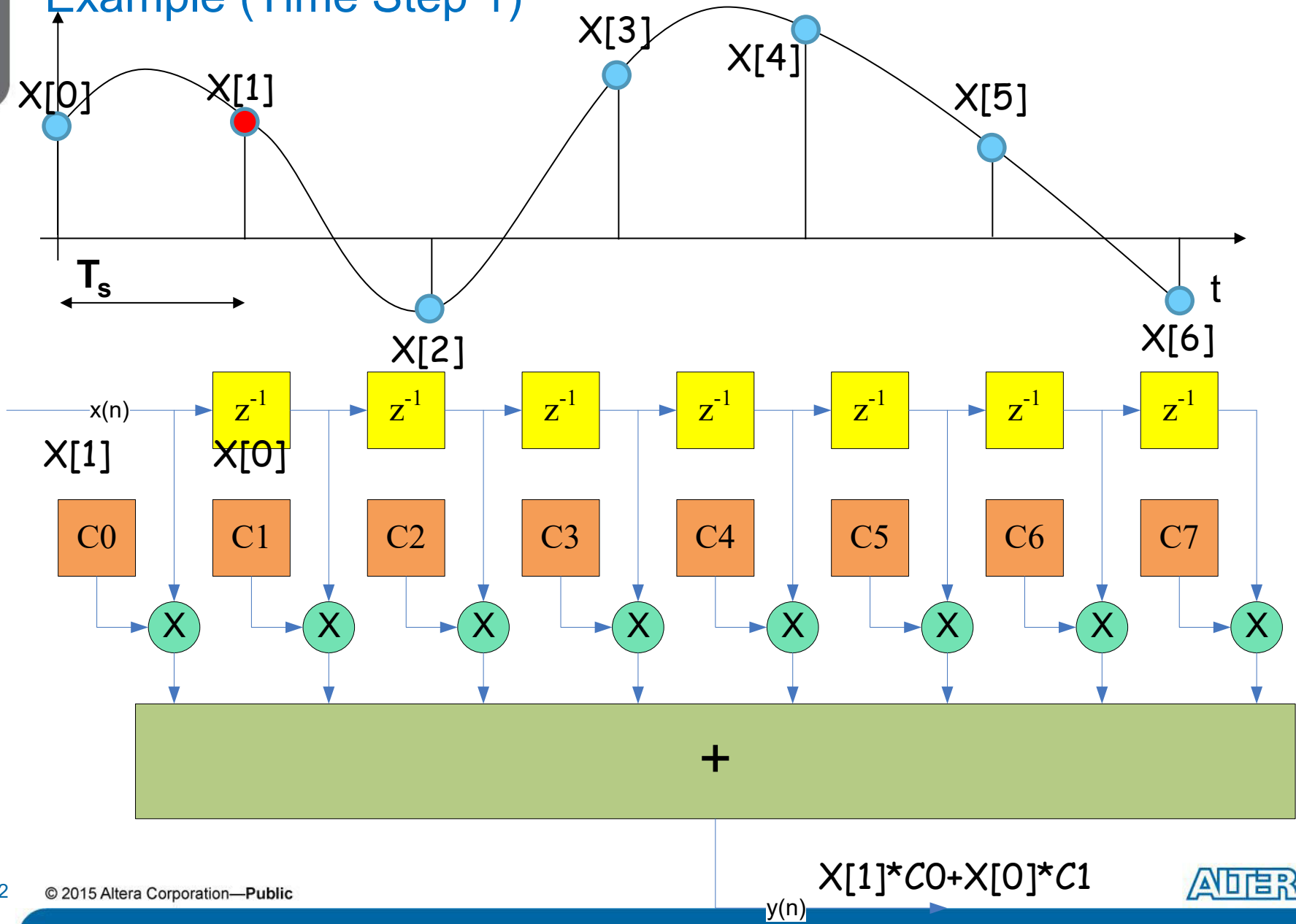
© 2015 Altera Corporation—Public

Example (Time Step 0)



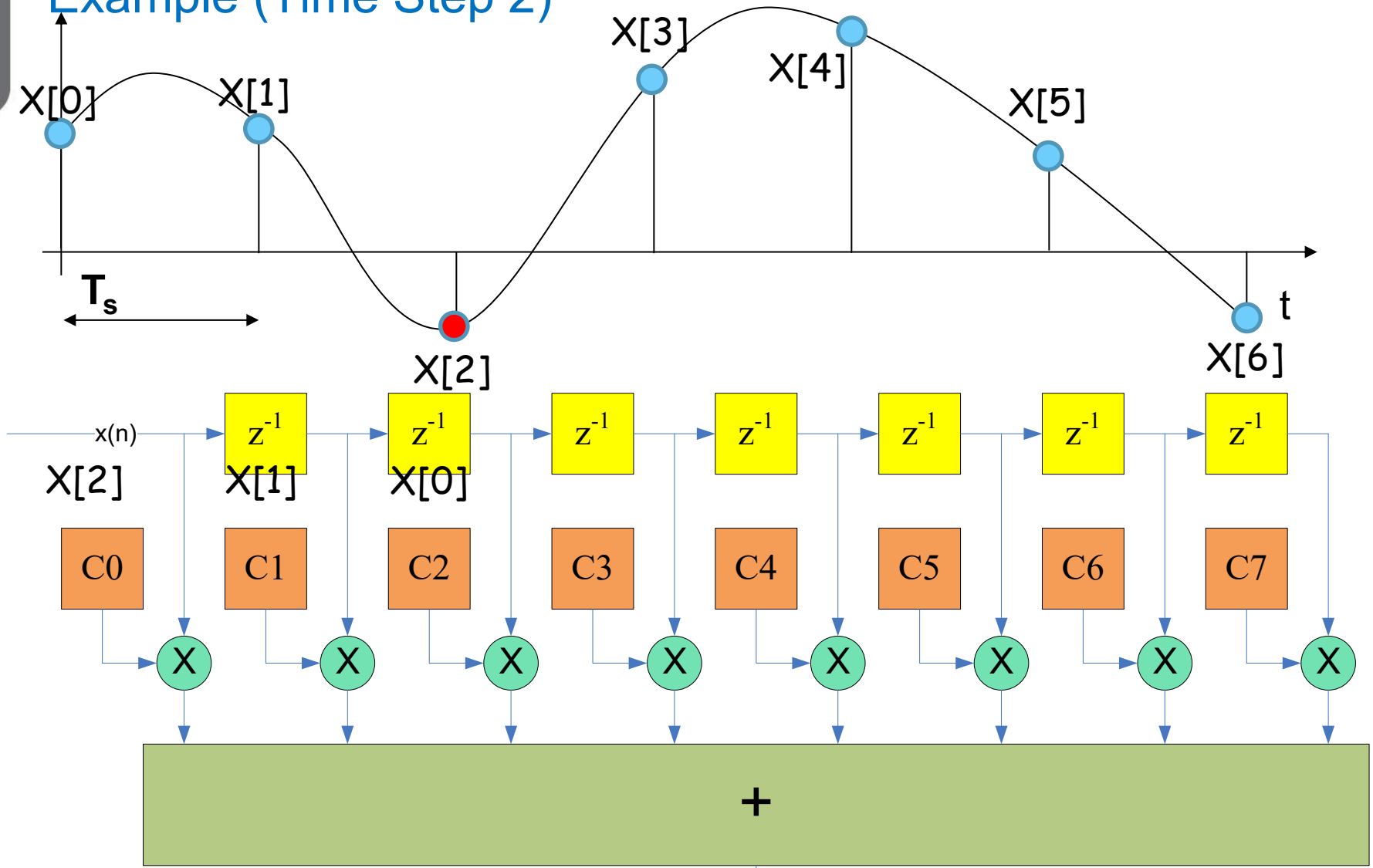
© 2015 Altera Corporation—Public

Example (Time Step 1)



© 2015 Altera Corporation—Public

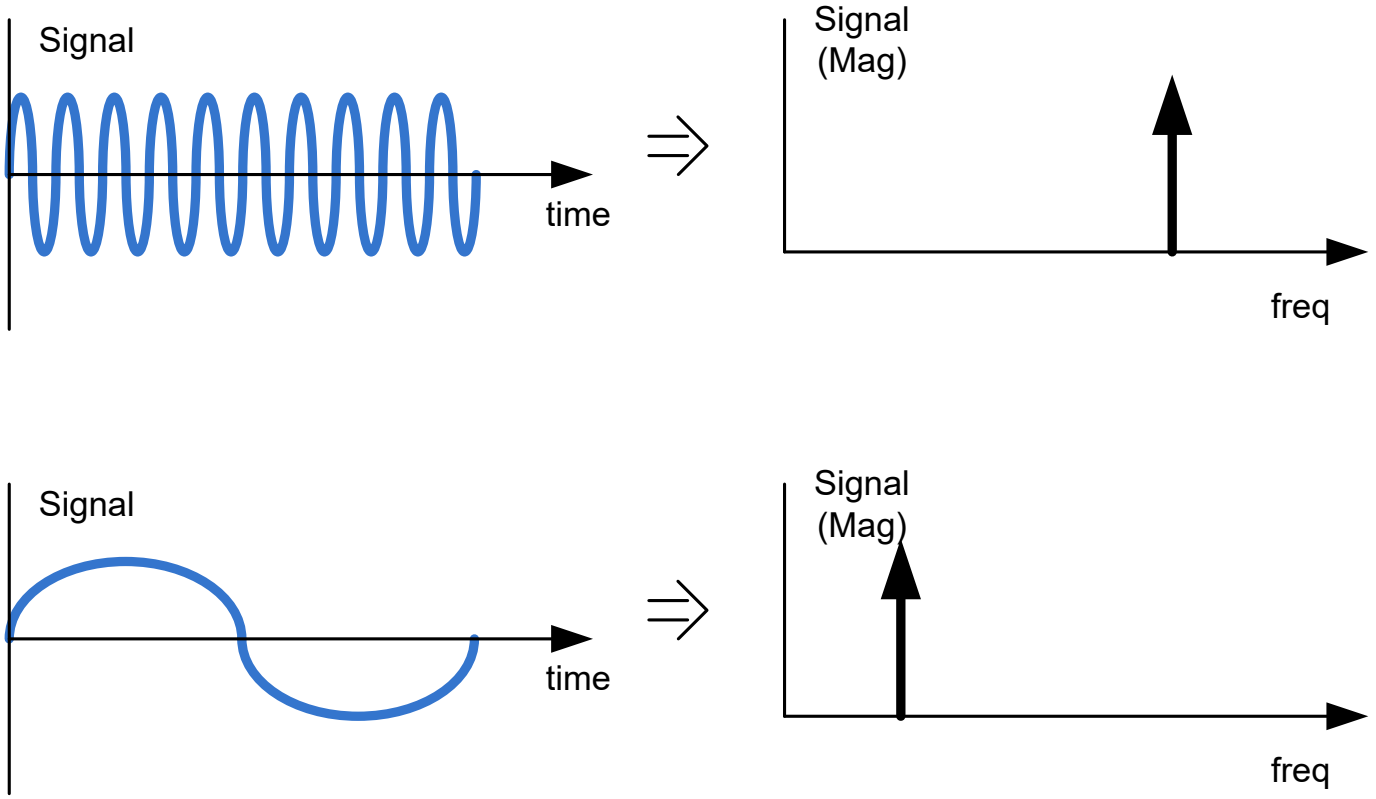
Example (Time Step 2)



© 2015 Altera Corporation—Public

Relationship between Time and Frequency

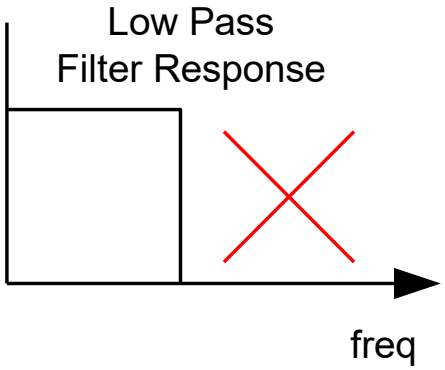
- **What is a Frequency?**
- **Quick Changes in Time = High Frequency**
- **Slow Changes in Time = Low Frequencies**



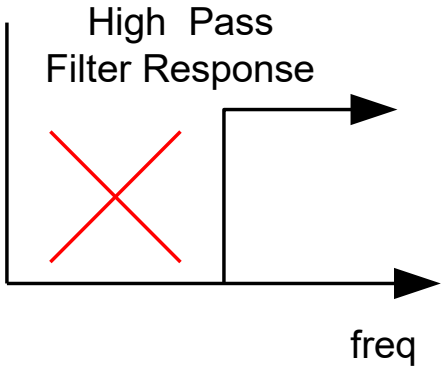
© 2015 Altera Corporation—Public

Example: Types of Filters

- **Low Pass Filters**
  - Pass All Frequencies Up to a Limit Frequency
  - Reject Frequencies Greater Than Limit Frequency



- **High Pass Filters**
  - Pass All Frequencies Above to a Limit Frequency
  - Reject Frequencies Less Than Limit Frequency



© 2015 Altera Corporation—Public

Designing Filters

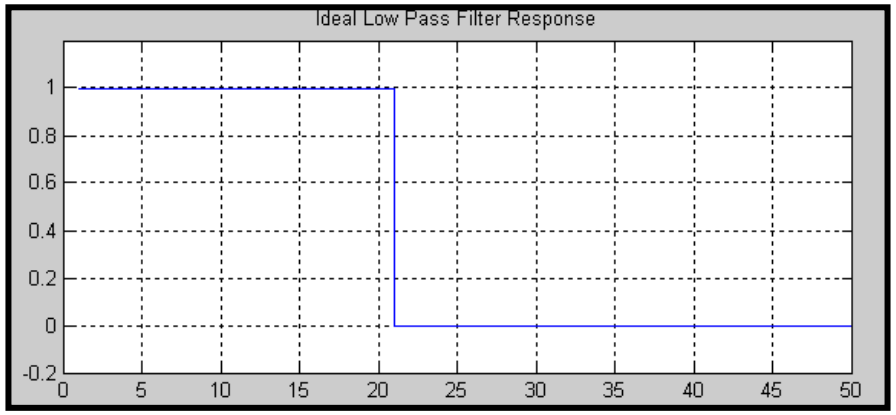
- **Get a desired response in the frequency domain (i.e. : low pass filter)**
- **Modify Desired response to possess certain "desirable" characteristics**
  - I.e. : finite in length (truncate the response)
- **Take the *Inverse Fourier Transform* to obtain the Impulse Response**

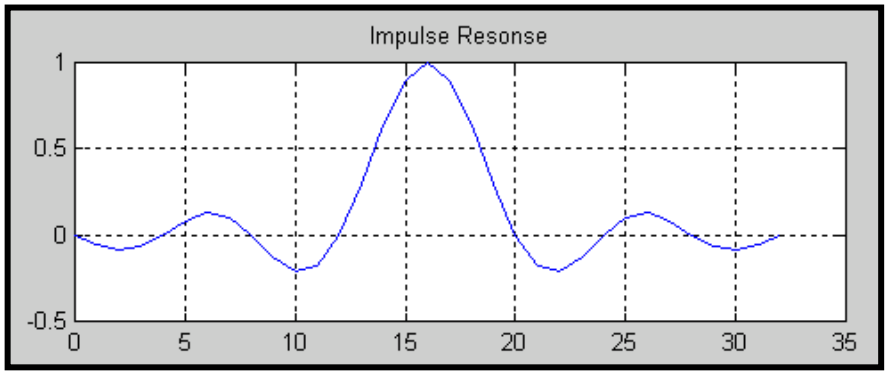
- ***The Impulse Response IS the value of the coefficients***

© 2015 Altera Corporation—**Public**

Filter Examples : Low Pass Filter

- **Low Pass Filter Frequency Response**
  - Start with Ideal Low Pass Filter
- **Take the inverse discrete Fourier Transform**
- **Resulting Filter Coefficients**
  - All Frequencies Up to desired Frequency are present





© 2015 Altera Corporation—Public

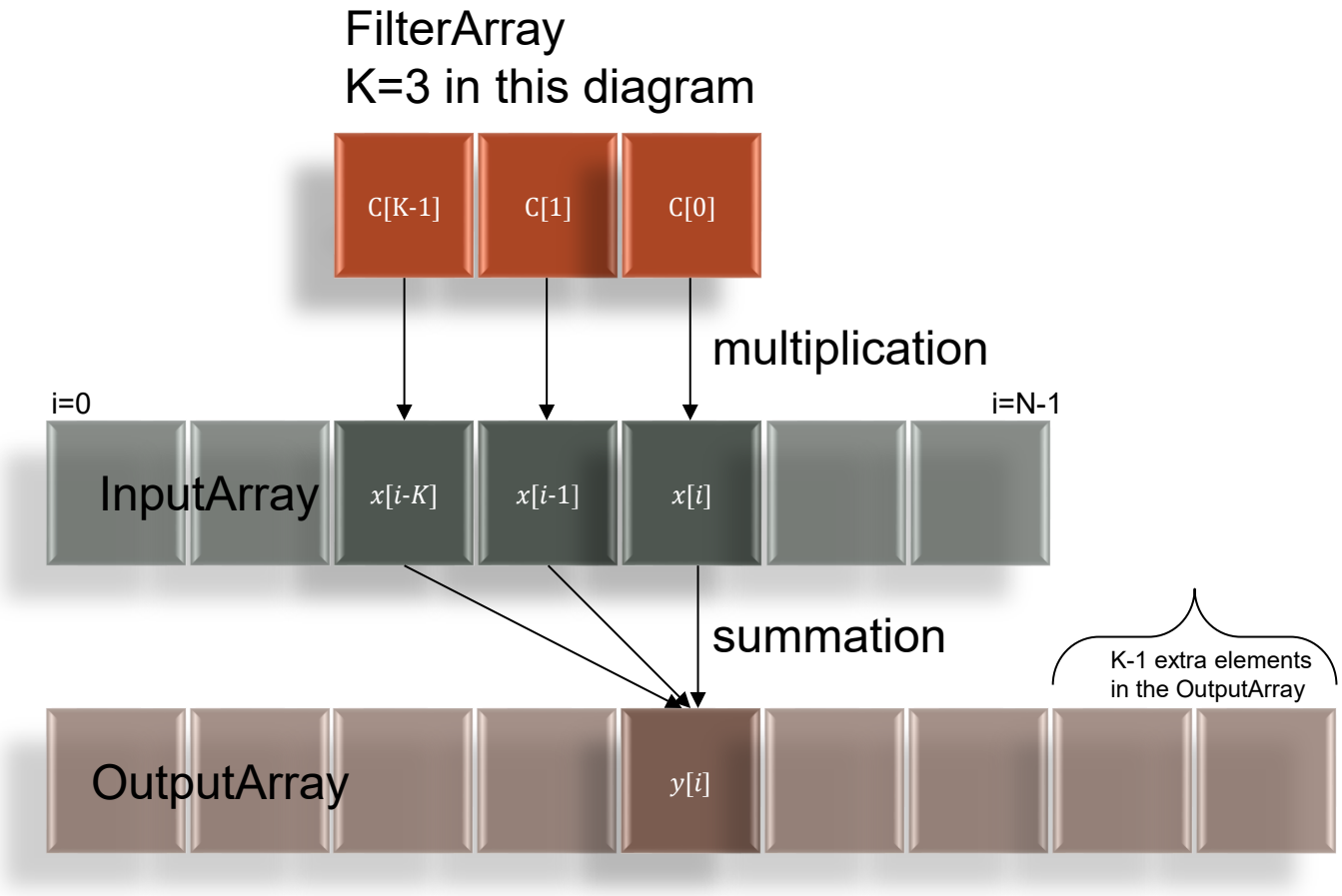
TDFIR HPEC Benchmark

Slides courtesy of Craig Lund



© 2015 Altera Corporation—Public

Graphical Representation



© 2015 Altera Corporation—Public

Benchmark Description

Parameter	Description	Data Set 1	Data Set 2
M	Number of filters to stream	64	20
N	Length of InputArray in complex elements	4096	1024
K	Length of FilterArray in complex elements	128	12
	Workload in MFLOP	268.44	1.97



We need some real-world data sizes if we are going to show results. For that we turn to the RADAR-oriented, HPEC Challenge benchmark suite. See the TDFIR kernel http://www.ll.mit.edu/HPECchallenge/tdfir.html

Their benchmark offers two datasets. The timing data we present in later slides uses Data Set 1.

- Note that you won't find direct convolution implementations optimized for giant arrays. It is faster to use an FFT algorithm in that case. Using an FFT for that is benchmarked using hpecChallenge's FDFIR kernel.

© 2015 Altera Corporation—Public

Using HPEC Challenge Data

```
% TDFIR comes with a file containing separate input data for each filter.
% The input data provided for each filter is exactly the same.
% The benchmark implementation ignores that fact and reads in all the copies.

% For this illustration we assume the data for each filter was read into rows
% and a separate row used for each filter
for f=1:M
  OutputStream(f,:) = conv (InputStream(f,:), FilterStream(f,:));
end
```

- **This slide illustrates how the HPEC Challenge tdFIR benchmark uses the data that it supplies, expressed in MATLAB.**
- **In words, the benchmark presents a batch of M separate filters to compute.**

© 2015 Altera Corporation—Public

Simple C Implementation

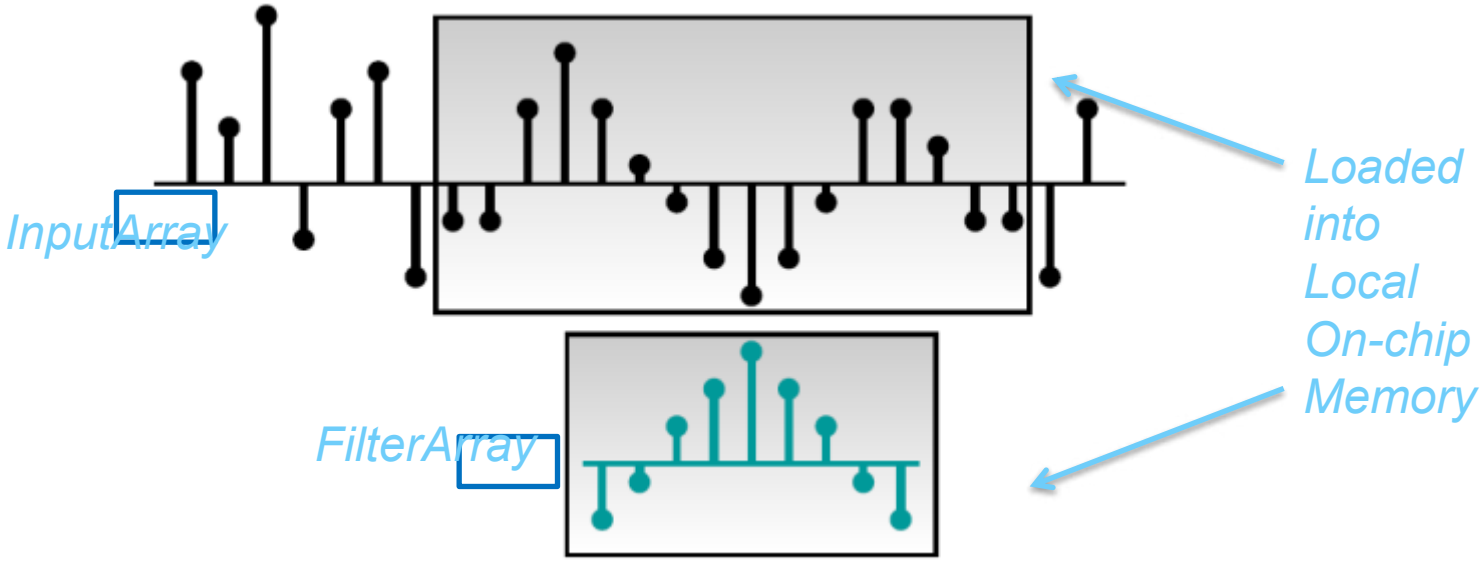
- **The obvious implementation in C is very simple.**
  - Often “good enough.”
- **This implementation exactly matches the fundamental equation of the FIR filter.**
- **Upcoming slides show alternative implementation methods that lead to an efficient FPGA realization.**

```
// Loop assumes OutputArray starts out full of zeros.
for ( int FilterElement = 0; FilterElement < K; FilterElement++ )
{ for ( int InputElement = 0; InputElement < N; InputElement++ )
  { OutputArray [ InputElement + FilterElement ]
      += InputArray [ InputElement ] * FilterArray[ FilterElement ];
  }
}
```

© 2015 Altera Corporation—Public

Traditional OpenCL Implementation

- **Accessing global memory is slow, so a more efficient implementation involves breaking the problem up into smaller segments that can fit in *local memory***
- **The *InputArray* needs to be broken up into smaller pieces and the convolution of each of these subregions can be computed independently**



© 2015 Altera Corporation—Public

Simplified Code Structure

```
// Hardcode these for efficiency
#define N 4096
#define K 128

__kernel void tdFIR
  ( __global const float *restrict InputArray, // Length N
    __global const float *restrict FilterArray, // Length K
    __global float *restrict OutputArray // Length N+K-1
  )
{
   __local float local_copy_input_array[2*K+N];
   __local float local_copy_filter_array[K];

   InputArray += get_group_id(0) * N;
   FilterArray += get_group_id(0) * K;
   OutputArray += get_group_id(0) * (N+K);

   // Copy from global to local
   local_copy_input_array[get_local_id(0)] = InputArray[get_local_id(0)];
   if (get_local_id(0) < K)
      local_copy_filter_array[get_local_id(0)] = FilterArray[get_local_id(0)];
   barrier(CLK_LOCAL_MEM_FENCE);

   // Perform Compute
   float result=0.0f;
   for (int i=0; i<K; i++) {
     result += local_copy_of_filter_array[K-1-i]*local_copy_of_input_array[get_local_id(0)+i];
   }
   OutputArray[get_local_id(0)] = result;
}
```

Ignoring Complex Math for simplicity

© 2015 Altera Corporation—Public

Not the most efficient for FPGA

- **Consider if we unrolled the compute loop**

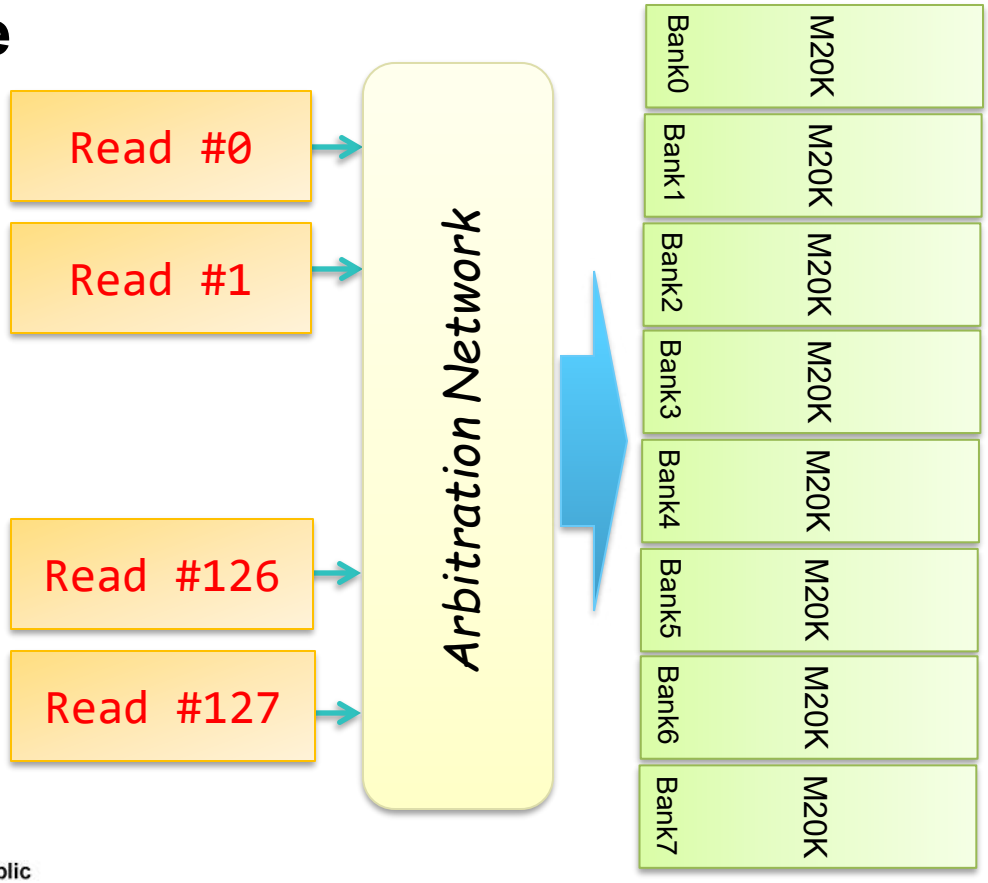
```
result = local_copy_of_filter_array[K-1]*local_copy_of_input_array[get_local_id(0)] +
          local_copy_of_filter_array[K-2]*local_copy_of_input_array[get_local_id(0)+1] +
          local_copy_of_filter_array[K-3]*local_copy_of_input_array[get_local_id(0)+2] +
          ...+
          local_copy_of_filter_array[0]*local_copy_of_input_array[get_local_id(0)+K-1];
```

- **We have something that looks somewhat like a good circuit for a FIR filter**
- **Several loads from memory fetch 128 coefficients from the *local FilterArray* and 128 elements from the *local InputArray***

© 2015 Altera Corporation—Public

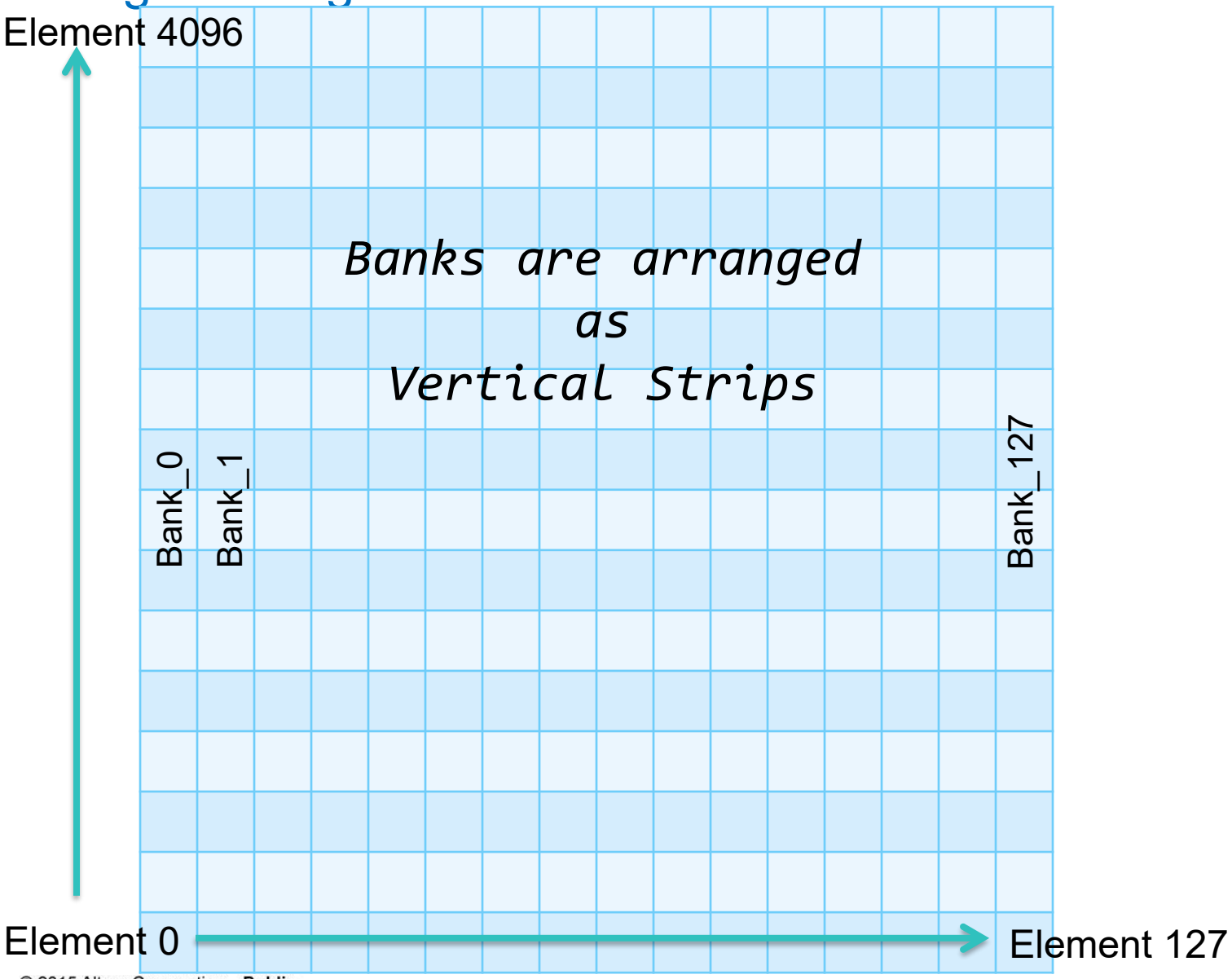
Reading from a banked Memory

- **On any given clock cycle, as long as all requests are asking for elements in different banks → we can service ALL requests!**
- **Altera's OpenCL compiler automatically builds this structure**



© 2015 Altera Corporation—Public

Using Banking to service all Reads



© 2015 Altera Corporation—Public

Disadvantages

- **Banked local memory structures are an inefficient way to handle FIR data reuse.**
  - Consumes LOTs of area and on-chip logic and memory resources

- ***Notice that every thread accesses almost the same data, but shifted by one position***
  - We really need to create a shift register structure!
  - It is the ultimate form of expressing the data reuse pattern for the FIR filter

© 2015 Altera Corporation—Public

Back to Loop Pipelining

```
// Hardcode these for efficiency
#define N 4096
#define K 128
 (HZ: only good for FGA implementation. User are responsible to convert DLP implementation to
Loop level parallelism (LLP) implementation.)
__kernel void tdFIR
  ( __global const float *restrict InputArray, // Length N
    __global const float *restrict FilterArray, // Length K
    __global float *restrict OutputArray, // Length N+K-1
    unsigned int totalLengthOfInputsConcatenated
  )
{
  float data_shift_register[K];
  for (int i=0; i<totalLengthOfInputsConcatenated; i++) {
     #pragma unroll
     for (int j=0; j<K; j++)
        data_shift_register[j]=data_shift_register[j+1];
     data_shift_register[K-1]=InputArray[i];

     float result=0.0f;
     #pragma unroll
     for (int j=0; j<K; j++)
        result += data_shift_register[K-1-j]*FilterArray[j];

     OutputArray[i] = result;
  }
}
```

Shift register implementation

For Simplicity Assume the FilterArray doesn't change

© 2015 Altera Corporation—Public

Consider the unrolled loops … Again

- **The first unrolled loop looks like:**

```
data_shift_register[0]=data_shift_register[1];
data_shift_register[1]=data_shift_register[2];
data_shift_register[2]=data_shift_register[3];
...
data_shift_register[K-1]=InputArray[i];
```

- **The second unrolled loop:**

```
result = data_shift_register[K-1]*FilterArray[0] +
           data_shift_register[K-2]*FilterArray[1] +
           data_shift_register[K-3]*FilterArray[2] +
           ...
           data_shift_register[0]*FilterArray[K-1];
```

- ***The key observation is that all array accesses are from a constant address***
  - Altera's OpenCL compiler will now build 128 floating point registers instead of a complex memory system

© 2015 Altera Corporation—Public

Loop Pipelining at Work

Iteration I	Iteration i+1
data_shift_register[0]	data_shift_register[0]
data_shift_register[1]	data_shift_register[1]
data_shift_register[2]	data_shift_register[2]
data_shift_register[3]	data_shift_register[3]
data_shift_register[4]	data_shift_register[4]
...	...
data_shift_register[K-1]	data_shift_register[K-1]

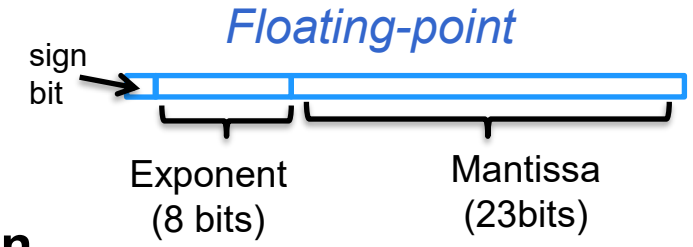


- Pipelining iterations in this loop is very simple because the dependencies are trivial
- Essentially becomes a shift register in hardware.

© 2015 Altera Corporation—Public

We handle Floating-Point !

- **Notice that the TDFIR is a Floating Point Application**
- **Floating-point formats represent a large range of real numbers with a finite number of bits**



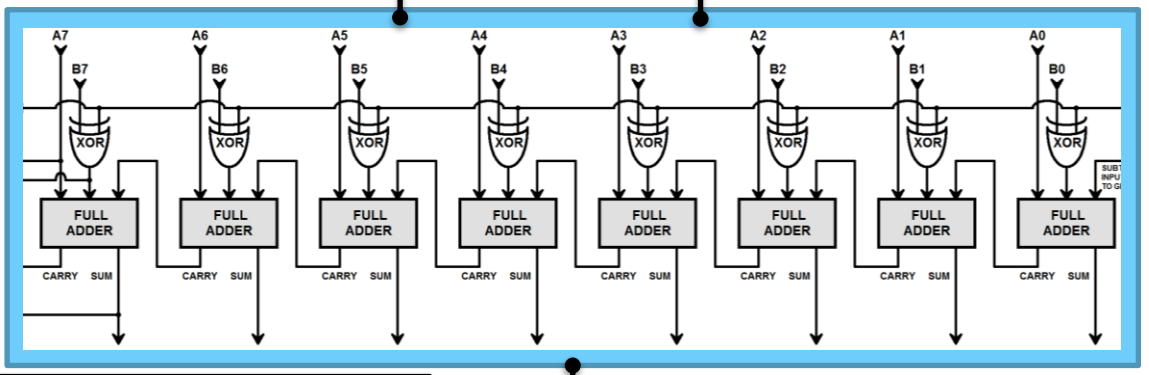
- **Extreme care is required to handle OpenCL compliant floating point on the FPGA**
  - Handling of Infs, NaNs
  - Denormalized Numbers
  - Rounding

© 2015 Altera Corporation—Public

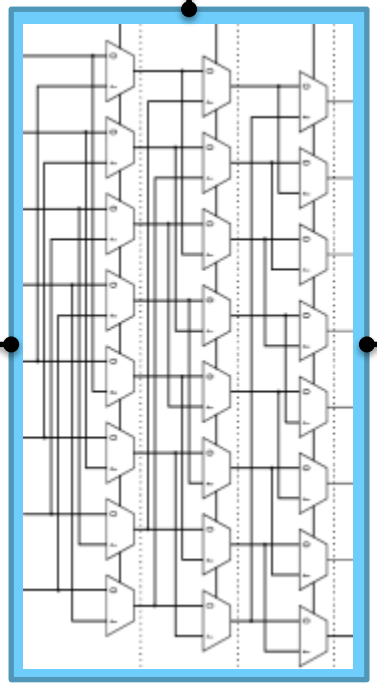
Example: Floating Point Addition

0	0x19	0x12345
0	0x15	0x54321
0	0x19	0x12345
0	0x19	0x05432
0	**0x19**	**0x17777**

Subtractor



4

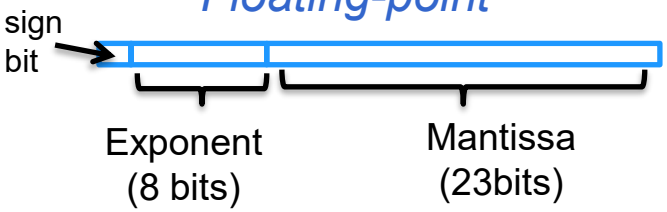


0x05432

Barrel Shifter

- Only a subset of circuitry shown
- Circuitry required for rounding modes, normalization, etc

Floating-point



© 2015 Altera Corporation—Public

OpenCL builds on Altera's Floating Point Technology

Function	ALUTs	Register	Multipliers (27x27)	Latency	Performance
Add/Subtract	541	611	n/a	14	497 MHz
Multiplier	150	391	1	11	431 MHz
Divider	254	288	4	14	316 MHz
Inverse	470	683	4	20	401 MHz
SQRT	503	932	n/a	28	478 MHz
Inverse SQRT	435	705	6	26	401 MHz
EXP	626	533	5	17	279 MHz
LOG	1,889	1,821	2	21	394 MHz

We can implement the full range of OpenCL math functions

© 2015 Altera Corporation—Public

Results of Task TDFIR

```
Num platforms = 1
Platform Chosen : Altera Corporation
Found 1 devices.
Loading tdfir.aocx...
Kernel Compilation Time: 0.562906 ms
tdFirVars->inputLength = 4096, resultLength =
4223,filterLen = 128
Done.  Latency: 0.001418 s.
      Verification: PASS
```

- **FPGA execution time corresponds to 189 GFLOP/s on BittWare S5-PCIe-HQ (S5PH-Q) board**



© 2015 Altera Corporation—Public

OpenCL Overhead

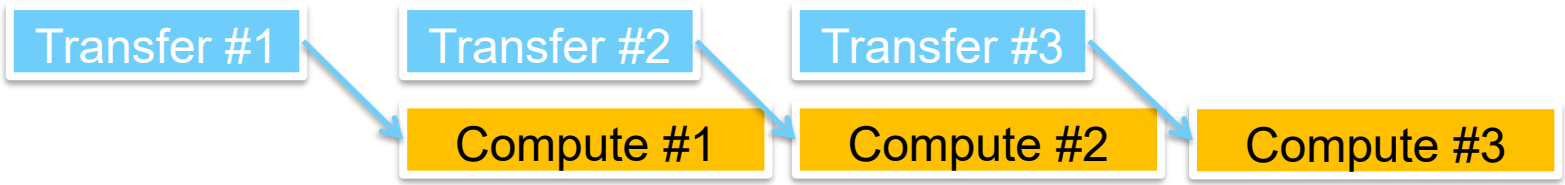
Dataset	M	N	K	Input Bytes	Filter Bytes	Output Bytes	Total MB transferred	Transfer Time (ms)
Large	64	4096	128	2097152	65536	2162176	4.35 MB	1.36
Small	20	1024	12	163840	1920	165600	0.36 MB	0.11

- **OpenCL is usually associated with PCIe attached acceleration hardware.**
- **The DMA of data over PCIe consumes time.**
- **Transfer Time is the amount of time to transfer all of the data back and forth to the FPGA using PCIe Gen2x8.**

© 2015 Altera Corporation—Public

Can overlap transfers and computation

- **For the most common case in HPEC, we likely want to continuously process batches of filters as time progresses.**
- **Can overlap transfer and compute.**



- **This pipelining approach can be used to hide transfer latency**
- **(HZ: done at the CPU side of code)**

© 2015 Altera Corporation—Public

Other Optimizations

- The *FilterArray* changes with every block of data to be filtered.
- Rather than constantly accessing these coefficients from memory, we shift in these coefficients to *coef_shift_register* structure at the beginning of every block

```
__kernel void tdFIR
...
{
  float data_shift_register[K];
  float coef_shift_register[K];
  int load_filter = 1;
  for (int i=0; i<totalLengthOfInputsConcatenated; i++) {
     if (load_filter) {
        #pragma unroll
        for (int j=0; j<K; j++)
           coef_shift_register[j]=coef_shift_register[j+1];
        coef_shift_register[K-1]=FilterArray[...];
        // Set load_filter to 0 after reading K coefficients
        ...
     }
   }
}
```


© 2015 Altera Corporation—Public

Comparisons



© 2015 Altera Corporation—Public

Evaluating the Potential of Graphics Processors for High Performance Embedded Computing

Shuai Mu[1], Chenxi Wang[1], Ming Liu[2], Dongdong Li[2], Maohua Zhu[1], Xiaoliang Chen[3], Xiang Xie[1], Yangdong Deng[1]

[1]Tsinghua University, [2]BeiHang University, [3]Chinese Academy of Sciences

{mus04ster, zhumaohua, yangdong.deng, lmingcsce}@gmail.com, wcx09@mails.tsinghua.edu.cn, xiexiang@tsinghua.org.cn,

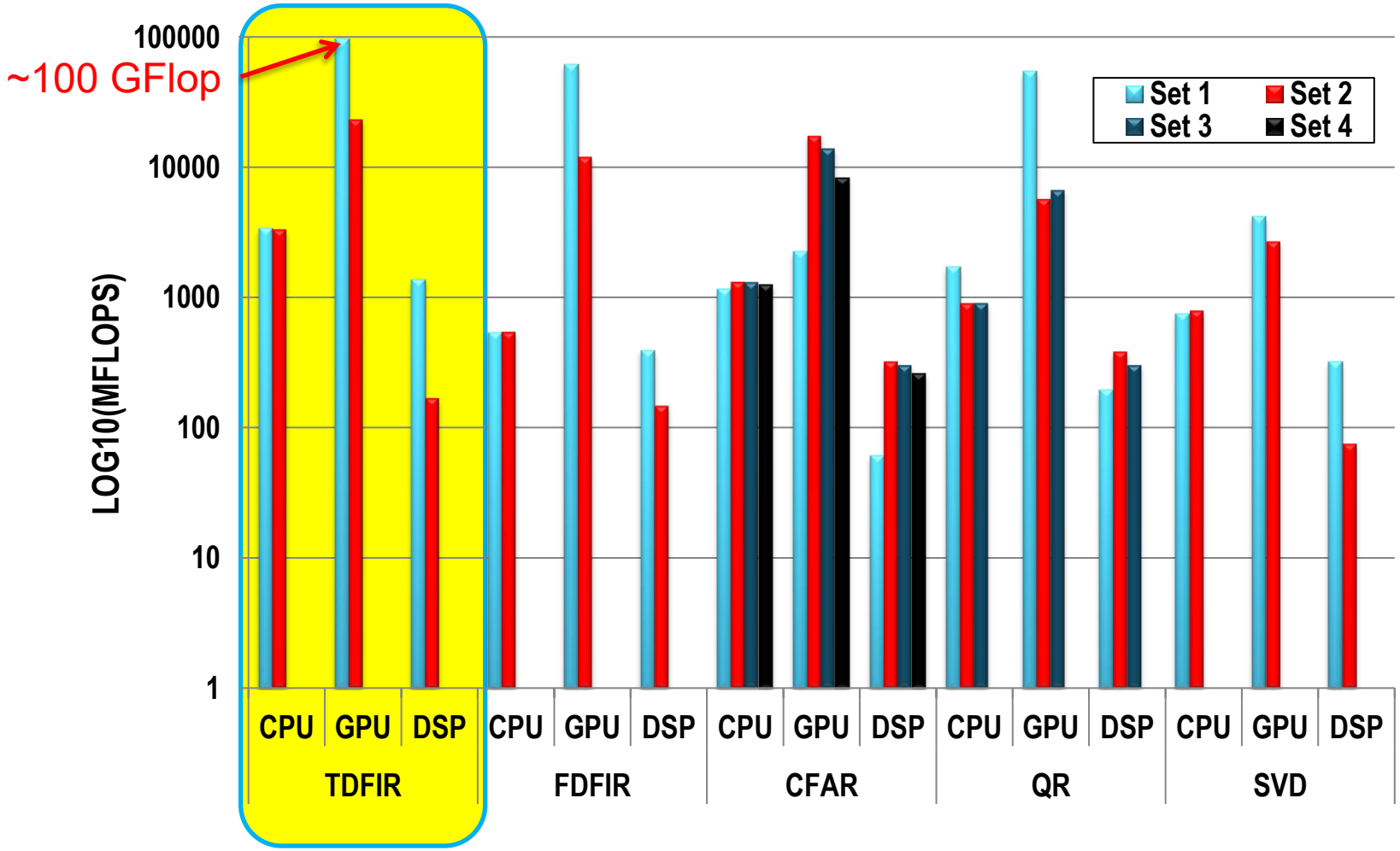
Kernels	Data Set	CPU Throughput (GFLOPS) *	GPU Throughput (GFLOPS) *	Speedup
TDFIR	Set 1	3.382	97.506	28.8
	Set 2	3.326	23.130	6.9
FDFIR	Set 1	0.541	61.681	114.1
	Set 2	0.542	11.955	22.1
CT	Set 1	1.194	17.177	14.3
	Set 2	0.501	35.545	70.9
PM	Set 1	0.871	7.761	8.9
	Set 2	0.281	21.241	75.6
CFAR	Set 1	1.154	2.234	1.9
	Set 2	1.314	17.319	13.1
	Set 3	1.313	13.962	10.6
	Set 4	1.261	8.301	6.6
GA	Set 1	0.562	1.177	2.1
	Set 2	0.683	8.571	12.5
	Set 3	0.441	0.589	1.4
	Set 4	0.373	2.249	6.0
QR	Set 1	1.704	54.309	31.8
	Set 2	0.901	5.679	6.3
	Set 3	0.904	6.686	7.4
SVD	Set 1	0.747	4.175	5.6
	Set 2	0.791	2.684	3.4
DB	Set 1	112.3	126.8	1.13
	Set 2	5.794	8.459	1.46

© 2015 Altera Corporation—Public

Performance Comparison

HZ: currently FPGAs have floating point units, much easier to verify than fixed point arithmetic.

- **GPU: NVIDIA Fermi, CPU: Intel Core 2 Duo (3.33GHz), DSP AD TigerSHARC 101**

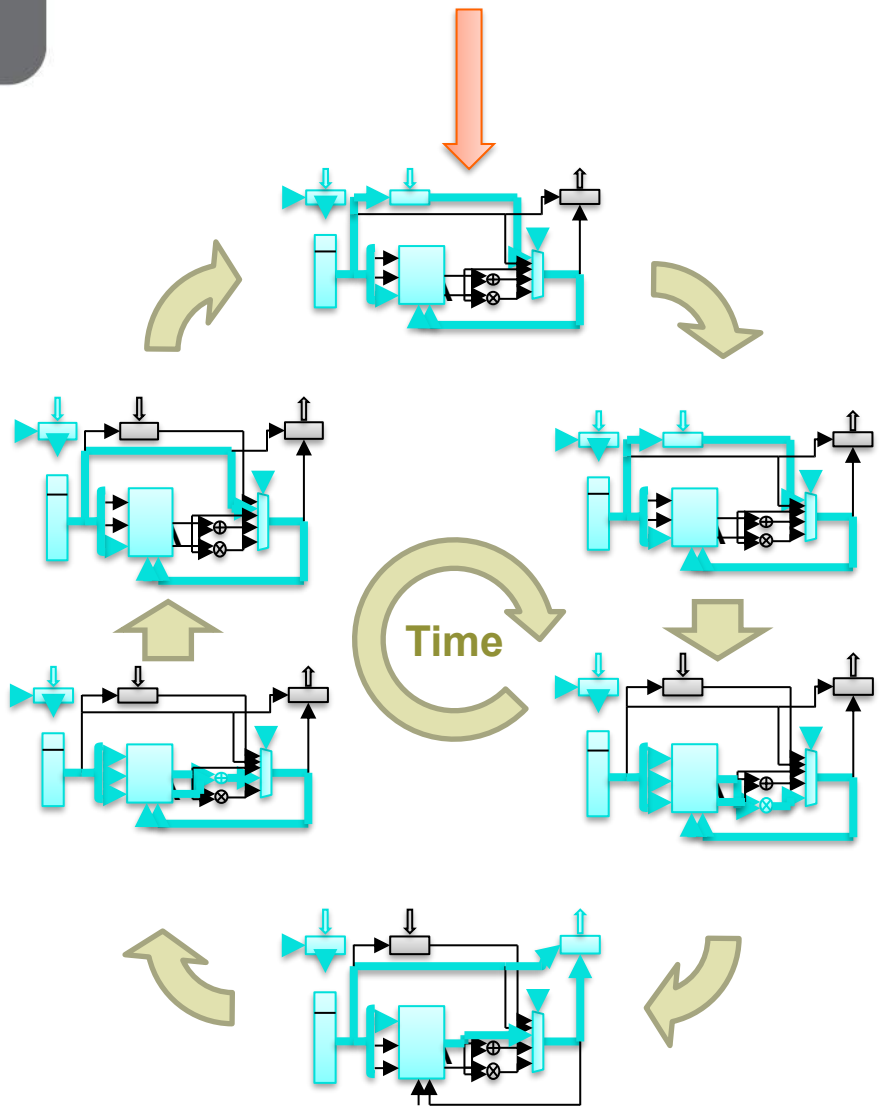


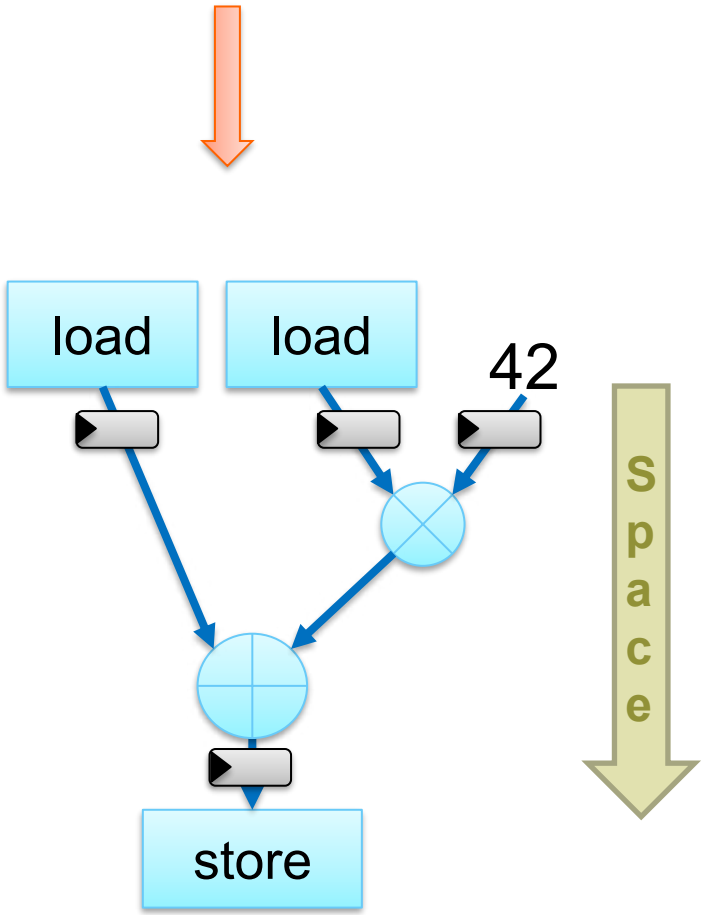
© 2015 Altera Corporation—Public

Case Study : Image Convolution

© 2015 Altera Corporation—Public

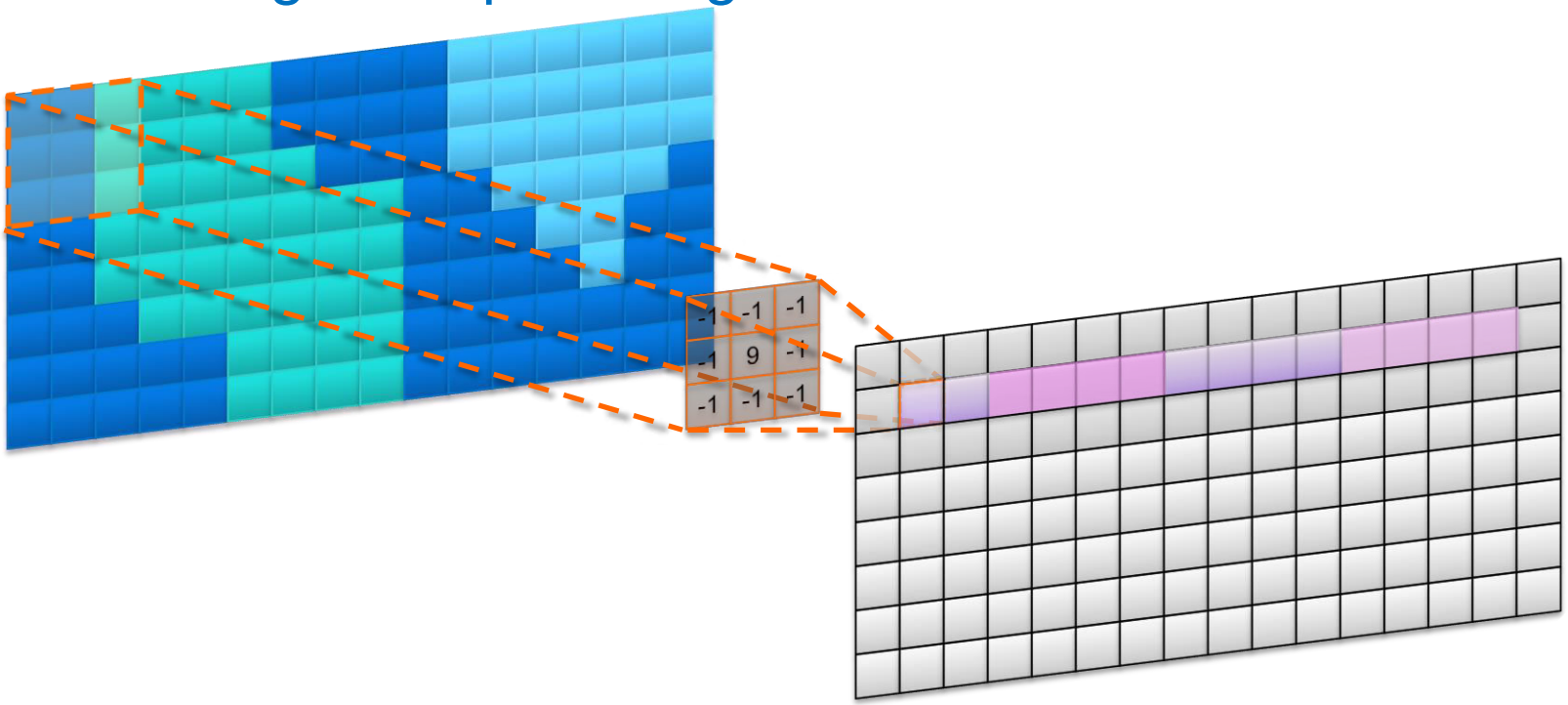
Sequential CPU Architecture vs. Dataflow Architecture





© 2015 Altera Corporation—Public

Motivating Example: Image Convolution Filters

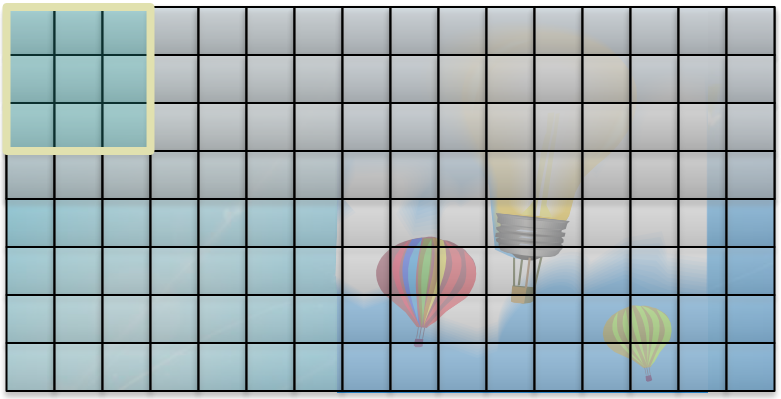


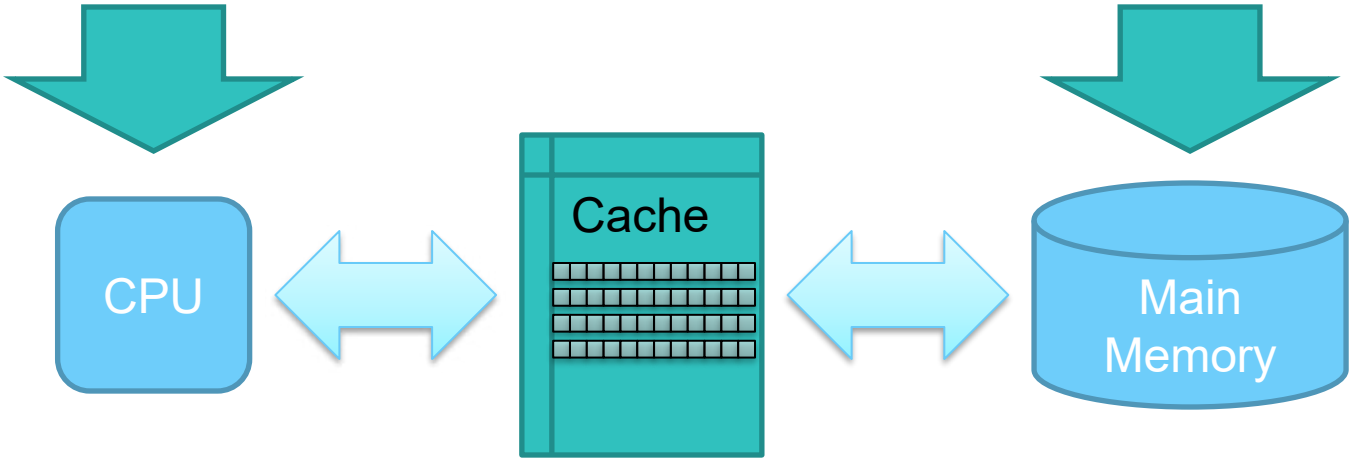
$$\mathrm{I}_{\mathrm{new}}[x][y] = \sum_{x'=-1}^{1} \sum_{y'=-1}^{1} \mathrm{I}_{\mathrm{old}}[x+x'][y+y'] \times \mathrm{F}[x'][y']$$

© 2015 Altera Corporation—Public

CPU/GPU Implementation

```
for(int y=1; y<height-1; ++y) {
  for(int x=1; x<width-1; ++x) {
    for(int y2=-1; y2<1; ++y2) {
      for(int x2=-1; x2<1; ++x2) {
        i2[y][x] += i[y+y2][x+x2]
                  * filter[y2][x2];
```



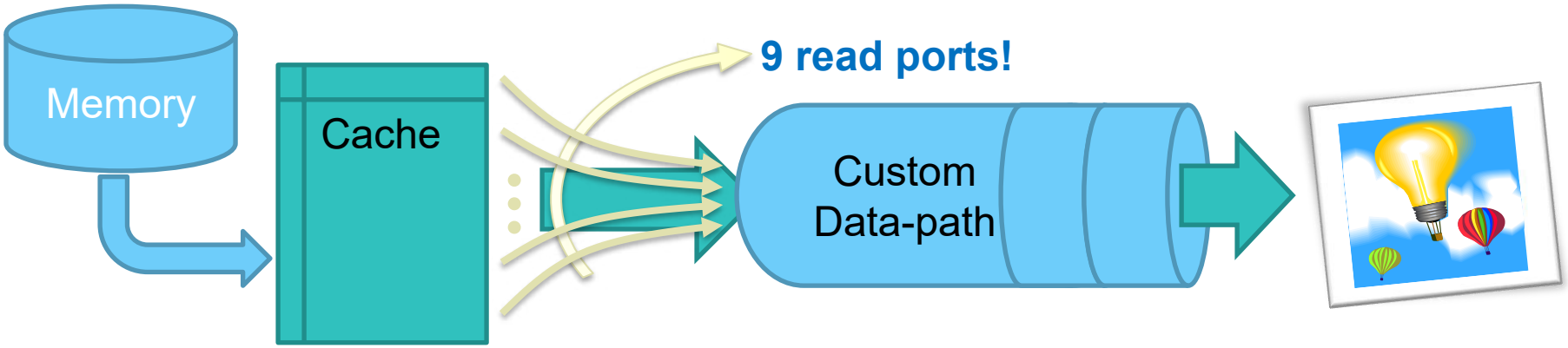


- A cache can hide poor memory access patterns

© 2015 Altera Corporation—Public

Cache Design Requirements

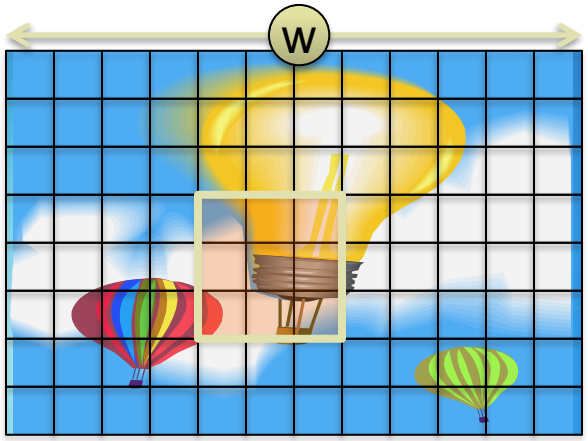
- Performance target: 1 pixel per cycle
- Cache requirements: 9 reads + 1 write per cycle



- Expensive hardware!
  - Power overhead
  - Cost overhead: More built in addressing flexibility than we need
- Why not customize the cache for the application?

© 2015 Altera Corporation—Public

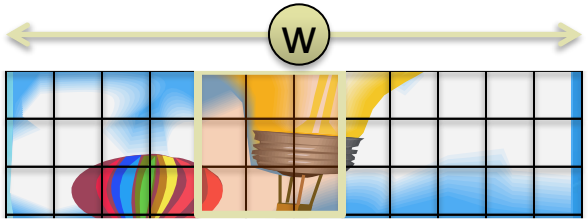
Optimizing the “Cache”



- Start out with the initial picture that is W pixels wide

© 2015 Altera Corporation—Public

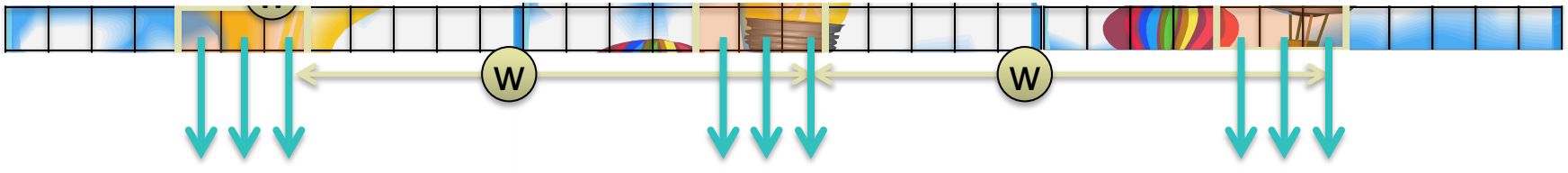
Optimizing the “Cache”



- Let’s remove all the lines that aren’t in the neighborhood of the window

© 2015 Altera Corporation—Public

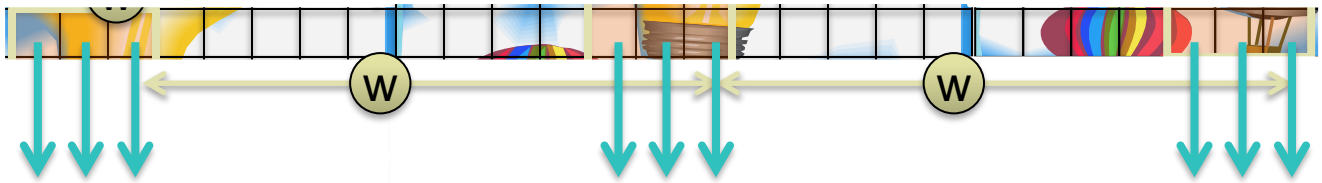
Optimizing the “Cache”



- Take all of the lines and arrange them as a 1D array of pixels

© 2015 Altera Corporation—Public

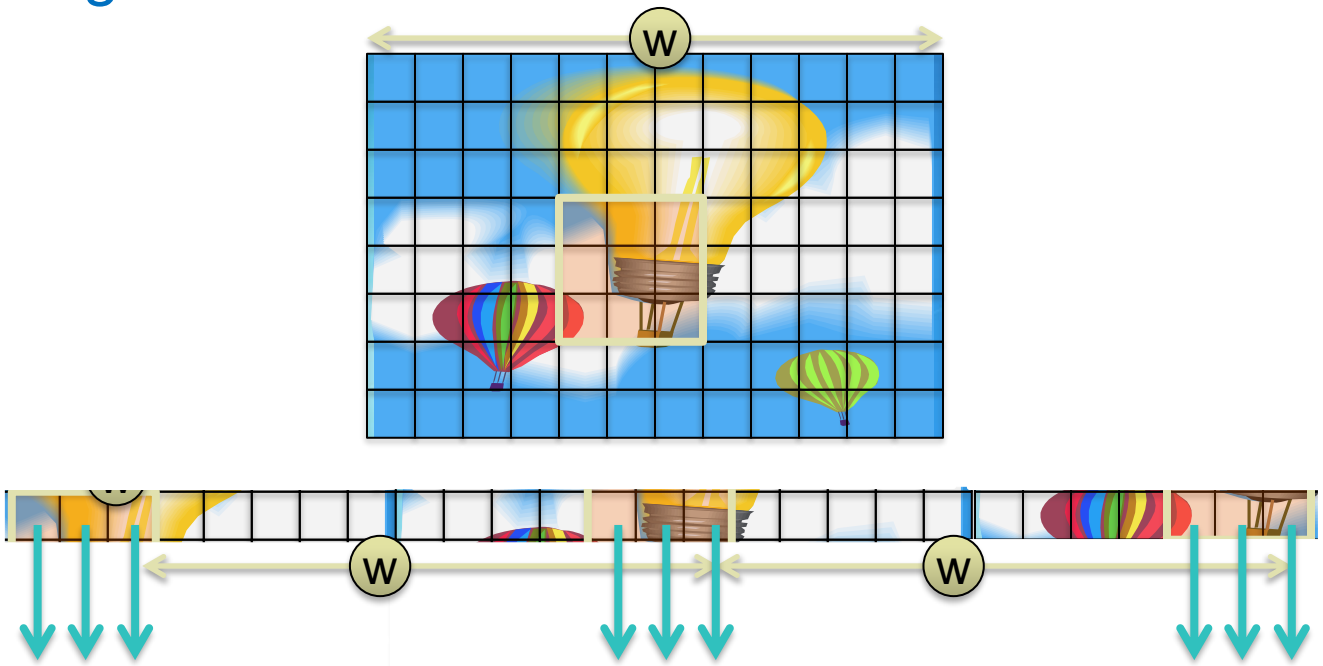
Optimizing the “Cache”



- The pixels at the edges that we don’t need for the computation

© 2015 Altera Corporation—Public

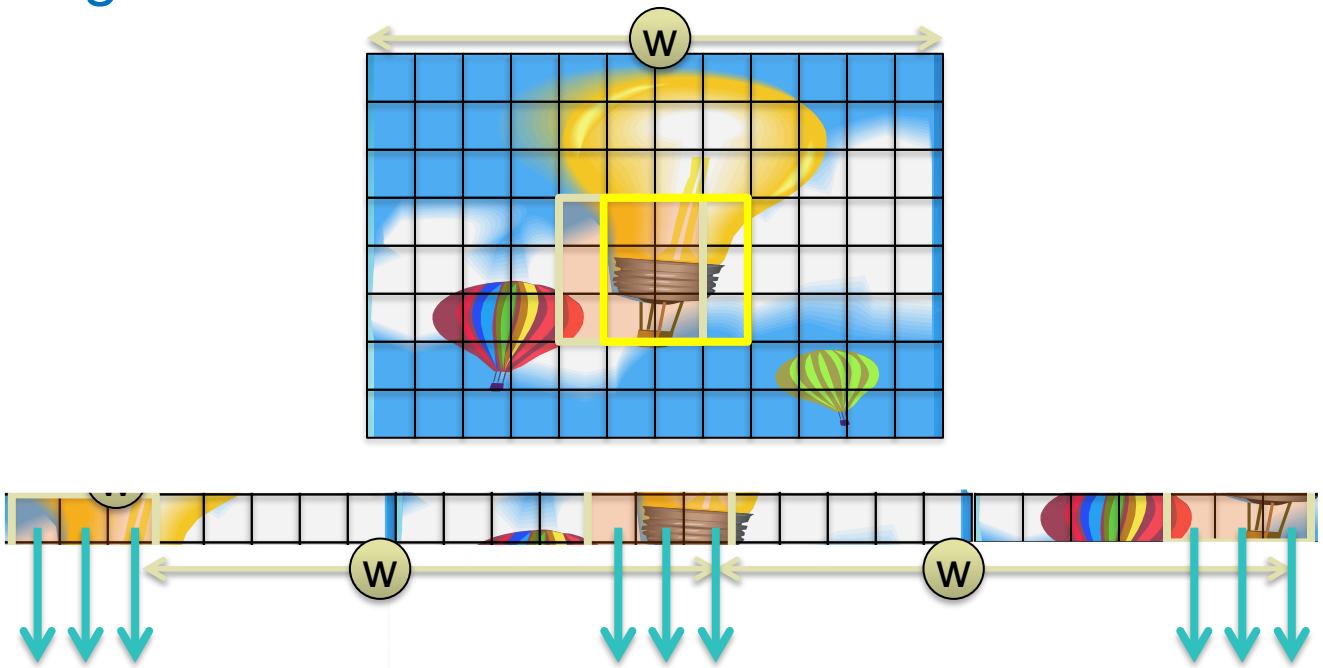
Optimizing the “Cache”



- What happens when we move the window one pixel to the right?

© 2015 Altera Corporation—Public

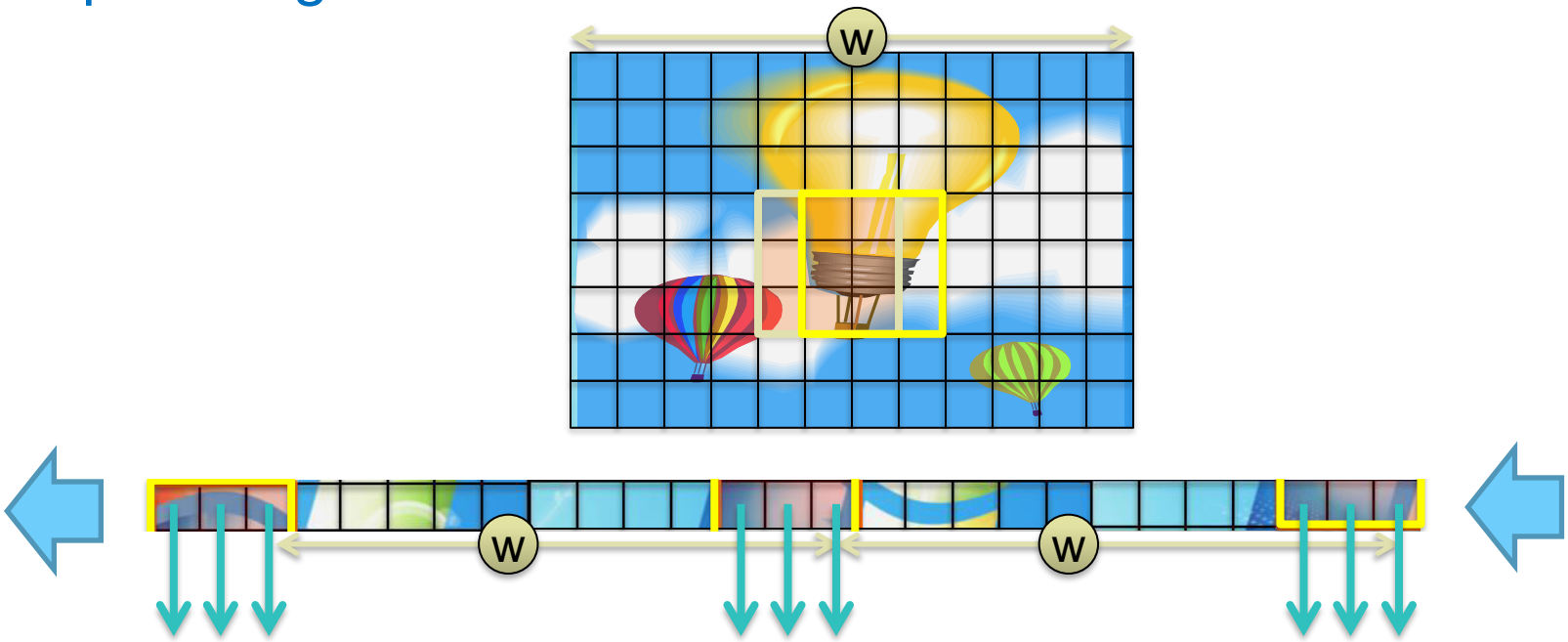
Optimizing the “Cache”



- What happens when we move the window one pixel to the right?

© 2015 Altera Corporation—Public

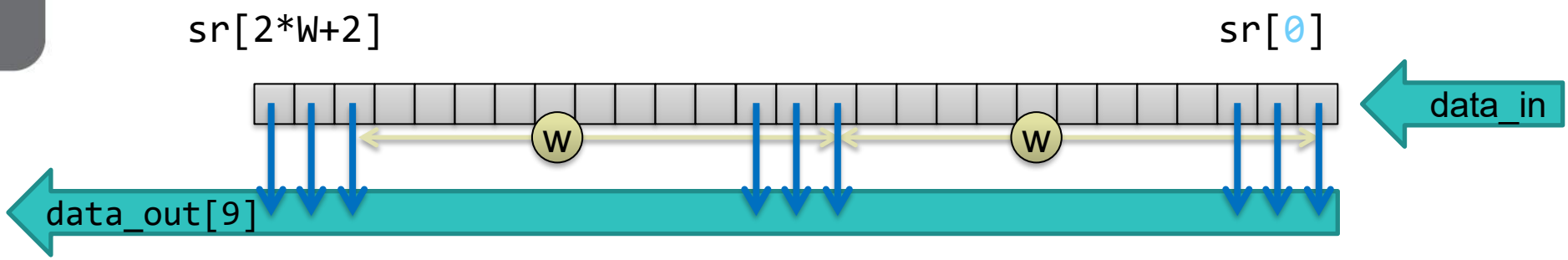
Optimizing the “Cache”



© 2015 Altera Corporation—Public

HZ: users need to have data flow or streaming flow in mind when coding in OpenCL for FPGAs

Shift Registers in software by leveraging *OpenCL tasks*




```
pixel_t sr[2*W+3];
while(keep_going) {
  // Shift data in
  #pragma unroll
  for(int i=1; i<2*W+3; ++i)
    sr[i] = sr[i-1]
  sr[0] = data_in;

  // Tap output data
  data_out = {sr[  0], sr[    1], sr[    2],
              sr[  w], sr[  w+1], sr[  w+2]
              sr[2*w], sr[2*w+1], sr[2*w+2]}
  // ...
}
```

Automatic loop pipelining

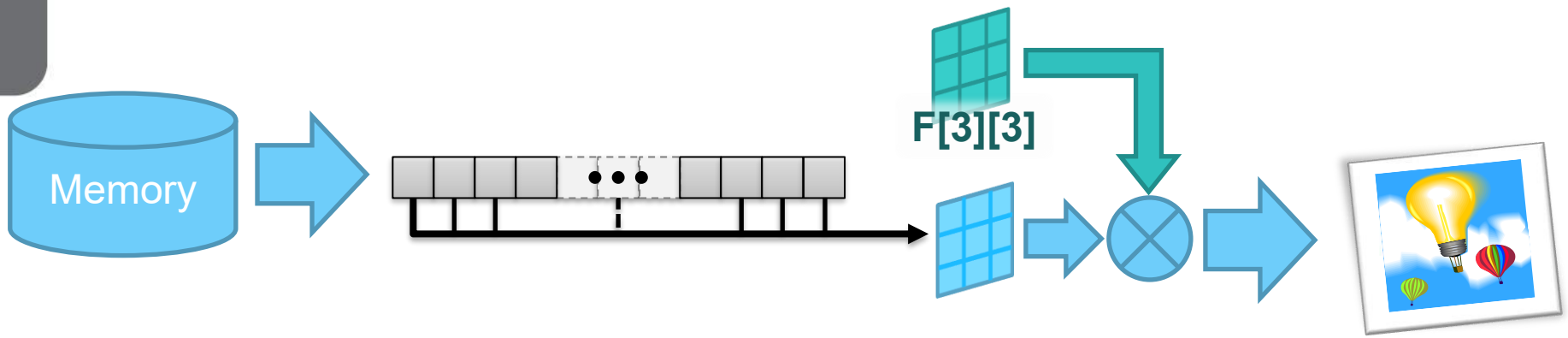
Shift register inference

OpenCL tasks:
Single-threaded kernels

Parallelism extracted automatically by the AOC compiler!

© 2015 Altera Corporation—Public

Data Flow Architecture using *Shift Registers*



- Optimal access to memory
- Takes advantage of FPGA architecture
  - Shift registers are very efficiently implemented in FPGAs
  - Data movement via flexible interconnect
- Low power, low cost, high performance

© 2015 Altera Corporation—**Public**

Kernel to Kernel Channels

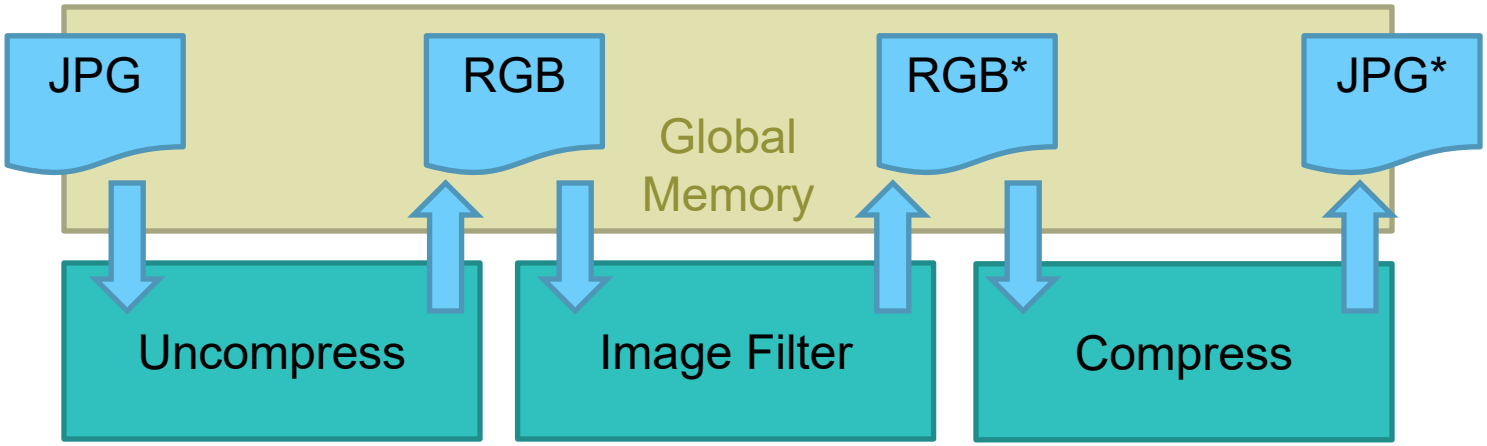
Harnessing Dataflow to Reduce Memory Bandwidth



© 2015 Altera Corporation—Public

OpenCL: Kernel-to-Kernel Communication

- **What can we do if two kernels need to communicate?**
  - Hand-off data from one kernel to another
  - Communicate some state or control information to another kernel
- **Communication between kernels in "vanilla" OpenCL:**
  - Data transfer between kernels can only happen through global memory
  - Global memory consistency is only guaranteed at kernel completion
- **Kernel-to-kernel communication consumes memory bandwidth and requires sequential execution of kernels**

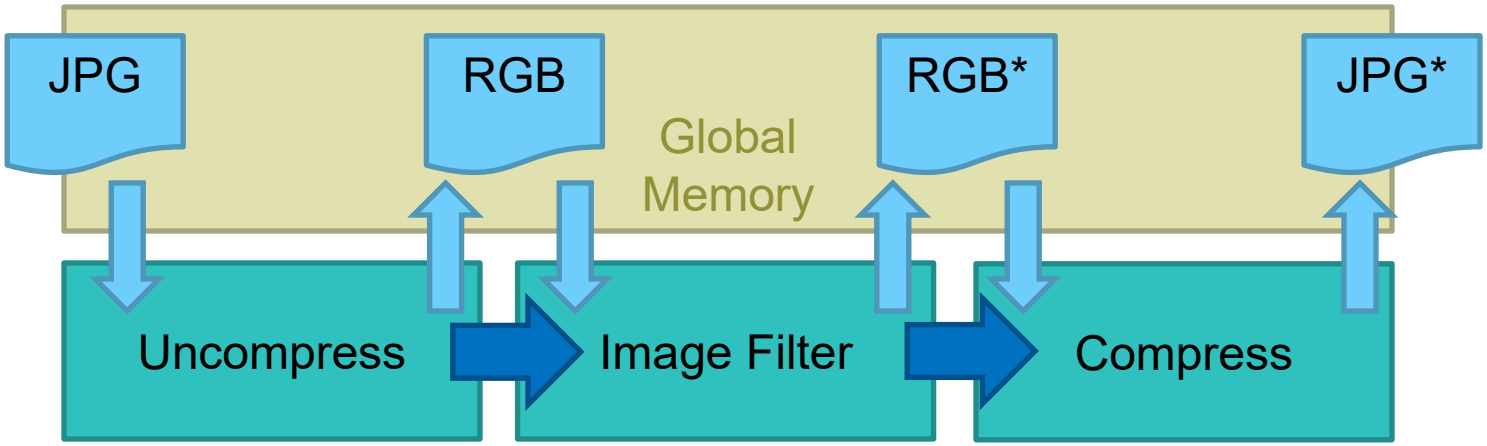


© 2015 Altera Corporation—Public

Altera Channels Extension: Kernel-to-Kernel

- **An FPGA has programmable routing**
- **Can't we just send data across wires between kernels?**
- **Advantages:**
  - Reduce memory bandwidth
  - Lower latency through fine-grained synchronization between kernels
  - Reduce complexity (wires are trivial compared to memory access)
    - Lower cost, lower area, higher performances
  - Enable modular dataflow design through small kernels exchanging data
  - Different workgroup sizes and degrees of parallelism in connected modules

HZ: users need to have data flow or streaming flow in mind when coding in OpenCL for FPGAs: Sliding window/shift registers seem to be the most importan pattern for FPGAs.



© 2015 Altera Corporation—Public

Use Model

- **A channel represents a FIFO or queue to store data**

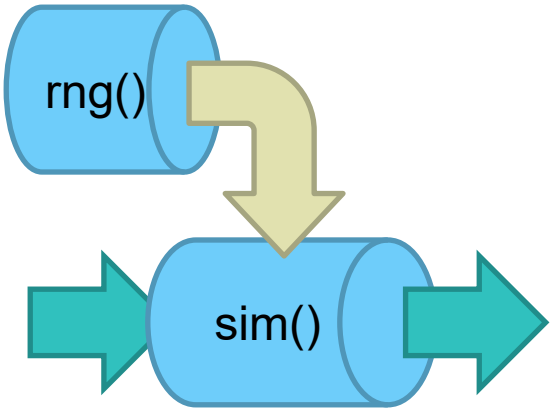
- **Channel declaration:**
```
channel int my_channel;
```
  - Create a queue:
```
value_type channel();
```
- **Channel write:**
```
write_channel_altera(my_channel, x);
```
  - Push data into the queue:
```
void write_channel_altera(channel &ch, value_type data);
```
- **Channel read:**
```
int y = read_channel_altera(my_channel);
```
  - Pop the first element from the queue
```
value_type read_channel_altera(channel &ch);
```

HZ: same as OpenCL pipes

© 2015 Altera Corporation—Public

Example 1: Multi-Stage Pipeline

- **An algorithm may be divided into multiple kernels:**
  - Modular design patterns
  - Partition the algorithm into kernels with different sizes and dimensions
  - Algorithm may naturally split into both single-threaded and NDRange kernels
- **Generating random data for a Monte Carlo simulation:**




```
kernel void rng(int seed) {
  int r = seed;
  while(true) {
    r = rand(r);
    write_channel_altera(
      RAND, r);
  }
}
```

Single-Threaded

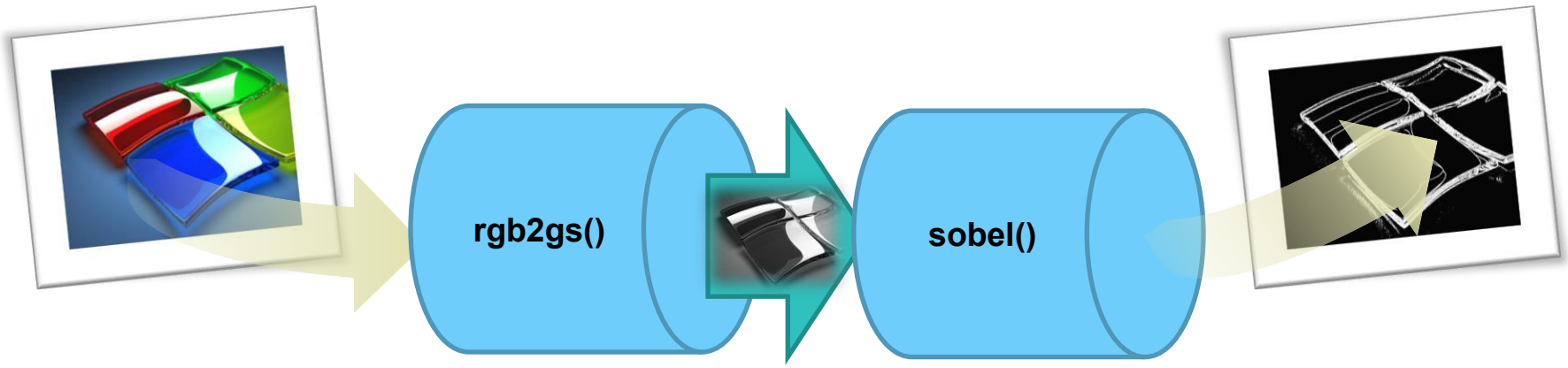
```
kernel void sim(...) {
  int gid = get_global_id(0);
  int rnd = read_channel_altera(
              RAND);
  out[gid] = do_sim(data, rnd);
}
```

NDRange

© 2015 Altera Corporation—Public

Example 2: Modular Design

- **Eg. Use the greyscale Sobel filter module to work on colour images**
- **We'll introduce a colourspace transformation kernel in the datapath to transform the data into greyscale**



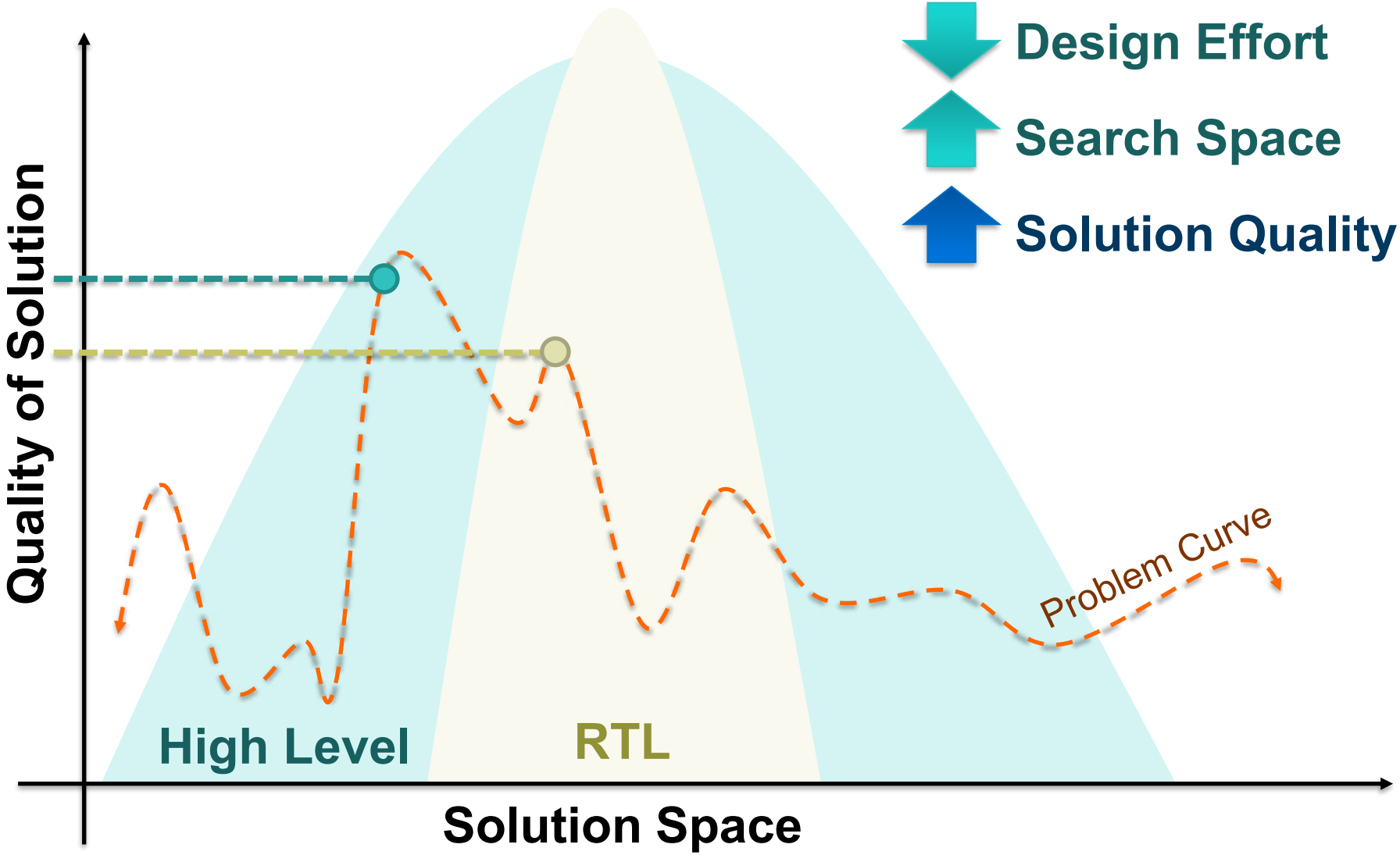
© 2015 Altera Corporation—Public

Case Study : GZIP Compression



© 2015 Altera Corporation—Public

Abstraction Yields Quality



© 2015 Altera Corporation—Public

Big Data →→→ Big Value

to enterprise and society

44x

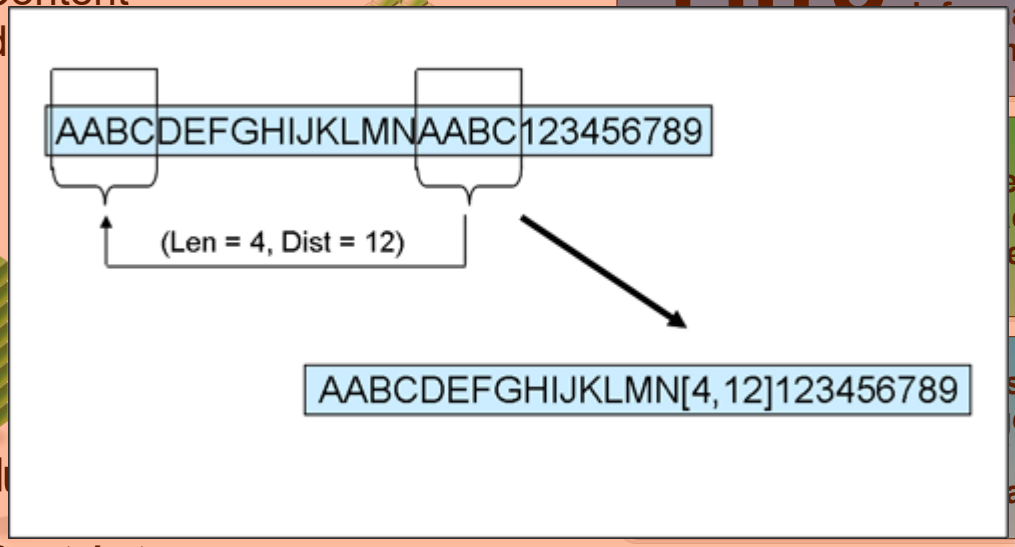
as much Data and Content Over Coming Decad

Kilobyte (kB)	1,000 Bytes
Megabyte (MB)	1,000 Kilobytes
Gigabyte (GB)	1,000 Megabytes
Terabyte (TB)	1,000 Gigabytes
Petabyte (PB)	1,000 Terabytes
Exabyte (EB)	1,000 Petabytes
Zettabyte (ZB)	1,000 Exabytes

2020
35 zettabytes

Vol

2009
800,000 petabytes





Of world's data is **unstructured**

1 in 3 Business leaders frequently make decisions based on ation they don't trust, or ave

ss leaders say they don't ccess to the information eed to do their jobs

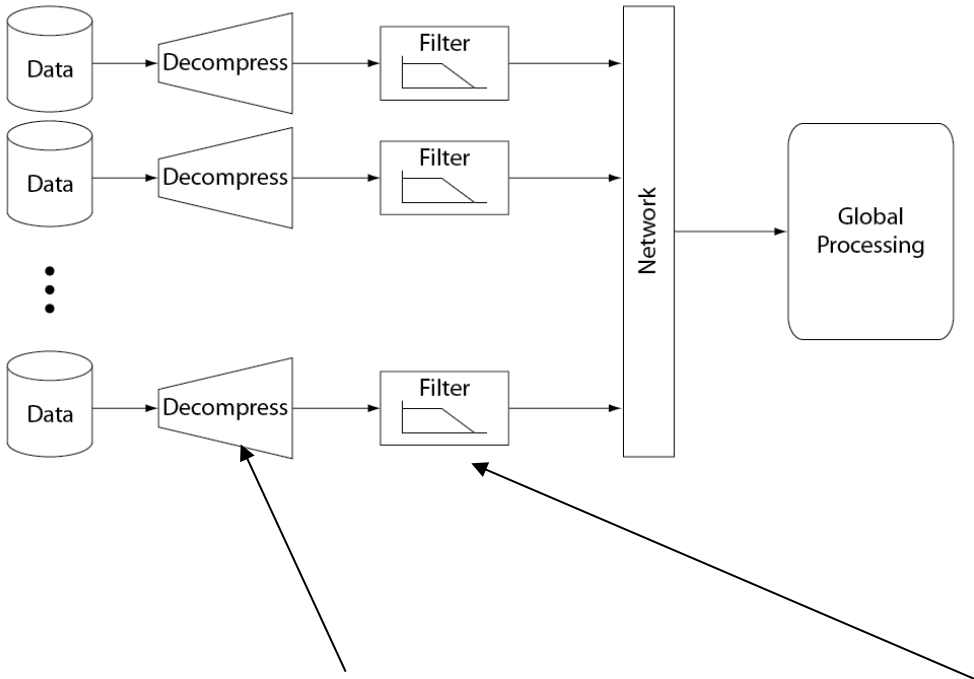
s cited "Business ence and analytics" as their visionary plans ance competitiveness

60% of CEOs need to do a better job capturing and understanding information rapidly in order to make swift business decisions

The resulting explosion of information (plus intermediate data) creates a need for a new kind of intelligence

© 2015 Altera Corporation—Public

Differentiation in Big Data Analytics
Acceleration through FPGAs





Dictionary based **stop word filtering** in FPGA on a representative dataset with blogs, boards, and news content

	Data Size	SystemT Runtime
Original	**1**	**1**
After filtering (100 word dictionary)	**0.75**	**0.69**
After filtering (1000 word dictionary)	**0.61**	**0.58**

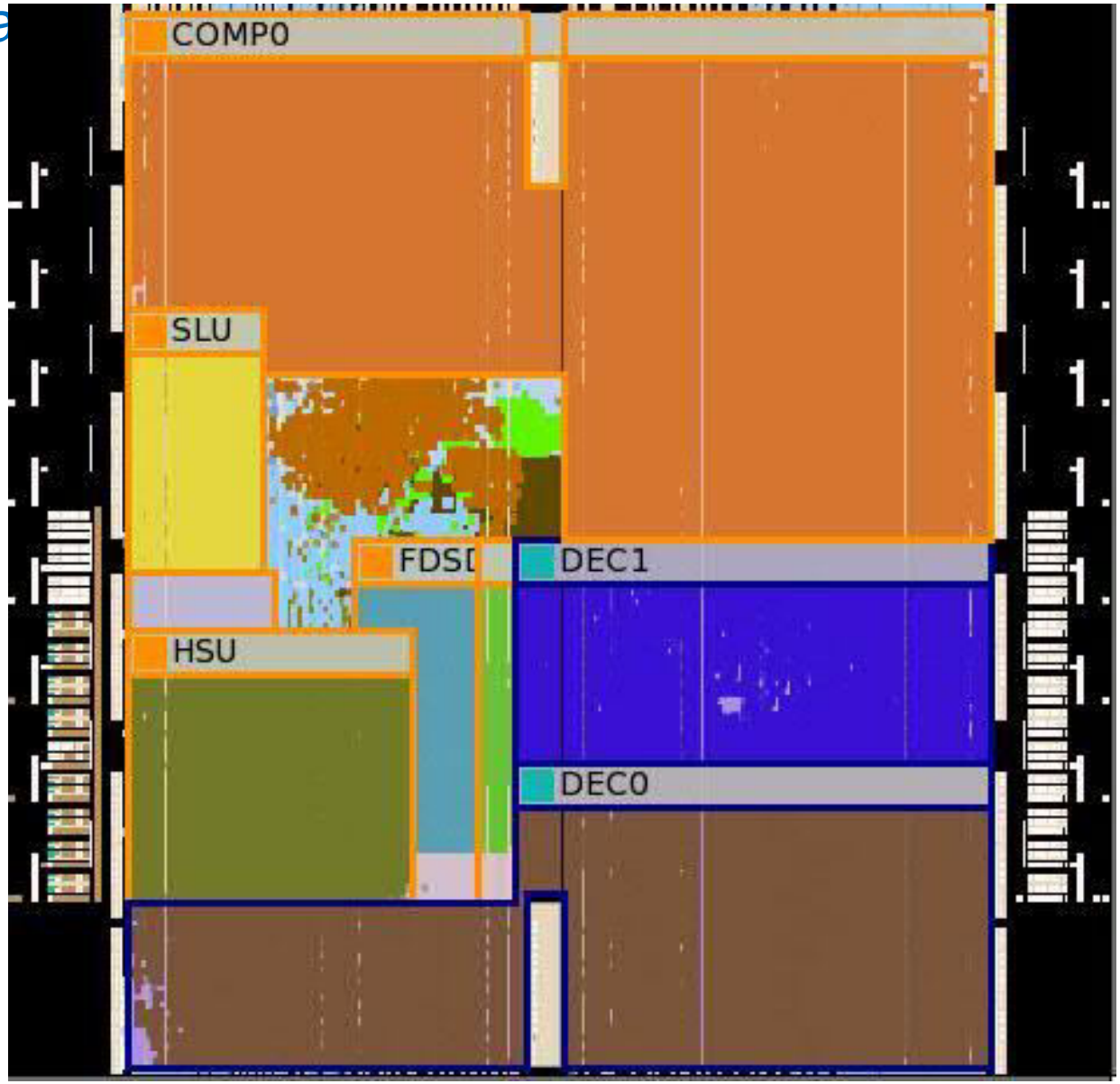
World's fastest gzip
- 4GB/s on a single stream
- 2-3X better than leading external vendor
- Equivalent of ~12 P7 cores
- 3-5x compression on big data text

Net result: Significant increase in capacity and throughput

© 2015 Altera Corporation—Public

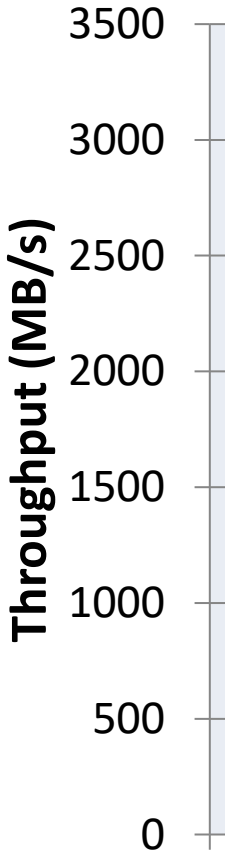
ALTERA

Chip ima



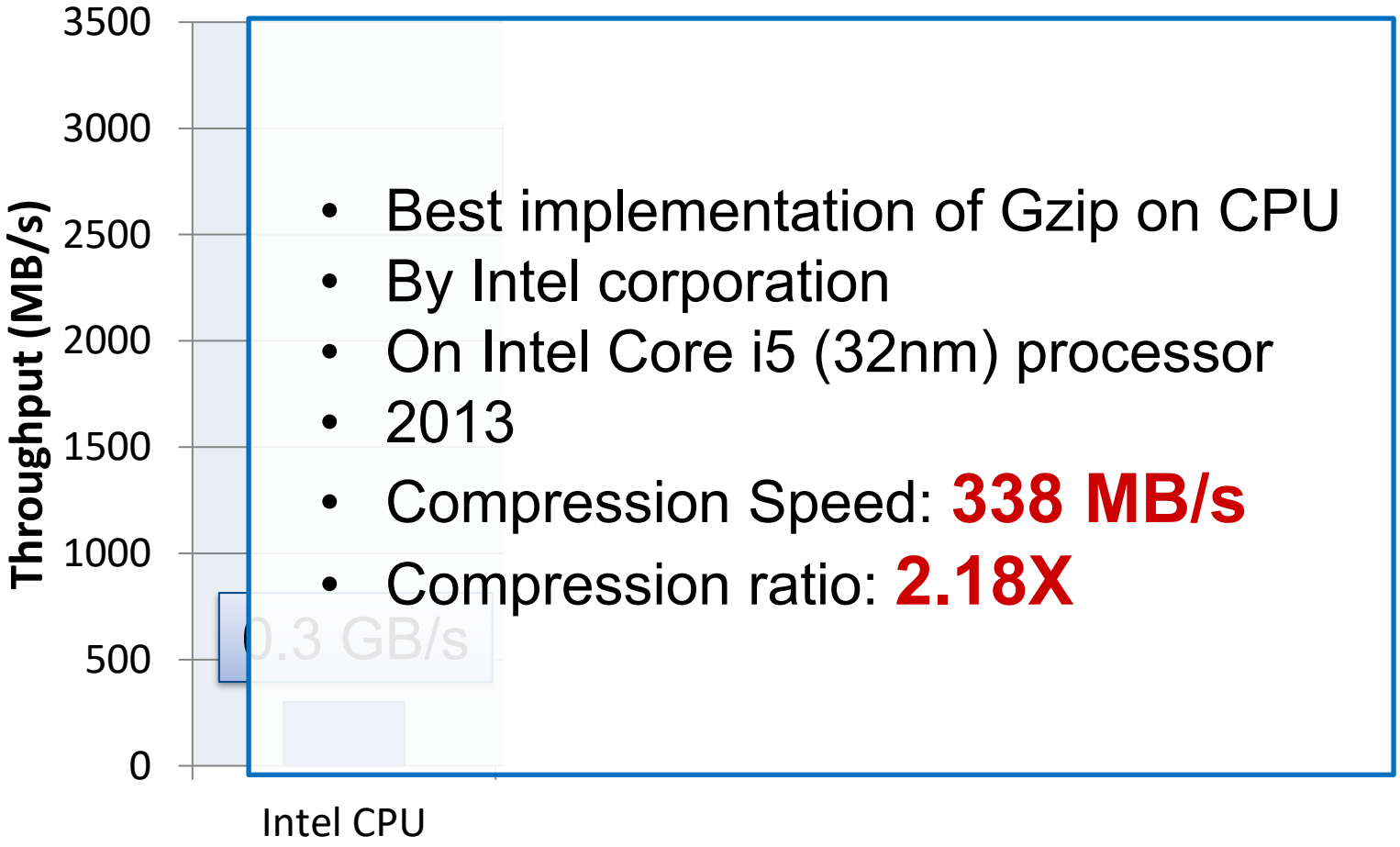
© 2015 Altera Corporation—Public

Comparison against CPU/Verilog



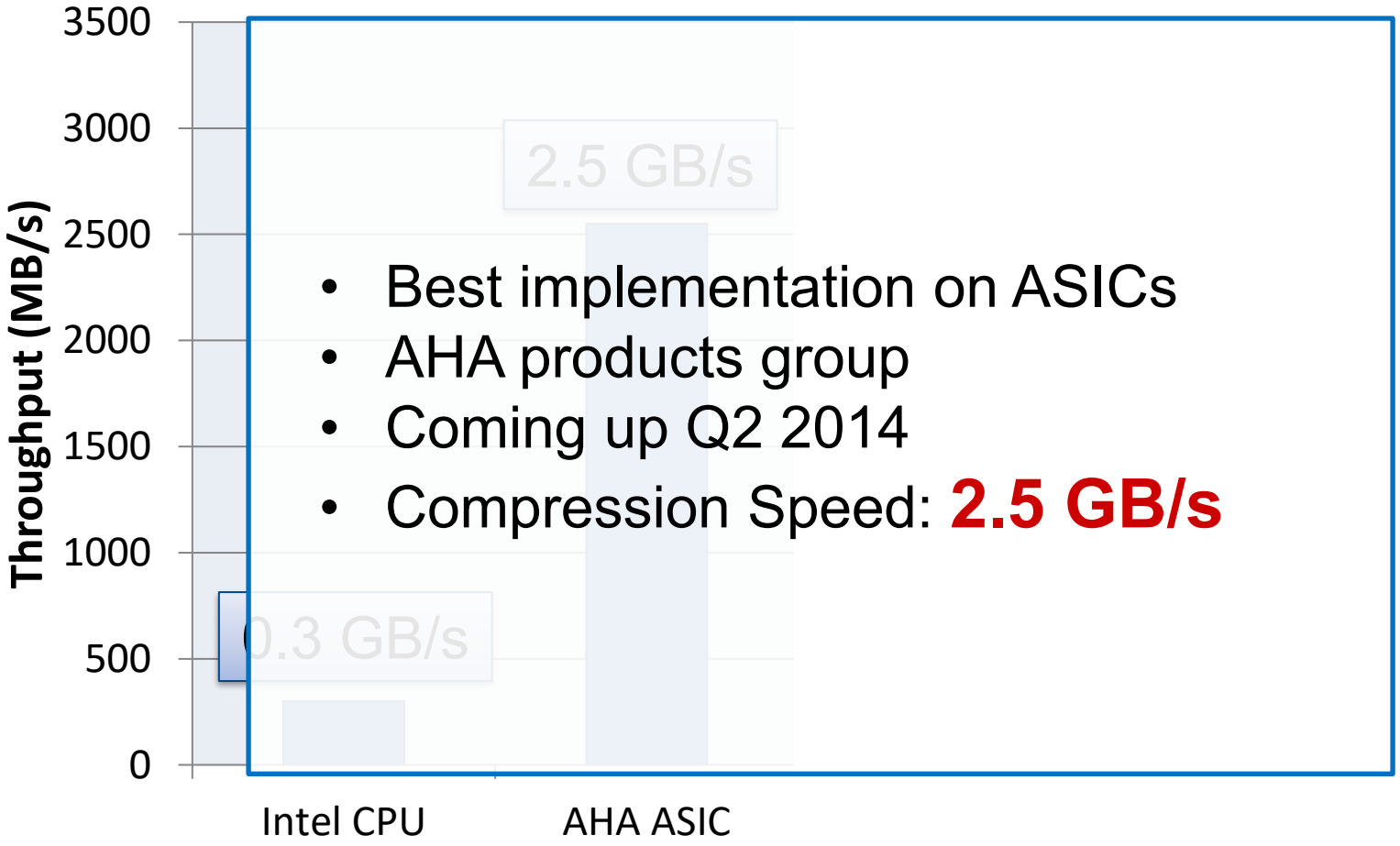
© 2015 Altera Corporation—Public

Comparison against CPU/Verilog



© 2015 Altera Corporation—Public

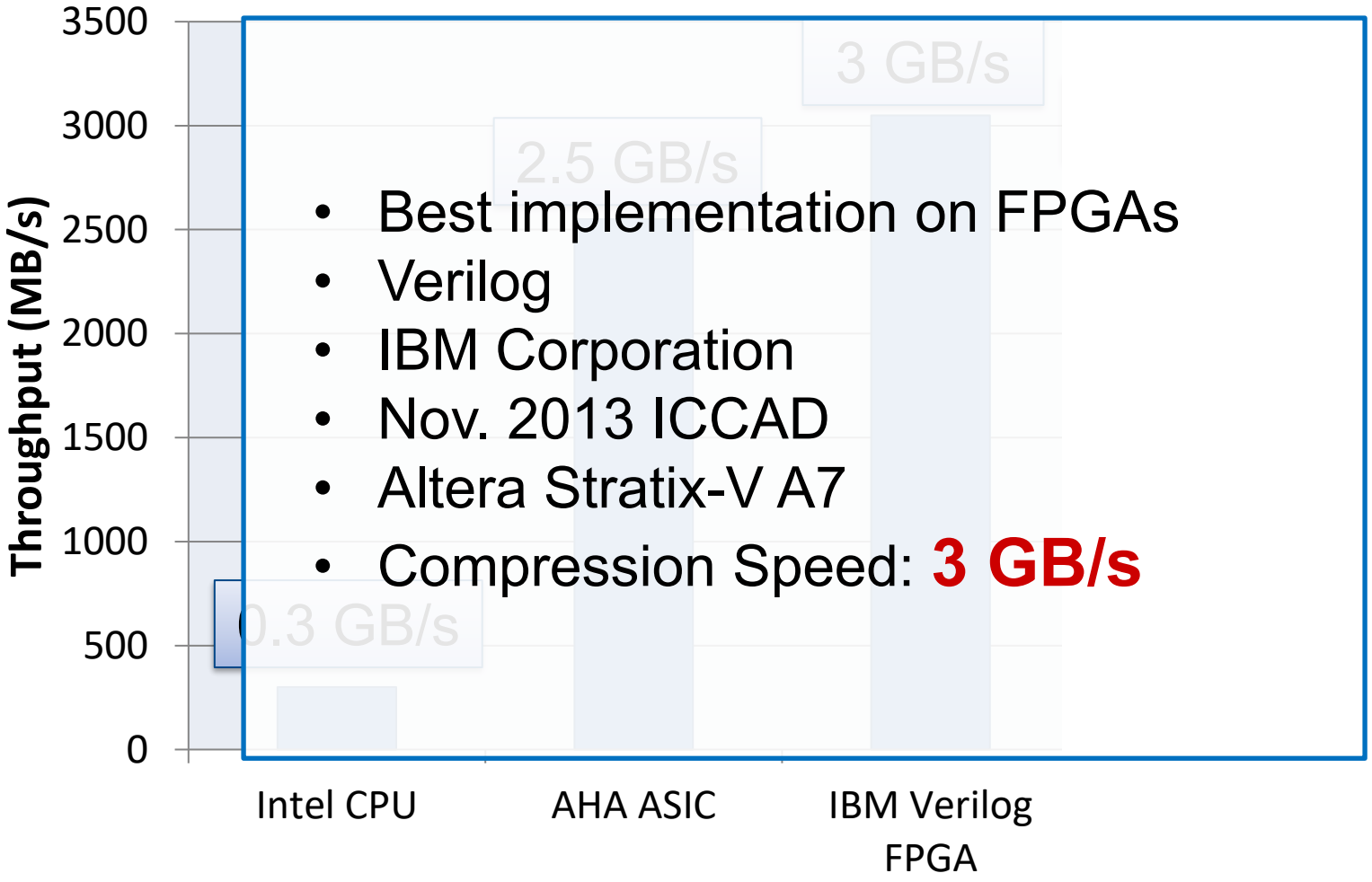
Comparison against CPU/Verilog



- Best implementation on ASICs
- AHA products group
- Coming up Q2 2014
- Compression Speed: **2.5 GB/s**

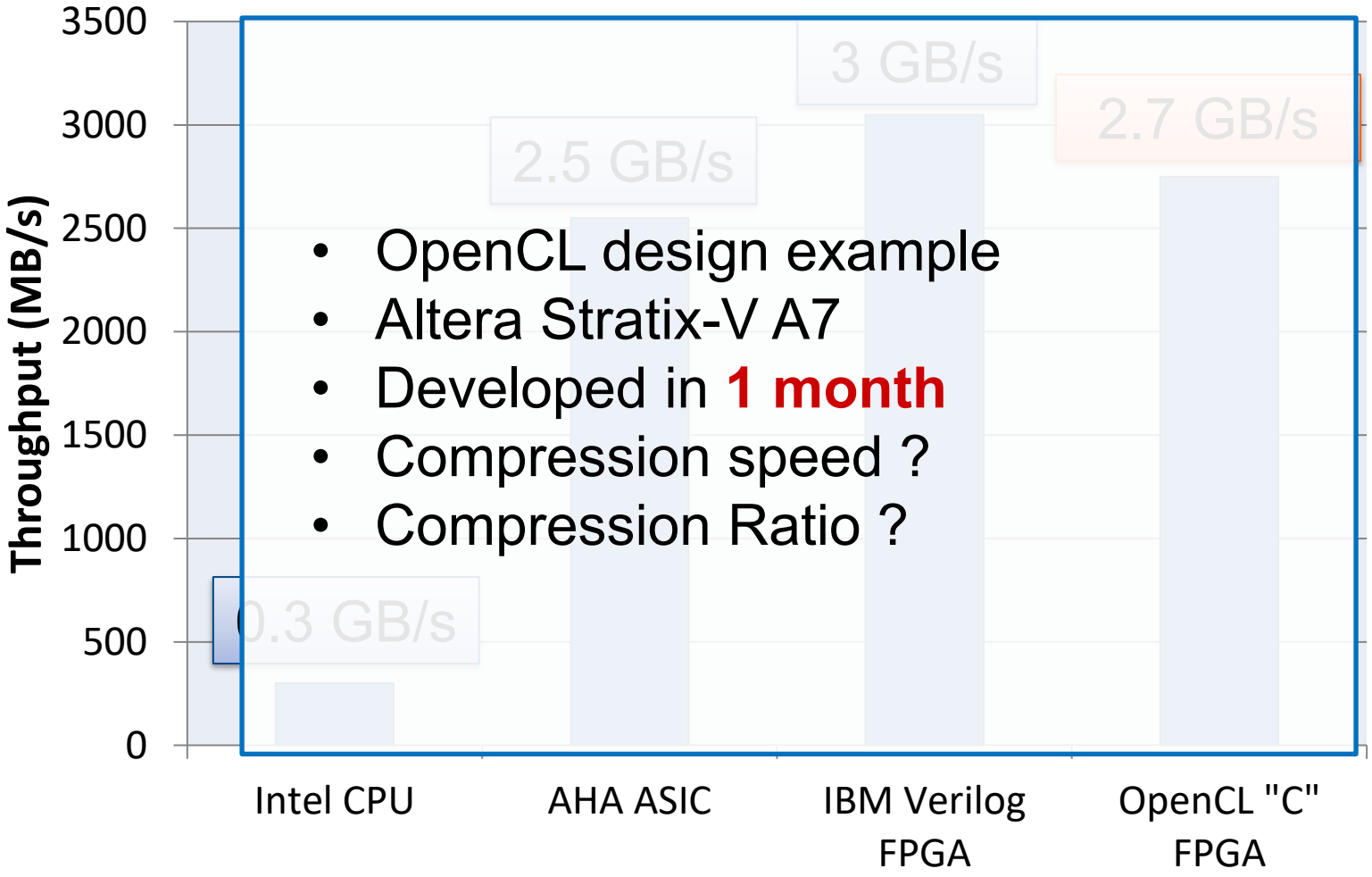
© 2015 Altera Corporation—Public

Comparison against CPU/Verilog



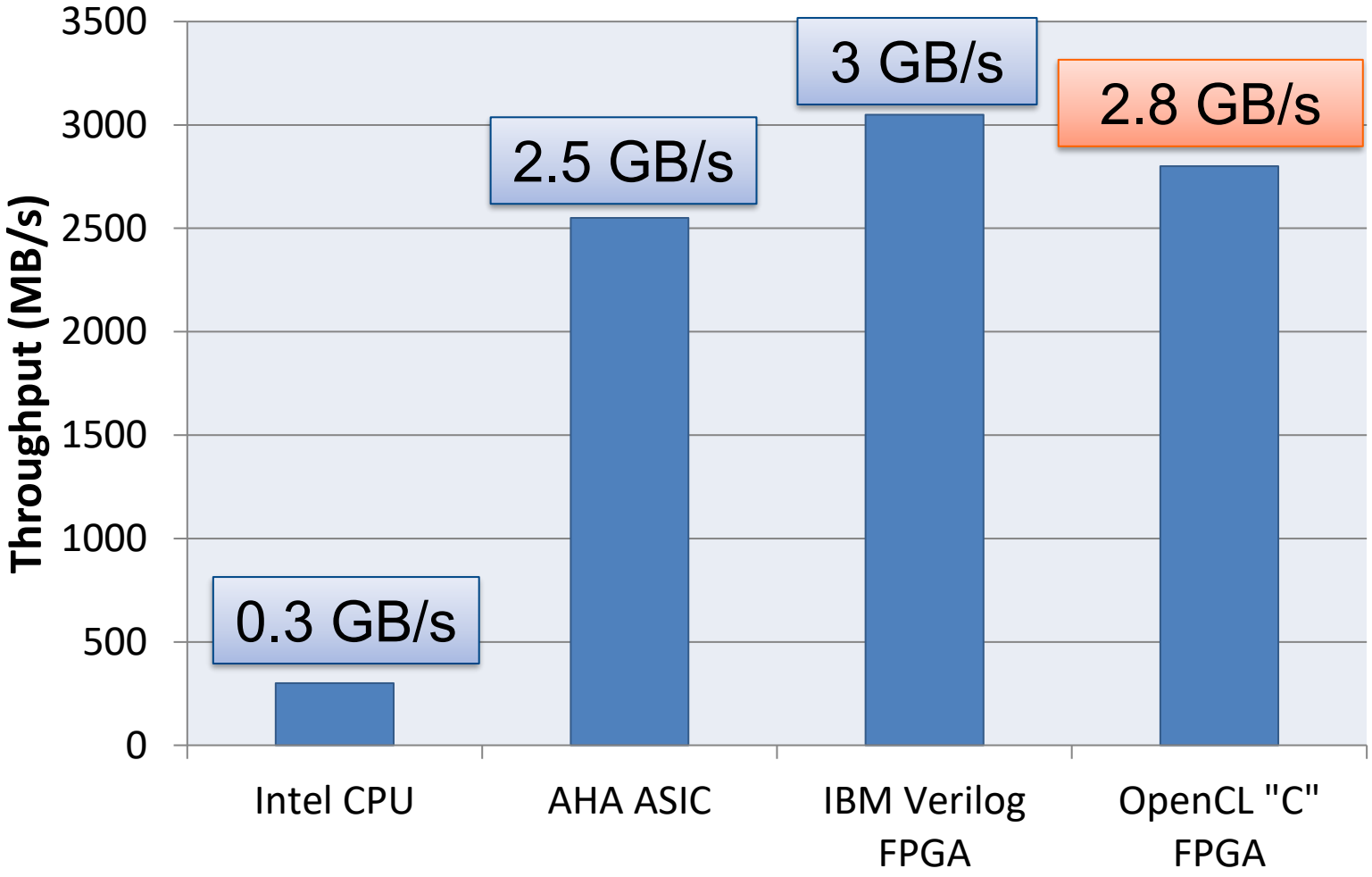
© 2015 Altera Corporation—Public

Comparison against CPU/Verilog



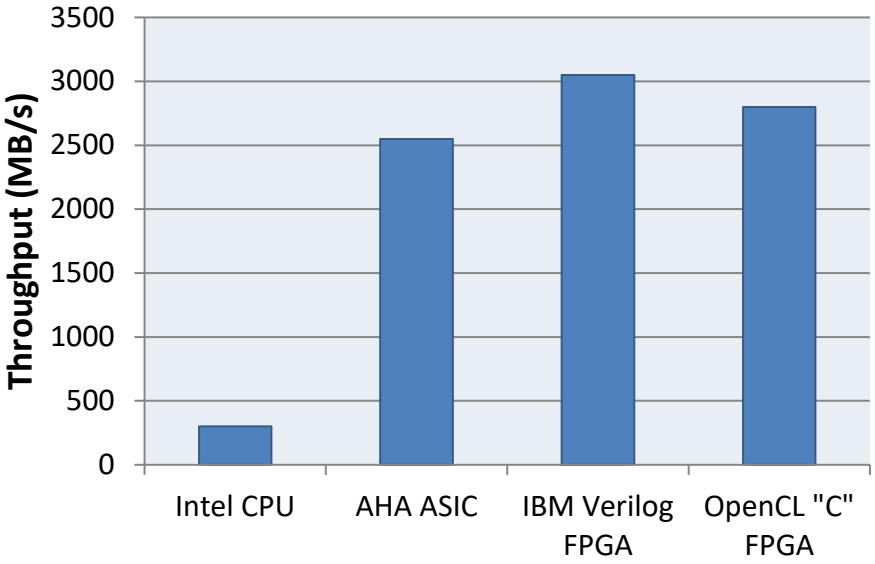
© 2015 Altera Corporation—Public

Comparison against CPU/Verilog



© 2015 Altera Corporation—Public

Comparison against CPU



Same compression ratio

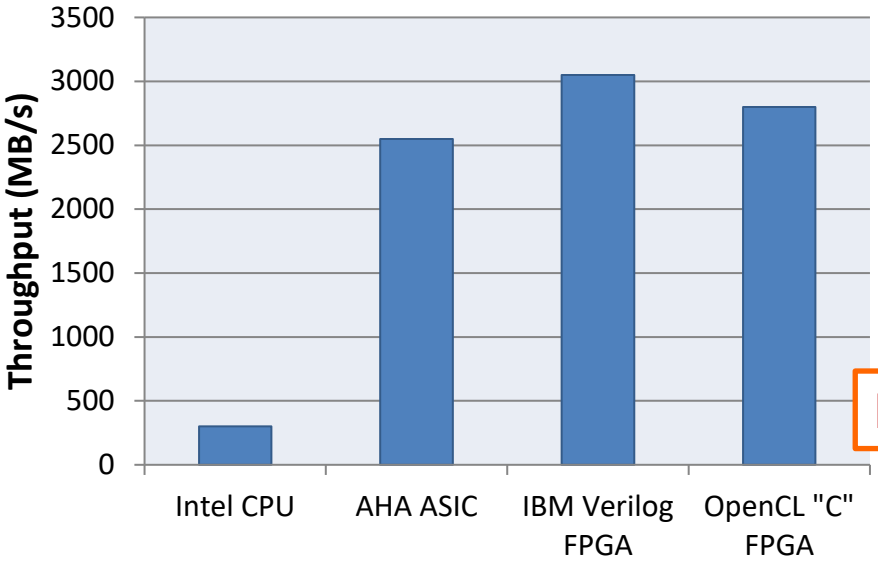
12X better performance/Watt

TABLE I: Comparison between our OpenCL FPGA and the best CPU implementation of Gzip.

	Performance	Performance/Watt	Compression Ratio
OpenCL FPGA	2.71 GB/s	111 MB/J	2.17×
Intel Gzip	338 MB/s	9.26 MB/J	2.18×
Gap	8.2× faster	12× better	on par

© 2015 Altera Corporation—Public

Comparison against Verilog



6% Slower

10% more resources

Much lower design effort and design time

1-Intern month of development time vs.

multiple person years

© 2015 Altera Corporation—Public

Thank You



© 2015 Altera Corporation—Public